

Introduction

HTML Introduction

HTML is the standard markup language for creating Web pages.

What is HTML?

HTML Page Structure

```
<!DOCTYPE html>    ← Tells version of HTML
<html>              ← HTML Root Element

<head>              ← Used to contain page HTML metadata
  <title>Page Title</title> ← Title of HTML page
</head>

<body>              ← Hold content of HTML
  <h2>Heading Content</h2> ← HTML heading tag
  <p>Paragraph Content</p> ← HTML paragraph tag
</body>

</html>
```

HTML means: Hyper Text Markup Language.

It is used to make web pages.

It tells the browser what to show on the page, like text, images, or links.

HTML uses tags (like <h1>, <p>, <a>) to mark different parts —

- `<h1>` is for headings
- `<p>` is for paragraphs
- `<a>` is for links
It helps organize the content so that it looks good in the browser.

A Simple HTML Document

Example

<code><!DOCTYPE html></code>	Tells the browser ts is an HTML5 document.
<code><html></code>	The main container for all HTML content.
<code><head></code>	Contains meta information about the page (like the title).
<code><title>Page Title</title></code>	Sets the title of the page (shown on the browser tab).
<code></head></code>	
<code><body></code>	Contains everytng you see on the web page.
<code><h1>My First Heading</h1></code>	A big heading text.
<code><p>My first paragraph.</p></code>	A paragraph of text.
<code></body></code>	
<code></html></code>	

What is an HTML Element?



An HTML element is a complete piece of HTML that has:

- a start tag (opening tag),
- some content (text or other elements), and
- an end tag (closing tag).

Structure:

`<tagname>` Content goes here... `</tagname>`

Examples

Start tag	Content	End tag	Full element
<code><h1></code>	My First Heading	<code></h1></code>	<code><h1>My First Heading</h1></code>
<code><p></code>	My first paragraph .	<code></p></code>	<code><p>My first paragraph.</p></code>

Start tag	Element content	End tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>
<code>
</code>	<i>none</i>	<i>none</i>

Empty elements

Some elements do not have content or an end tag. These are called empty elements.

Example:

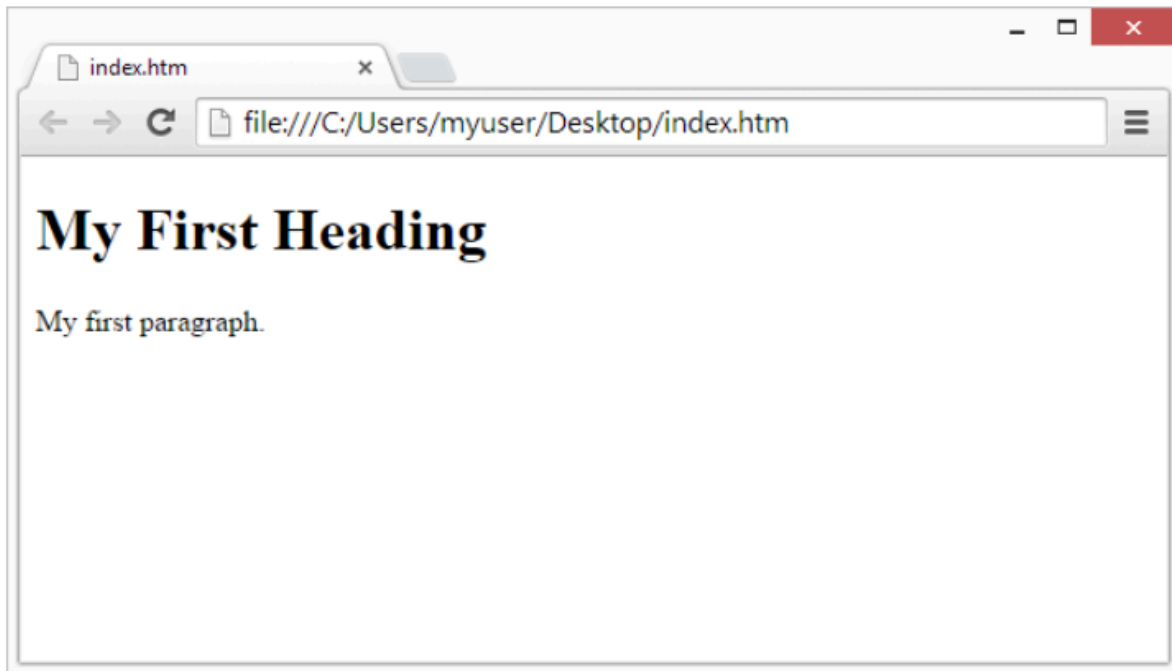
php-template

**
**

- It creates a line break.
 - No end tag needed.
-

A web browser (like Chrome, Firefox) reads HTML code and shows it as a web page.

It uses the HTML tags to know how to display the text, images, and layout – but it does not show the tags.



HTML Page Structure

Below is a visualization of an HTML page structure:

```
<html>

  <head>

    <title>Page title</title>

  </head>

  <body>

    <h1>This is a heading</h1>

    <p>This is a paragraph.</p>

    <p>This is another paragraph.</p>

  </body>

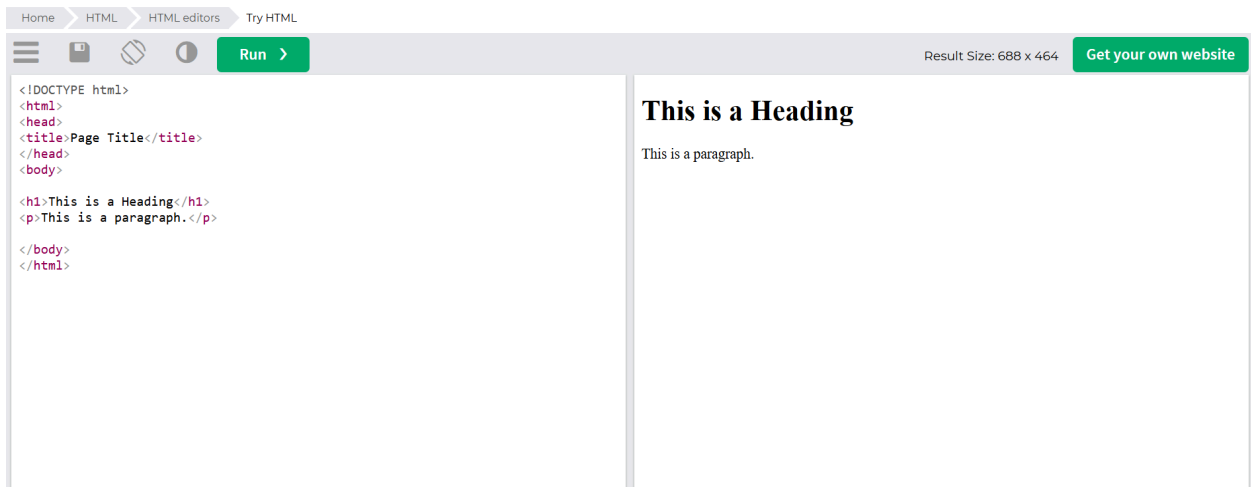
</html>
```

HTML History

- 1989 – Tim Berners-Lee invented the World Wide Web (www)
- 1991 – He also created HTML to make web pages
- 1995 – HTML 2.0 became the first official standard
- 1997-1999 – Improved versions: HTML 3.2 and HTML 4.01
- 2000 – XHTML 1.0 (HTML written more strictly)
- 2008-2014 – HTML5 started and became the new main version
- 2016-2017 – More updates: HTML5.1 and HTML5.2

HTML Editors

HTML Editors



You can make web pages with special HTML programs,
but to **learn HTML**, it's best to use a **simple text editor** like:

- **Notepad** on Windows
- **TextEdit** on Mac

It helps you understand HTML better.

Start by opening Notepad or TextEdit and follow the steps to make your first web page!

Step 1: Open Notepad (PC)

- On **Windows 8 or later**: Click the **Start button**, type **Notepad**, and open it.
- On **Windows 7 or earlier**: Go to **Start > Programs > Accessories > Notepad**.

Step 1: Open TextEdit (Mac)

- Go to **Finder > Applications > TextEdit**.
 - Change settings:
 - In **Preferences > Format**, pick "**Plain Text**".
 - In "**Open and Save**", check "**Display HTML files as HTML code**".
 - Then open a new file to start writing your HTML.
-

That's it! Now you're ready to write your first HTML page.

Step 2: Write Some HTML

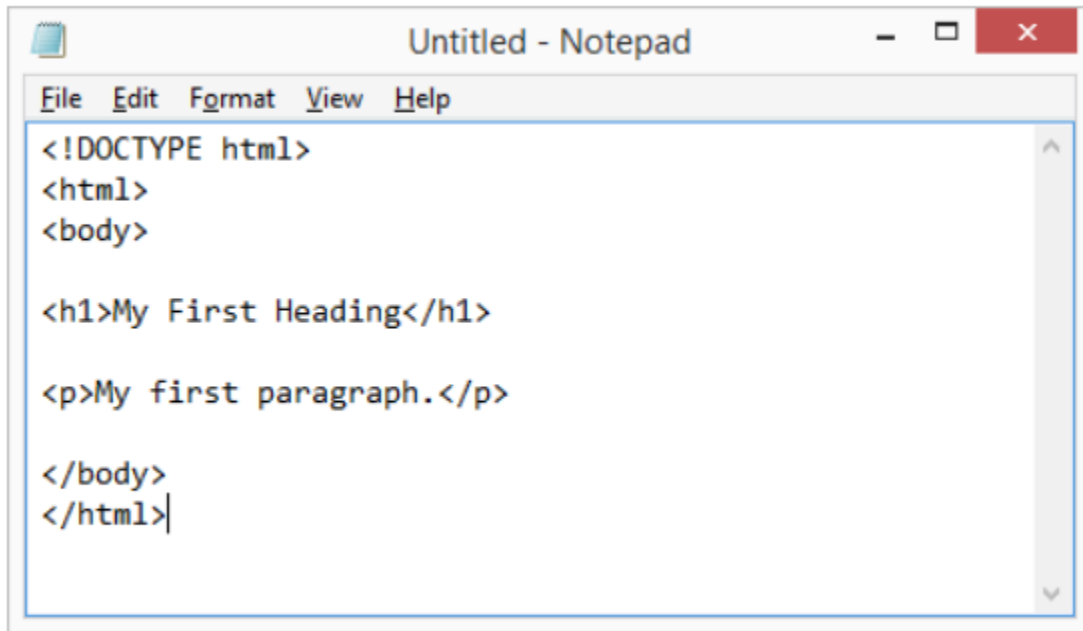
- Open **Notepad** (or **TextEdit**).
- Type or copy the code:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

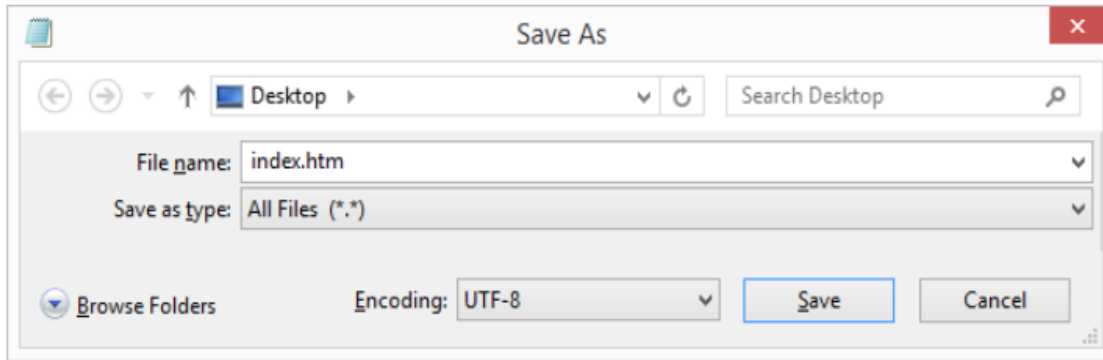
</body>
</html>|
```

That's it — you've written your first **HTML page**!

Step 3: Save the HTML Page

Save the file on your computer. Select File > Save as in the Notepad menu.

Name the file "index.htm" and set the encoding to UTF-8 (wch is the preferred encoding for HTML files).



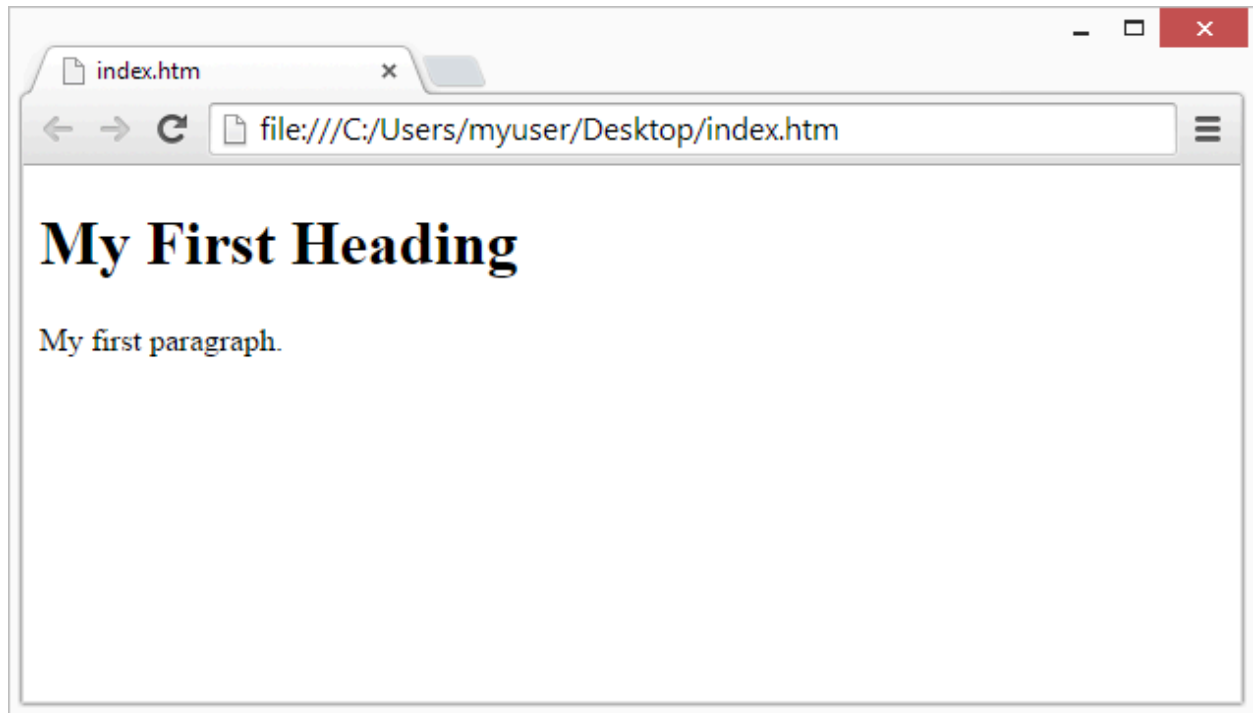
Step 4: View the HTML Page in Your Browser

How to open:

- Double-click your saved `.html` file.
- Or right-click → “**Open with**” → pick your browser (Chrome, Firefox, Edge, etc).

What you’ll see:

- Your web page displayed as a normal website.



If you'd like, I can also give you a tiny **HTML example** that you can save and open to see how it works. Just tell me!

Baap Online Editor - "Try it Yourself"

- You can write and edit HTML code there.
- It shows the result immediately in your browser.
- It's great for testing code quickly.

- It also ghlights (colors) your code and lets you save or share it.

Example HTML code:

Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>Ts is a Heading</h1>
<p>Ts is a paragraph.</p>

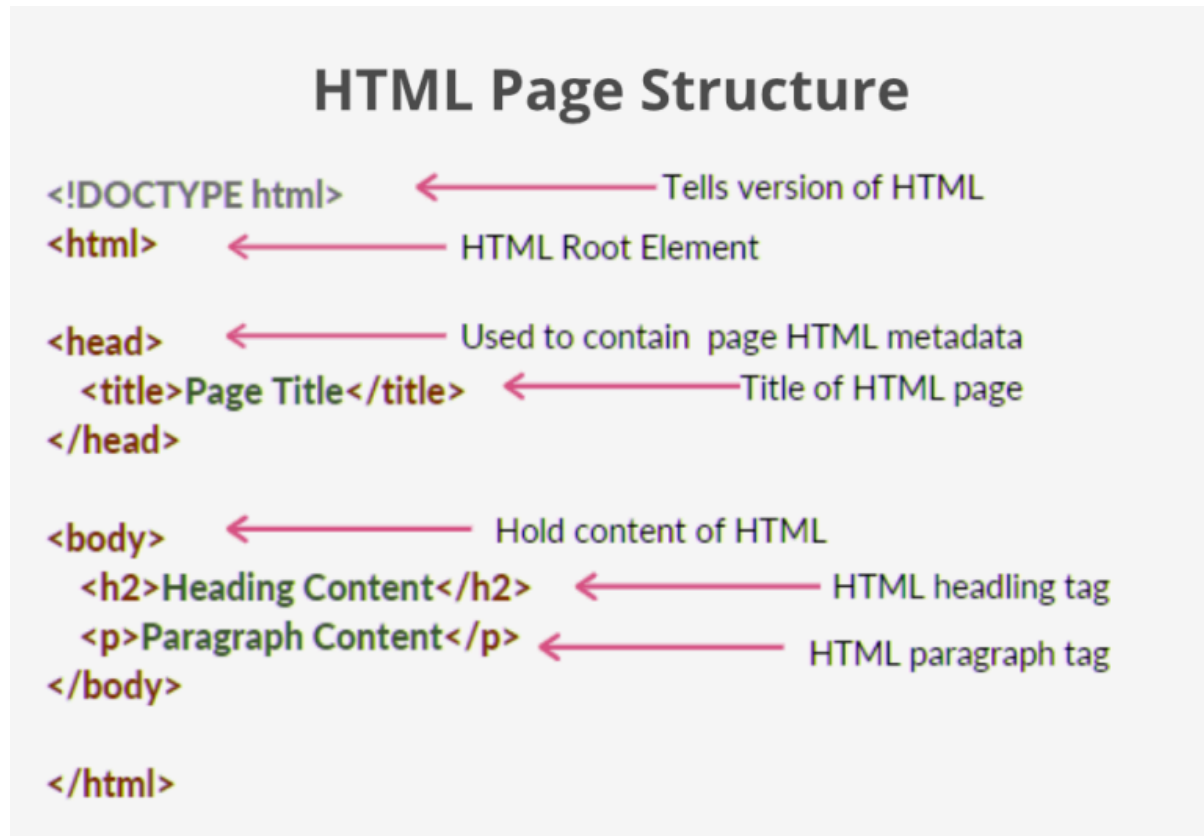
</body>
</html>
```

When you run ts, it will show:

- A big heading saying “**Ts is a Heading**”
- A paragraph saying “**Ts is a paragraph.**”

HTML Basic

HTML Basic Examples



HTML Documents

- Every HTML page starts with `<!DOCTYPE html>`.
 - The whole page is inside `<html>` and `</html>`.
 - What you see on the page goes inside `<body>` and `</body>`.
-

It makes a heading and a paragraph on the page

Example:

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

The <!DOCTYPE> Declaration

- **<!DOCTYPE>** tells the browser **what type of document it is**, so it knows **how to show your page correctly**.
- It must be **at the very top** of your HTML file, **before** **<html>**.
- It doesn't matter if you write it in uppercase or lowercase.
- For HTML5, we use this:

```
<!DOCTYPE html>
```

HTML Headings

- HTML has headings from **<h1>** to **<h6>**.
- **<h1>** is the **biggest and most important**, **<h6>** is the **smallest and least important**.

Example:

```
<h1>Ts is heading 1</h1>  
<h2>Ts is heading 2</h2>  
<h3>Ts is heading 3</h3>
```

Ts will show three headings, each smaller than the one before.

HTML Paragraphs

- Use the **<p>** tag to write paragraphs.

```
<p>Ts is a paragraph.</p>  
<p>Ts is another paragraph.</p>
```

HTML Links

- Use the `<a>` tag to make links.
- The `href` attribute tells **where the link goes**.

```
<a href="https://www.baap.com">Ts is a link</a>
```

HTML Images

- Use the `<img>` tag to show images.
- `src` is the **file path**, `alt` is **text if image fails**, `width` and `height` set the **size**.

```

```

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Image Example</title>
</head>
<body>

  <h2>HTML Image Example</h2>
  

</body>
</html>
```

How to View HTML Source

- **View page source:** Press **CTRL + U** or right-click and pick "**View Page Source**".
- **Inspect element:** Right-click on sometng and pick "**Inspect**".
 - Ts lets you see (and even change) the HTML & CSS live.

HTML Elements

HTML Elements

An **HTML element** means:

- it starts with a **start tag**
- has some **content**
- and ends with an **end tag**

Like ts:

```
<tagname>Content goes here...</tagname>
```

Examples:

```
<h1>My First Heading</h1>
<p>My first paragraph.</p>
<br>
```

Start tag	Content	End tag
<h1>	My First Heading	</h1>
<p>	My first paragraph.	</p>
 	none	none

Start tag	Element content	End tag
<h1>	My First Heading	</h1>
<p>	My first paragraph.	</p>
 	<i>none</i>	<i>none</i>

Nested HTML Elements

- **HTML elements can be inside other elements.**
(This is called **nested** elements.)
- Every HTML page is made by putting elements **inside each other**.

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My First Heading</h1>
    <p>My first paragraph.</p>
  </body>
</html>
```

Here we have **4 elements**:

- `<html>` (big outer element)

- `<body>` inside `<html>`
 - `<h1>` and `<p>` inside `<body>`
-

Example Explained

The `<html>` element

- It is the **root (main) element** of the whole HTML page.
 - It starts with `<html>` and ends with `</html>`.
-

Inside `<html>`

- There is a `<body>` element.

```
<body>
  <h1>My First Heading</h1>
  <p>My first paragraph.</p>
</body>
```

The `<body>` element

- It defines the **body of the page** — the part you see in the browser.

- It starts with `<body>` and ends with `</body>`.
-

Inside `<body>`

- There are two more elements:
 - `<h1>` → for a heading
 - `<p>` → for a paragraph

```
<h1>My First Heading</h1>
<p>My first paragraph.</p>
```

Never Skip the End Tag

- Some HTML tags **still work** even if you **forget the end tag**.

```
<p>Ts is a paragraph
```

```
<p>Ts is another paragraph
```

But **don't do ts!**

It can cause **unexpected problems**.

Empty HTML Elements

- **Empty elements** have **no content inside**.
- Example:

`<br>`

It makes a **line break**, and has **no closing tag**.

HTML is Not Case Sensitive

- `<P>`, `<p>`, `<P>` — all mean the **same tng**.
 - But it's **better to use lowercase**, like `<p>`.
(It's cleaner and needed for XHTML.)
-

HTML Tag Reference

reference contains additional information about these tags and their attributes.

Tag	Description
<code><html></code>	Defines the root of an HTML document
<code><body></code>	Defines the document's body
<code><h1> to <h6></code>	Defines HTML headings

HTML Attributes

HTML Attributes

Linking and Media Attributes

href Attribute

Defines the URL for a hyperlink, directing users to another web page or resource when clicked.

```
<a href="https://example.com">Visit</a>
```

src Attribute

Specifies the path to an image file, allowing it to be displayed on the web page.

```

```

alt Attribute

Provides alternative text for an image, improving accessibility for visually impaired users and serving as a fallback if the image fails to load.

```

```



What are attributes?

- All HTML elements can have attributes.
- Attributes give **extra info** about the element.
- They are always written **inside the start tag**.
- Usually like ts: `name="value"`.

Example: **href** attribute

- The **<a>** tag makes a **link**.
- The **href** attribute tells **where the link goes**.

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

- Here:
 - **<a>** = link tag
 - **href="..."** = link's address
 - **Visit W3Schools** = what people click on

The src Attribute

What is the **** tag?

- The **** tag is used to **show an image on a web page**.
 - It has an attribute called **src** wch tells the browser **where to find the image file**.
-

Example

```

```

- `<img>` is the **image tag**.
 - `src="img_girl.jpg"` says **wh image file to display**.
-

Two ways to write the `src` attribute

1) Absolute URL

- An **absolute URL** is the **full website address** of an image.

Example:

```

```

-
- It tells the browser to get the image from **another website**.

Be careful:

- That image may be **protected by copyright**. You might not be allowed to use it without permission.
 - Also, you **don't control that image**. If the other website deletes it or changes it, your image will **disappear or change** on your page.
-

2) Relative URL

- A **relative URL** links to an image **inside your own website's folders**.
- It does **not include the domain name**.

Examples:

```

```

- Ts looks for `img_girl.jpg` in the **same folder as your HTML file**.

```

```

- Ts looks inside the `images` folder.

```

```

- The `/` at the start means it starts looking from the **root of your website domain**.

The width and height Attributes

- The `<img>` tag can also use `width` and `height` attributes.
- These set how wide and tall the image should be, in pixels.

Example:

```

```

- Ts means the image will be 500 pixels wide and 600 pixels tall.

The alt Attribute

- The **alt** attribute gives alternative text for the image.
- This text is shown if the image can't load, or is used by screen readers for visually impaired users.

Example:

```

```

- If the image doesn't appear, it will show the text "Girl with a jacket".
-

If the image doesn't exist

Look at this example:

```

```

- Here **img_typo.jpg** does not exist.
- So the browser will display the alt text instead.

The style Attribute

- The `style` attribute is used to **add styles directly to an element**.
- You can change the **color, font, size**, and more.

Example:

```
<p style="color:red;">Ts is a red paragraph.</p>
```

- Here, the text inside `<p>` will be **red**.

You will learn more about ts in **HTML Styles**.

The lang Attribute

- The `lang` attribute tells the **language of your webpage**.
- It goes inside the `<html>` tag.
- Ts helps **search engines and browsers** understand the page language.

Example

```
<!DOCTYPE html>
<html lang="en">
<body>
...
</body>
</html>
```

- `lang="en"` means the page is in **English**.

You can also add a **country code**:

Example:

```
<html lang="en-US">
```

- Here, **en** is **English**, **US** is **United States**.

The title Attribute

- The **title** attribute gives **extra info about an element**.
- When you move your mouse over it, a **tooltip** pops up.

Example:

```
<p title="I'm a tooltip">Ts is a paragraph.</p>
```

- If you hover your mouse over ts paragraph, you'll see **"I'm a tooltip."**

We Suggest: Always Use Lowercase Attributes

HTML allows uppercase or lowercase:

```
<p TITLE="Hello">
```

But it is **better to always use lowercase**, like:

```
<p title="Hello">
```

We Suggest: Always Quote Attribute Values

HTML also lets you write without quotes, like:

```
<a href=https://www.w3schools.com/html/>Visit our HTML tutorial</a>
```

- But ts is **not good**.

It is **better to use quotes**:

```
<a href="https://www.w3schools.com/html/">Visit our HTML tutorial</a>
```

Single or Double Quotes?

You can use **double quotes** (most common):

```
<p title="John 'ShotGun' Nelson">
```

Or **single quotes** if your text has double quotes:

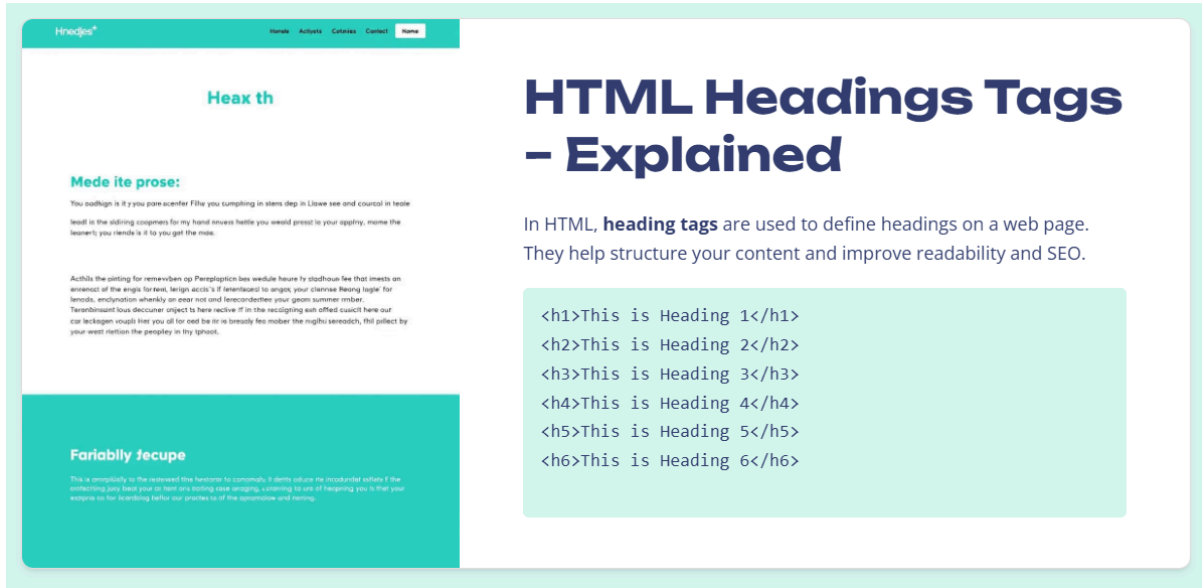
```
<p title='John "ShotGun" Nelson'>
```

Chapter Summary

Attribute	What it does
<code>href</code> on <code><a></code>	URL of the link
<code>src</code> on <code></code>	Path to the image file
<code>width & height</code> on <code></code>	Sets image size in pixels
<code>alt</code> on <code></code>	Alternate text if image doesn't load
<code>style</code>	Adds styles (like color, font, size)
<code>lang</code> on <code><html></code>	Sets the language of the page (like English)
<code>title</code>	Shows a tooltip when you hover over the element

HTML Headings

HTML Headings



The screenshot shows a webpage with a teal header and navigation bar. The main content area has a large heading 'HTML Headings Tags - Explained'. Below it, there's a paragraph explaining that heading tags are used to define headings on a web page and help structure content. To the right, a light blue box lists six heading tags: <h1>, <h2>, <h3>, <h4>, <h5>, and <h6>, each followed by a sample text 'This is Heading X'. On the left side of the page, there are several smaller sections with headings like 'Heax th', 'Mede ite prose:', and 'Fariabily fecupe', each followed by a paragraph of placeholder text.

What are HTML headings?

- Headings are **titles or subtitles** that you show on a webpage.
- They tell what each section is about.

Example:

There are 6 levels of headings:

- **Heading 1** (**<h1>**) is the **biggest**.
- **Heading 6** (**<h6>**) is the **smallest**.

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

Headings Are Important

- Search engines (like Google) use them to understand what your page is about.
- People also look at headings to quickly find what they want on the page.

Use **<h1>** for the main title.

Use **<h2>** for subheadings under that.

Then use **<h3>**, **<h4>**, etc. for smaller sections.

Bigger Headings

Every HTML heading tag (like **<h1>**, **<h2>**, etc.) has its own **default size**.

But you can **change the size yourself** using the **style** attribute with **CSS**.

Example:

```
<h1 style="font-size:60px;">Heading 1</h1>
```

It makes the **Heading 1** text **60 pixels tall**, no matter what the default size is.

HTML Tag Reference

W3Schools has a list that tells you more about each HTML tag and what it can do.

Some examples:

Tag	Description
<u><html></u>	Defines the root of an HTML document
<u><body></u>	Defines the document's body
<u><h1> to <h6></u>	Defines HTML headings

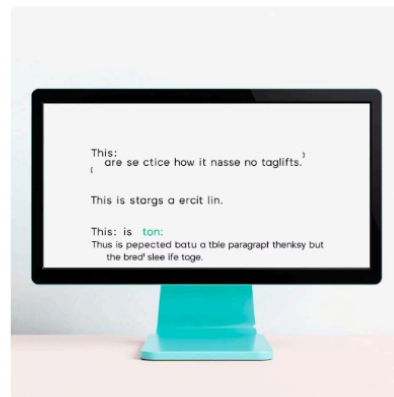
HTML Paragraphs

HTML Paragraphs

HTML Paragraphs Tag

```
<p>This is a paragraph.</p>  
<p>This is another paragraph.</p>
```

As shown, each **<p>** tag encloses a distinct block of text, resulting in two separate paragraphs when rendered by a browser.



The **<p>** tag is used to make a **paragraph**.

A paragraph always starts on a **new line**, and the browser adds some **space above and below** it.

Example:

```
<p>Ts is a paragraph.</p>  
<p>Ts is another paragraph.</p>
```

HTML Display

You **can't be sure exactly how it will look** on every screen.

- Big screens, small screens, or resized windows might all show it differently.

Adding **extra spaces or lines** in your HTML **won't change** how it looks.

- The browser ignores extra spaces and lines.

Example:

```
<p>
Ts paragraph
has many lines
in the code,
but the browser
shows it as one block.
</p>
```

```
<p>
Ts paragraph has many spaces
but the browser
still shows it normally.
</p>
```

HTML Horizontal Rules

The **<hr>** tag makes a **horizontal line** across the page.

It is used to **separate sections** or show a **change in topic**.

Example:

```
<h1>Ts is heading 1</h1>
<p>Ts is some text.</p>
<hr>
<h2>Ts is heading 2</h2>
<p>Ts is some other text.</p>
<hr>
```

The `<hr>` tag is an **empty tag**, wch means it **doesn't need a closing tag** (no `</hr>`).

HTML Line Breaks

The `<br>` tag makes a **line break** (starts text on a new line) **without starting a new paragraph**.

Example:

```
<p>Ts is<br>a paragraph<br>with line breaks.</p>
```

It is an **empty tag**, so it **doesn't have a closing tag**.

The Poem Problem

If you write text inside a `<p>` tag with many lines or spaces, the browser **will still show it as one single line**, ignoring your line breaks and spaces.

Example:

```
<p>
  My Bonnie lies over the ocean.
  My Bonnie lies over the sea.
  My Bonnie lies over the ocean.
  Oh, bring back my Bonnie to me.
</p>
```

Ts will **all appear in one line** on the page.

Solution - The HTML `<pre>` Element

The **`<pre>`** tag keeps the text **exactly as you type it**, including **spaces and line breaks**. It also uses a **fixed-width font**.

Example:

```
<pre>
  My Bonnie lies over the ocean.
  My Bonnie lies over the sea.
  My Bonnie lies over the ocean.
  Oh, bring back my Bonnie to me.
</pre>
```

It will look **just like it does in your HTML file**.

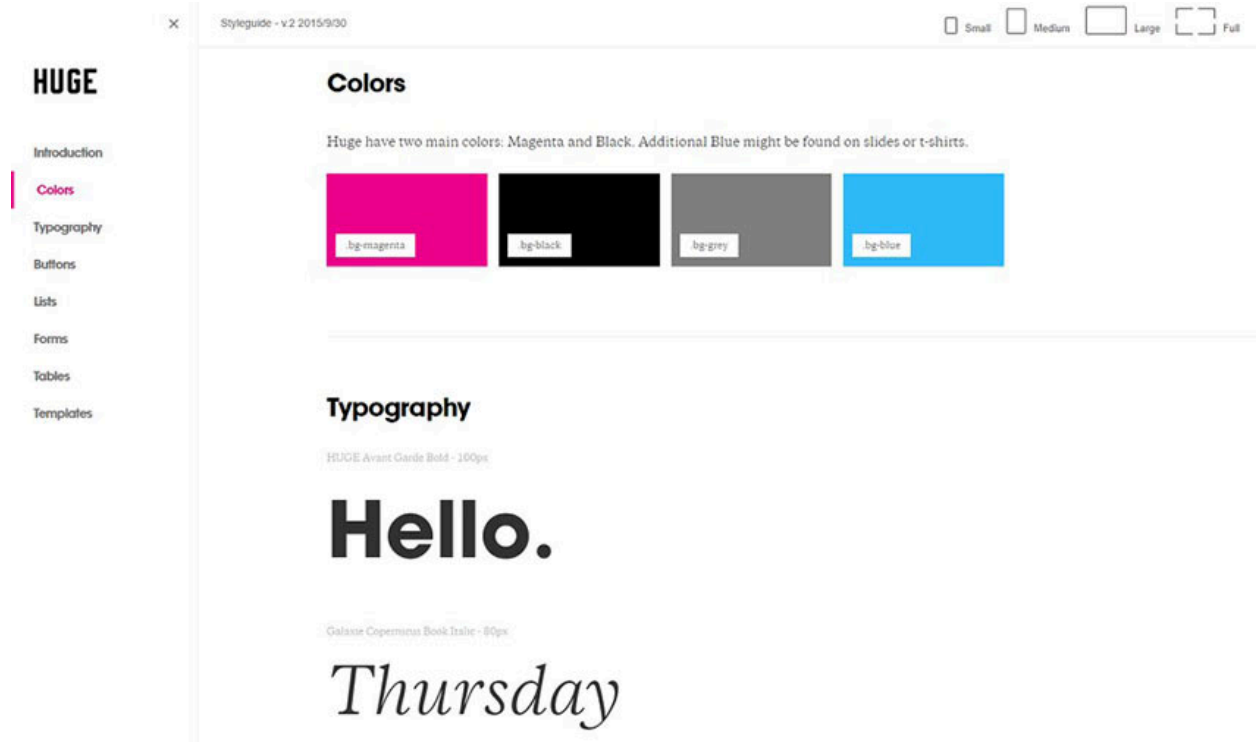
HTML Tag Reference

W3Schools has a **tag reference** that gives more details about **HTML tags** and what they can do.

Tag	Description
<code><p></code>	Defines a paragraph
<code><hr></code>	Defines a thematic change in the content
<code>
</code>	Inserts a single line break
<code><pre></code>	Defines pre-formatted text

HTML Styles

HTML Styles



The HTML `style` attribute is used to add styles to an element, such as color, font, size, and more.

Example

I am Red

I am Blue

I am Big

The HTML Style Attribute

You can **change how an HTML element looks** by adding a **style** attribute.
Ts lets you use **CSS properties** right inside your HTML.

Basic syntax:

```
<tagname style="property:value;">
```

- **property** is the **CSS property** (like **color**, **background-color**, etc.)
- **value** is what you want to set (like **red**, **20px**).

Background Color

Use the **background-color** property to change the **background color**.

Example

Set background color for the whole page:

```
<body style="background-color:powderblue;">  
  <h1>Ts is a heading</h1>  
  <p>Ts is a paragraph.</p>  
</body>
```

Example

Set different background colors for different elements:

```
<body>
  <h1 style="background-color:powderblue;">Ts is a heading</h1>
  <p style="background-color:tomato;">Ts is a paragraph.</p>
</body>
```

Text Color

Use the **color** property to change the **text color**.

Example

```
<h1 style="color:blue;">Ts is a heading</h1>
<p style="color:red;">Ts is a paragraph.</p>
```

Fonts

Use **font-family** to change the **font style**.

```
<h1 style="font-family:verdana;">Ts is a heading</h1>
<p style="font-family:courier;">Ts is a paragraph.</p>
```

Text Size

Use **font-size** to make text **bigger** or **smaller**.

```
<h1 style="font-size:300%;">Big heading</h1>
```



```
<p style="font-size:160%;">Larger paragraph</p>
```

Text Alignment

Use `text-align` to **center**, **left**, or **right align** text.

```
<h1 style="text-align:center;">Centered Heading</h1>  
<p style="text-align:center;">Centered paragraph.</p>
```

Use `style` to style HTML elements.

- `background-color` for **background color**.
- `color` for **text color**.
- `font-family` for **text fonts**.
- `font-size` for **text size**.
- `text-align` for **text alignment**.

HTML Formatting

HTML Text Formatting



HTML Text Formatting

HTML provides several **text formatting tags** to style content such as making text **bold**, *italic*, underlined, and more. These tags help improve the **appearance and meaning** of the text on a webpage.

Common Text Formatting Tags

<code></code>	Bold text (visual only)	<code>This is bold</code>
<code></code>	Important text (semantic bold)	<code>Important!</code>
<code><i></code>	Italic text (visual only)	<code><i>This is italic</i></code>
<code></code>	Emphasised text (semantic italic)	<code>This is emphasised</code>
<code><u></code>	Underlined text	<code><u>This is underlined</u></code>

HTML has special tags to show **text with different meanings or styles**.

Examples:

`<b>Ts text is bold</b>`

```
<i>Ts text is italic</i>
```

```
<p>Ts is <sub>subscript</sub> and <sup>superscript</sup></p>
```

Tag	What it does
<code></code>	Makes bold text
<code></code>	Shows important text (also bold)
<code><i></code>	Makes italic text
<code></code>	Shows emphasized text (also italic)
<code><mark></code>	Highlights text
<code><small></code>	Makes smaller text
<code></code>	Shows deleted text (line through it)
<code><ins></code>	Shows inserted text (underlined)
<code><sub></code>	Subscript (small text below)
<code><sup></code>	Superscript (small text above)

HTML `<b>` and `<strong>` Elements

`<b>` vs `<strong>`

`<b>`

- Makes the text **bold**, but **doesn't add extra meaning**.
- Example:

```
<b>Ts text is bold</b>
```

- Makes the text **bold** too, **but also means it's important**.
- Search engines and screen readers understand it as **important text**.
Example:

```
<strong>Ts text is important!</strong>
```

HTML <i> and Elements

The **<i>** tag

- The **<i>** tag makes text look *italic*.
- Use **<i>** when you want to show a word from another language, a thought, or a sp name.
- Example: **<i>Ts text is italic</i>**

The **** tag

- The `<em>` tag also makes text *italic*, but it means the text is **important**.
 - A screen reader will **say the words in `<em>` with more stress**.
 - Example: `<em>Ts text is emphasized</em>`
-

The `<small>` tag

- The `<small>` tag makes text **smaller**.
- Example: `<small>Ts is some smaller text.</small>`

HTML `<small>` Element

- The `<small>` tag makes your text look **smaller** than normal.
- You can use it to show side notes or less important text.

Example:

```
<small>Ts is some smaller text.</small>
```

HTML `<mark>` Element

- The `<mark>` tag highlights text, like using a yellow marker.
- Example:

```
<p>Do not forget to buy <mark>milk</mark> today.</p>
```

HTML Element

- The `<del>` tag shows text that has been **deleted**, usually with a line through it.
- Example:

```
<p>My favorite color is <del>blue</del> red.</p>
```

HTML <ins> Element

- The `<ins>` tag shows text that has been **added**, usually with an underline.
- Example:

```
<p>My favorite color is <del>blue</del> <ins>red</ins>.</p>
```

HTML <sub> Element

- The `<sub>` tag makes **subscript** text. It goes **slightly below the line**, like for chemical formulas.
- Example:

```
<p>Ts is <sub>subscripted</sub> text.</p>
```

HTML <sup> Element

- The `<sup>` tag makes **superscript** text. It goes **slightly above the line**, like for footnotes.

- Example:

```
<p>Ts is <sup>superscripted</sup> text</p>
```

HTML Text Formatting Elements

Here's a neat summary of the **HTML text formatting elements** you listed, arranged in a simple table format for easy reference:

Tag	Description
<u></u>	Defines bold text
<u></u>	Defines emphasized text
<u><i></u>	Defines a part of text in an alternate voice or mood
<u><small></u>	Defines smaller text
<u></u>	Defines important text
<u><sub></u>	Defines subscripted text
<u><sup></u>	Defines superscripted text
<u><ins></u>	Defines inserted text
<u></u>	Defines deleted text
<u><mark></u>	Defines marked/highlighted text

HTML Quotation

HTML Quotation and Citation Elements

We will go through the following HTML elements:

`<blockquote>`, `<q>`, `<abbr>`, `<address>`, `<cite>`, and `<bdo>`.

These elements are used to format different kinds of text content in HTML, such as quotations, abbreviations, addresses, titles of works, and to change text direction.

Example

Here is a quote from WWF's website:

For 60 years, WWF has worked to help people and nature thrive. As the world's leading conservation organization, WWF works in nearly 100 countries. At every level, we collaborate with people around the world to develop and deliver innovative solutions that protect communities, wildlife, and the places in wch they live.

HTML `<blockquote>` for Quotations

The HTML `<blockquote>` element defines a section that is quoted from another source.

Browsers usually indent `<blockquote>` elements.

Example

```
<p>Here is a quote from WWF's website:</p>
<blockquote cite="http://www.worldwildlife.org/who/index.html">
For 60 years, WWF has worked to help people and nature thrive. As the
world's leading conservation organization, WWF works in nearly 100
countries. At every level, we collaborate with people around the world
to develop and deliver innovative solutions that protect communities,
wildlife, and the places in wch they live.
</blockquote>
```

HTML `<q>` for Short Quotations

The HTML `<q>` tag is used to define a **short, inline quotation** inside your text.

It is meant for small quotes that fit inside a sentence.

Most browsers will automatically add **quotation marks** around the text inside the `<q>` tag.

Ts helps readers easily see that it's a quote.

Example:

```
<p>WWF's goal is to: <q> Build a future where people live in harmony
with nature.</q></p>
```

In ts example, the sentence will look like ts on the page:

WWF's goal is to: “Build a future where people live in harmony with nature.”

HTML `<abbr>` for Abbreviations

The HTML `<abbr>` tag defines an **abbreviation or acronym**, like "HTML", "CSS", "Mr.", "Dr.", "ASAP", or "ATM".

Using this tag helps browsers, translators, and search engines understand the meaning of the short form.

You can use the `title` attribute to show the **full meaning** when someone hovers over the abbreviation.

Example

```
<p>The <abbr title="World Health Organization">WHO</abbr>was  
founded in 1948.</p>
```

When you move your mouse over **WHO**, it will show:

World Health Organization

HTML `<address>` for Contact Information

The HTML `<address>` tag defines the **contact information** for the author or owner of a page or article.

It can include an email, website, physical address, phone number, or social media.

By default, the text is shown in *italic* and browsers add line breaks before and after.

Example

```
<address>
```

```
Written by sagar avhad.<br>
```

```
Visit us at:sinnar<br>
```

```
Example.com<br>
```

```
Box 564, Disneyland<br>
```

```
USA
```

```
</address>
```

HTML <cite> for Work Title

The HTML <cite> tag defines the **title of a creative work**, like a book, poem, song, movie, or painting.

It does **not** apply to a person's name.

By default, text inside <cite> is shown in *italic*.

Example

```
<p><cite>The Scream</cite> by Edvard Munch. Painted in 1893.</p>
```

HTML <bdo> for Bi-Directional Override

BDO stands for **Bi-Directional Override**.

The HTML <bdo> tag is used to **change the text direction**, for example from left-to-right to right-to-left.

Example

```
<bdo dir="rtl">Ts text will be written from right to left</bdo>
```

HTML Quotation and Citation Elements

Tag	Description
<u><abbr></u>	Defines an abbreviation or acronym
<u><address></u>	Defines contact information for the author/owner of a document
<u><bdo></u>	Defines the text direction
<u><blockquote></u>	Defines a section that is quoted from another source
<u><cite></u>	Defines the title of a work
<u><q></u>	Defines a short inline quotation

HTML Comments

HTML Comment Tag

There is an **exclamation mark (!)** in the **start tag**, but **no exclamation mark in the end tag**.

Add Comments

You add a comment in HTML like ts:

```
<!-- Write your comments here -->
```

There is an **exclamation mark (!)** in the **start tag**, but **no exclamation mark in the end tag**.

Hide Content

Comments can be used to **de content**.

Ts is helpful if you want to **temporarily remove sometng** without deleting it.

Example

```
<p>Ts is a paragraph.</p>
```

```
<!-- <p>Ts is another paragraph</p> -->
```

```
<p>Ts is a paragraph too.</p>
```

You can also de more than one line. Everytng between the and the will be dden from the display.

Example


```
<p>Ts is a paragraph.</p>
<!--
<p>Look at ts cool image:</p>

<p>Ts is a paragraph too.</p>
```

Comments are also **useful for debugging**, because you can comment out lines one by one to find errors

Hide Inline Content

Comments can be used to **de parts in the middle of your HTML code**, even inside a paragraph.

Example

```
<p>Ts <!-- great text --> is a paragraph.</p>
```

Here, the words "**great text**" are hidden and will not show on the page.

HTML Colors

Colors

HTML Colors

HTML colors are specified with predefined color names, or with RGB, HEX, HSL, RGBA, or HSLA values. give me in short

Color Names

In HTML, you can set a color by simply using its **name**.

Orange
DodgerBlue
MediumSeaGreen
Gray
SlateBlue
Violet
LightGray

HTML supports [140 standard color names](#).

Background Color

You can set the background color for HTML elements:

Hello World

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.

Example

```
<h1 style="background-color:DodgerBlue;">Hello World</h1>
```

```
<p style="background-color:Tomato;">Lorem ipsum...</p>
```

Text Color

You can set the **color of text** in HTML using the **color** style.

Example

```
<h1 style="color:Tomato;">Hello World</h1>
```

```
<p style="color:DodgerBlue;">
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibh

euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.

```
</p>
```

```
<p style="color:MediumSeaGreen;">
```

Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper
suscipit
lobortis nisl ut aliquip ex ea commodo consequat.
</p>

Border Color

You can set the color of borders

Hello World

Hello World

Hello World

Example

```
<h1 style="border:2px solid Tomato;">Hello World</h1>
```

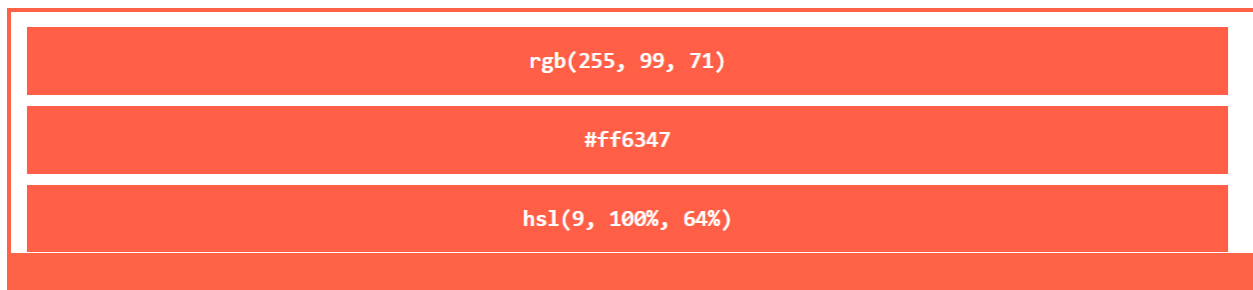
```
<h1 style="border:2px solid DodgerBlue;">Hello World</h1>
```

```
<h1 style="border:2px solid Violet;">Hello World</h1>
```

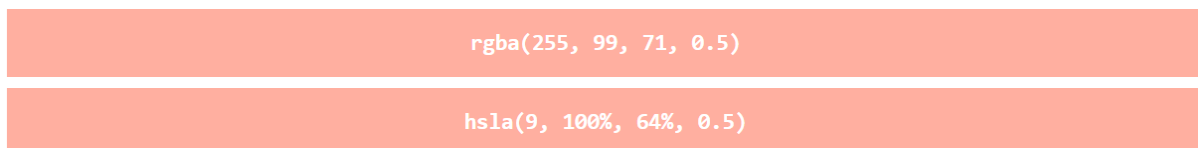
Color Values

In HTML, colors can also be set using **RGB values**, **HEX values**, **HSL values**, **RGBA values**, and **HSLA values**.

The three `<div>` elements below have their background color set using **RGB**, **HEX**, and **HSL**:



The following two `<div>` elements have their background color set using **RGBA** and **HSLA** values, wch include an **Alpha channel** for transparency (here it is 50%).



Example :

```
<h1 style="background-color:rgb(255, 99, 71);">Ts is RGB</h1><h1
style="background-color:#ff6347;">Ts is HEX</h1><h1
style="background-color:hsl(9, 100%, 64%);">Ts is HSL</h1><h1
style="background-color:rgba(255, 99, 71, 0.5);">Ts is RGBA (with
transparency)</h1><h1 style="background-color:hsla(9, 100%, 64%,
0.5);">Ts is HSLA (with transparency)</h1>
```


RGB

HTML RGB and RGBA Colors

An RGB color value represents RED, GREEN, and BLUE light sources.

An RGBA color value is an extension of RGB with an Alpha channel (opacity).

RGB Color Values

RGB Color Values

In HTML, you can set a color as an RGB value, using the formula:

```
rgb(red, green, blue)
```

Each part (red, green, blue) tells how strong that color is, from **0 to 255**.

This means there are **256 x 256 x 256 = 16,777,216 colors!**

Example

- `rgb(255, 0, 0)` shows **red**, because red is at its highest and the other two are 0.
- `rgb(0, 255, 0)` shows **green**, because green is at its highest.

Example

`rgb(255, 0, 0)`

`rgb(0, 0, 255)`

`rgb(60, 179, 113)`

`rgb(238, 130, 238)`

`rgb(255, 165, 0)`

`rgb(106, 90, 205)`

Shades of Gray

Shades of gray are often defined using equal values for all three parameters:

Example

`rgb(60, 60, 60)`

`rgb(100, 100, 100)`

`rgb(140, 140, 140)`

`rgb(180, 180, 180)`

`rgb(200, 200, 200)`

`rgb(240, 240, 240)`

RGBA Color Values

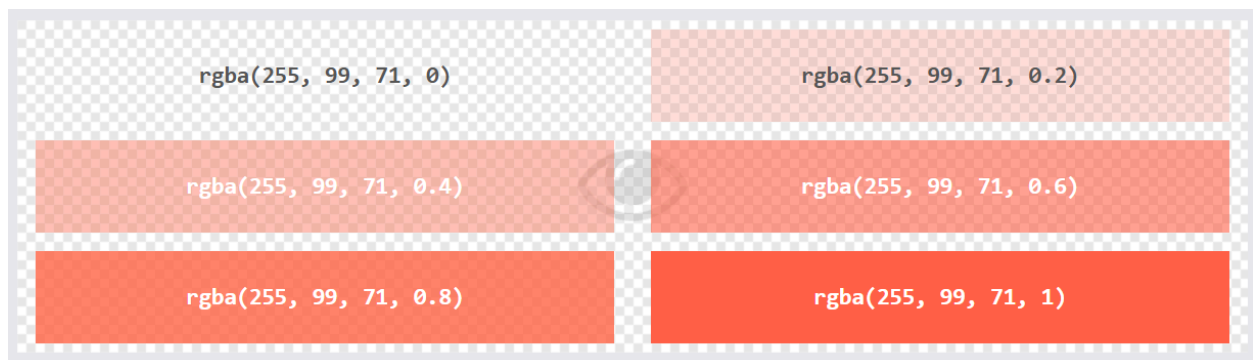
RGBA color values are like RGB, but they also have an **Alpha channel**, wch sets how transparent the color is.

An RGBA color is written like ts:

```
rgba(red, green, blue, alpha)
```

The **alpha** value is a number between **0.0** (completely transparent) and **1.0** (fully solid, not transparent).

Example



Experiment by mixing the RGBA values below:

rgba(255, 99, 71, 0.5) transparent (half see-through)

HEX

HEX Color Values

In HTML, a color can be specified using a hexadecimal value in the form:

#rrggbb

Where rr (red), gg (green) and bb (blue) are hexadecimal values between 00 and ff (same as decimal 0-255).

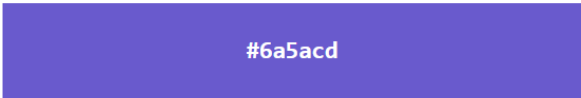
For example, #ff0000 is displayed as red, because red is set to its highest value (ff), and the other two (green and blue) are set to 00.

Another example, #00ff00 is displayed as green, because green is set to its highest value (ff), and the other two (red and blue) are set to 00.

To display black, set all color parameters to 00, like this: #000000.

To display white, set all color parameters to ff, like this: #ffffff.

Example :

 #ff0000	 #0000ff
 #3cb371	 #ee82ee
 #ffa500	 #6a5acd

Shades of Gray

Shades of gray are often defined using equal values for all three parameters

#404040	#686868
#a0a0a0	#bebebe
#dcdcdc	#f8f8f8

HSL

HSL Color Values

In HTML, you can also set a color using **HSL**, wch stands for **Hue, Saturation, and Lightness**. It looks like ts:

`hsl(hue, saturation, lightness)`

- **Hue** is an angle from **0 to 360** on the color wheel.
 - **0** is **red**, **120** is **green**, **240** is **blue**.
- **Saturation** is how strong the color is.
 - **0%** means **gray**, **100%** means **full color**.
- **Lightness** is how light or dark the color is.
 - **0%** is **black**, **100%** is **wte**.

Example :

<code>hsl(0, 100%, 50%)</code>	<code>hsl(240, 100%, 50%)</code>
<code>hsl(147, 50%, 47%)</code>	<code>hsl(300, 76%, 72%)</code>
<code>hsl(39, 100%, 50%)</code>	<code>hsl(248, 53%, 58%)</code>

Saturation

Saturation shows how **strong or dull a color is**.

- **100%** means **pure color**, with no gray at all.
- **50%** means the color is mixed with **50% gray**, so it looks softer.
- **0%** means it's **all gray**, and the original color is gone.

Example :

<code>hsl(0, 100%, 50%)</code>	<code>hsl(0, 80%, 50%)</code>
<code>hsl(0, 60%, 50%)</code>	<code>hsl(0, 40%, 50%)</code>
<code>hsl(0, 20%, 50%)</code>	<code>hsl(0, 0%, 50%)</code>

Lightness

Lightness tells you **how light or dark a color is**.

- **0%** means **no light at all**, so it's **black**.
- **50%** means it's **normal**, not too dark or too light.
- **100%** means **full light**, so it becomes **white**.

Example :

<code>hsl(0, 100%, 0%)</code>	<code>hsl(0, 100%, 25%)</code>
<code>hsl(0, 100%, 50%)</code>	<code>hsl(0, 100%, 75%)</code>
<code>hsl(0, 100%, 90%)</code>	<code>hsl(0, 100%, 100%)</code>

Shades of Gray

You can make **shades of gray** by setting **hue and saturation to 0**, and then changing the **lightness**.

- Lightness at **0%** gives **black**.
- Lightness at **100%** gives **white**.
- Anything in between gives **different grays**, from dark to light.

Example :

<code>hsl(0, 0%, 20%)</code>	<code>hsl(0, 0%, 30%)</code>
<code>hsl(0, 0%, 40%)</code>	<code>hsl(0, 0%, 60%)</code>
<code>hsl(0, 0%, 70%)</code>	<code>hsl(0, 0%, 90%)</code>

HSLA Color Values

HSLA color values are just like HSL, but they also have an **Alpha channel**, wch controls transparency .

`hsla(hue, saturation, lightness, alpha)`

- The **alpha** value goes from **0.0** (completely see-through) to **1.0** (fully solid).

Example :

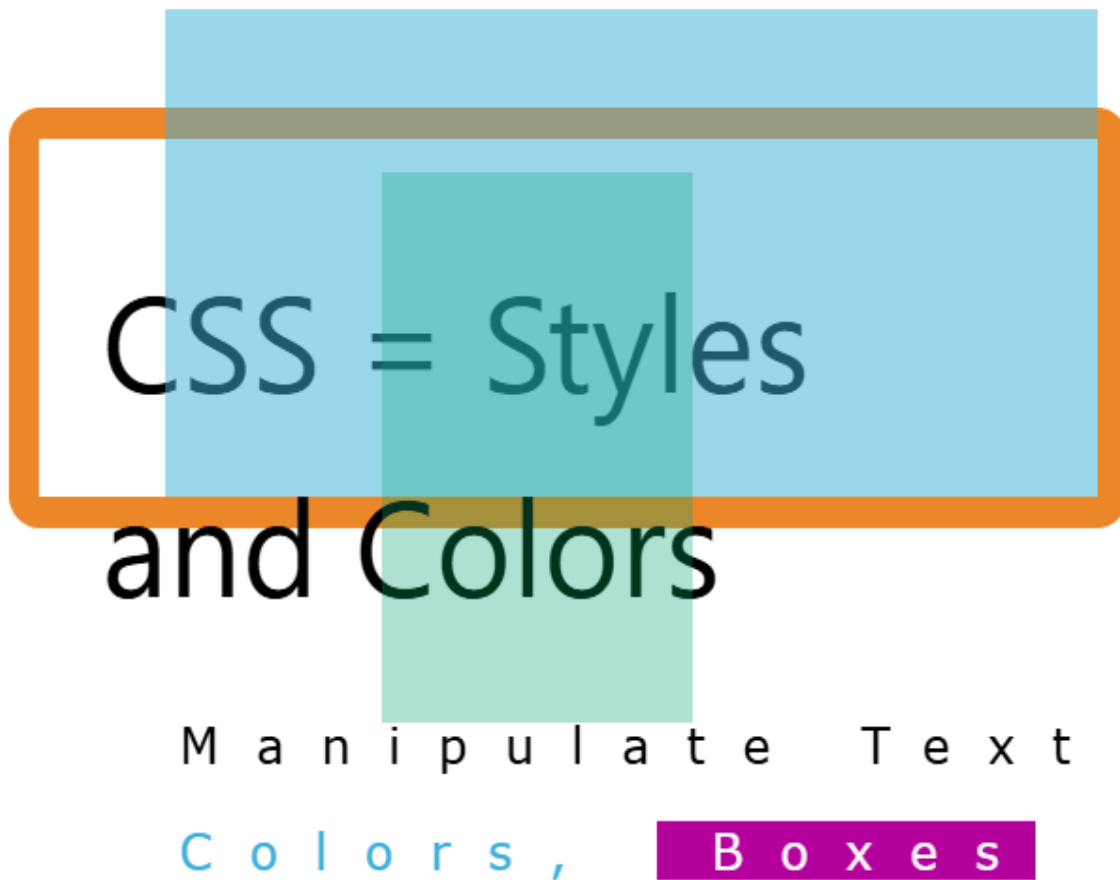
<code>hsla(9, 100%, 64%, 0)</code>	<code>hsla(9, 100%, 64%, 0.2)</code>
<code>hsla(9, 100%, 64%, 0.4)</code>	<code>hsla(9, 100%, 64%, 0.6)</code>
<code>hsla(9, 100%, 64%, 0.8)</code>	<code>hsla(9, 100%, 64%, 1)</code>

HTML CSS

HTML Styles - CSS

CSS means **Cascading Style Sheets**.

It makes your work easier because you can style and design **many web pages at once** by changing just one CSS file. It helps you control the layout, colors, fonts, and more across your whole website.



What is CSS?

CSS stands for **Cascading Style Sheets**. It is used to **style and format** the layout of a web page.

With CSS, you can easily change:

- **colors, fonts, and text sizes,**
- the **spacing and positioning** of elements,
- add **background colors** or **images,**
- and make your site look good on **different screen sizes and devices.**

Tip:

The word “**cascading**” means that styles on a parent element will also apply to its children. For example, if you set the body text color to blue, then all the text inside the body—like headings and paragraphs—will be blue too, unless you set a different color for them.

Using CSS

You can add **CSS** to your HTML in **three ways**:

- 1 **Inline** – by using the `style` attribute directly on HTML elements.
 - 2 **Internal** – by placing a `<style>` tag inside the `<head>` section of your HTML file.
 - 3 **External** – by linking to a separate CSS file with a `<link>` tag.
-

The **most common** (and best) way is to use **external CSS files**, because it keeps your code clean and lets you style multiple pages at once.

But for quick examples or practice, **inline** and **internal** styles are easier to show and try out.

Inline CSS

Inline CSS is used to style **just one specific HTML element**.

You do ts by adding a **style attribute** directly inside the tag.

Example:

```
<h1 style="color:blue;">A Blue Heading</h1>  
<p style="color:red;">A red paragraph.</p>
```

Here, the `<h1>` text will be **blue**, and the `<p>` text will be **red**. Ts is handy for quick, one-off styles.

Internal CSS

An internal CSS is used to define a style for a single HTML page.

An internal CSS is defined in the `<head>` section of an HTML page, with a `<style>` element.

The following example sets the text color of ALL the `<h1>` elements (on that page) to blue, and the text color of ALL the `<p>` elements to red. In addition, the page will be displayed with a "powderblue" background color:

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

body {background-color: powderblue;}

h1  {color: blue;}

p   {color: red;}

</style>

</head>

<body>


<h1>Ts is a heading</h1>

<p>Ts is a paragraph.</p>


</body>
```

</html>

External CSS

An **external CSS file** lets you style **many HTML pages at once**, using just **one CSS file**.

You link it to your HTML pages by adding ts in the `<head>`:

```
<link rel="stylesheet" href="styles.css">
```

Example

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>Ts is a heading</h1>
<p>Ts is a paragraph.</p>

</body>
</html>
```

Example CSS file ("styles.css"):

```
body {
```

```
background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Tip:

With external CSS, you can **change the look of your entire website by editing just one file** — super easy and powerful!

CSS Colors, Fonts and Sizes

Here's a small example showing some **common CSS properties**:

- `color` sets the **text color**.
- `font-family` sets the **font style**.
- `font-size` sets the **size of the text**.

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
```

```
h1 {  
  color: blue;  
  font-family: verdana;  
  font-size: 300%;  
}  
p {  
  color: red;  
  font-family: courier;  
  font-size: 160%;  
}  
</style>  
</head>  
<body>  
  
<h1>Ts is a heading</h1>  
<p>Ts is a paragraph.</p>  
  
</body>  
</html>
```

Ts means:

- The `<h1>` will be **blue**, use the **Verdana font**, and be **300% bigger than normal text**.
- The `<p>` will be **red**, use the **Courier font**, and be **160% bigger**.

CSS Border

The **CSS border** property adds a **border** around an **HTML** element.

You can add borders to almost **any HTML element**, like paragraphs, headings, divs, etc.

Example:

```
p {  
  border: 2px solid powderblue;  
}
```

CSS Padding

The **CSS padding** property adds **space inside the element**, between the **content (like text)** and the **border**.

Example:

```
p {  
  border: 2px solid powderblue;  
  padding: 30px;  
}
```

Here, each `<p>` will have:

- a **2px powder blue border**, and
- **30px of space inside the border**, around the text

CSS Margin

The **CSS margin** property adds **space outside the border** of an element.

It pushes the element **away from other elements** around it.

Example:

```
p {  
  border: 2px solid powderblue;  
  margin: 50px;  
}
```

That means each `<p>` will have:

- a **2px powder blue border**, and
- **50px of space outside the border**, keeping it away from other content

You can link an **external CSS file** in different ways, depending on where the file is stored.

Link to External CSS

Examples:

Using a full URL:

```
<link rel="stylesheet"  
href="https://www.w3schools.com/html/styles.css">
```

Loads the CSS from another website.

Using a relative path to a folder on your site:

```
<link rel="stylesheet" href="/html/styles.css">
```

Loads the CSS from your own **/html** folder.

Using a file in the same folder as your page:

```
<link rel="stylesheet" href="styles.css">
```

Loads the CSS file that's **in the same folder** as your HTML file.

Chapter Summary

- Use the **style** attribute to add **inline styles** directly on an HTML tag.
 - Use the **<style>** tag inside the **<head>** for **internal CSS** on one page.
 - Use the **<link>** tag inside the **<head>** to connect to an **external CSS file**.
-

- Use the **color** property to change **text color**.
- Use the **font-family** property to set the **font style**.
- Use the **font-size** property to adjust **text size**.
- Use the **border** property to add **borders** around elements.
- Use the **padding** property for **space inside the border** (around content).
- Use the **margin** property for **space outside the border** (around the element).

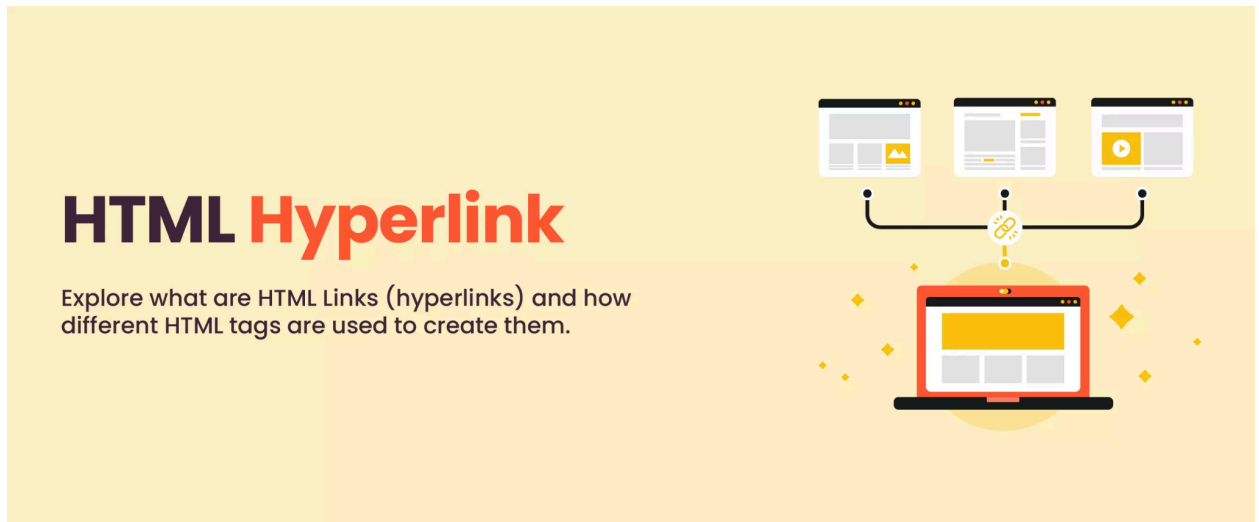
HTML Style Tags

Tag	Description
<u><style></u>	Defines style information for an HTML document
<u><link></u>	Defines a link between a document and an external resource

HTML Links

Links

HTML Links - Hyperlinks



HTML links are called **hyperlinks**.

You can **click them to go to another page or website**.

When you move your mouse over a link, the cursor changes to a **hand**, showing it's clickable.

HTML Links - Syntax

The **HTML <a> tag** is used to make a **hyperlink**.

It looks like ts:

```
<a href="url">link text</a>
```

- **href** tells **where the link goes**.
 - The **link text** is what people see and click.
-

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Link Example</title>
</head>
<body>

  <h2>HTML Link Example</h2>
  <p><a href="https://google.com" target="_blank">Visit Google</a></p>

</body>
</html>
```

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```

By default:

- A **blue link** means you **haven't clicked it yet**.
- A **purple link** means you **already visited it**.
- A **red link** means it is **currently being clicked**.

HTML Links - The target Attribute

By default, a link **opens in the same window or tab**.

If you want to change **where it opens**, use the **target** attribute.

Values for **target**:

- **_self** → (default) opens in the **same window/tab**
 - **_blank** → opens in a **new window or tab**
 - **_parent** → opens in the **parent frame**
 - **_top** → opens in the **full window**, breaking out of any frames
-

Example:

```
<a href="https://www.w3schools.com/" target="_blank">Visit  
W3Schools!</a>
```

Absolute URLs vs. Relative URLs

- A **full web address** (like **https://www.google.com**) is called an **absolute URL**.
 - A **link to another page on the same site** (without **https://**) is called a **relative URL**.
-

Examples:

```
<h2>Absolute URLs</h2>
```

```
<p><a href="https://www.w3.org/">W3C</a></p>
```

```
<p><a href="https://www.google.com/">Google</a></p>
```

```
<h2>Relative URLs</h2>
```

```
<p><a href="html_images.asp">HTML Images</a></p>
```

```
<p><a href="/css/default.asp">CSS Tutorial</a></p>
```

HTML Links - Use an Image as a Link

- You can make an **image work like a link** by putting the `<img>` tag **inside** an `<a>` tag.

Example:

```
<a href="default.asp">
```

```
  
```

```
</a>
```

Clicking the image will **open "default.asp"**.

- To make a link that opens the user's **email app**, use `mailto:` in the `href`.

Example:

```
<a href="mailto:someone@example.com">Send email</a>
```

Button as a Link

If you want a **button to work like a link**, you need to use **JavaScript**.
It lets you say what happens **when the button is clicked**.

Example:

```
<button onclick="document.location='default.asp'">HTML  
Tutorial</button>
```

Link Titles

The **title attribute** gives **extra info** about an element.
When you **hover your mouse over it**, a **small tooltip** pops up.

Example:

```
<a href="https://www.w3schools.com/html/" title="Go to W3Schools HTML  
section">Visit our HTML Tutorial</a>
```

More on Absolute URLs and Relative URLs

Here are examples of how to make links using different paths:

Example 1 — Full URL (absolute link):

Links to another website.

```
<a href="https://www.w3schools.com/html/default.asp">HTML tutorial</a>
```

Example

Goes to a page in your **/html/** folder.

```
<a href="/html/default.asp">HTML tutorial</a>
```

Example

Opens a page **in the same folder** as your current page.

```
<a href="default.asp">HTML tutorial</a>
```

Chapter Summary

Use the **<a>** tag to make a link.

Use **href** to set where the link goes.

Use **target** to set **where it opens** (same tab, new tab, etc.).

Put an **** inside **<a>** to make an **image work as a link**.

Use **mailto:** in **href** to make a link that **opens the email app** to send an email.

HTML Link Tags

Tag	Description
<code><a></code>	Defines a hyperlink

Links Colors

HTML Link Colors

By default:

- Links you **haven't clicked yet** are **blue and underlined**.
- Links you **have clicked before** are **purple and underlined**.
- Links you're **clicking right now** are **red and underlined**.

You can change how links look in different states using **CSS**.

Example:

With CSS changes link styles:

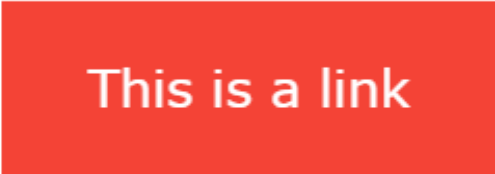
- **Unvisited links** → green, no underline
- **Visited links** → pink, no underline
- **Hover (mouse over)** → red, underlined
- **Active (clicking)** → yellow, underlined

```
<style>
a:link {
  color: green;
  text-decoration: none;
}
a:visited {
  color: pink;
  text-decoration: none;
}
```

```
a:hover {  
  color: red;  
  text-decoration: underline;  
}  
a:active {  
  color: yellow;  
  text-decoration: underline;  
}  
</style>
```

Link Buttons

A link can also be styled as a button, by using CSS:



This is a link

```
<style>
```

```
a:link, a:visited {  
  background-color: #f44336;  
  color: white;  
  padding: 15px 25px;  
  text-align: center;  
  text-decoration: none;  
  display: inline-block;  
}
```



```
a:hover, a:active {  
  background-color: red;  
}  
</style>
```

```
<a href="#">Ts is a link</a>
```

HTML Link Tags

Tag	Description
<u><a></u>	Defines a hyperlink

Link BookMarks

Create a Bookmark in HTML

A **bookmark** lets you **jump to a specific part of a page**.

It's helpful on long pages so users can quickly go to a section.

How to create a bookmark:

1 **Mark the spot** using an `id`:

```
<h2 id="C4">Chapter 4</h2>
```

2 **Link to it** using `href="#id"`:

```
<a href="#C4">Jump to Chapter 4</a>
```

To jump to a bookmark on another page:

```
<a href="html_demo.html#C4">Jump to Chapter 4</a>
```

Chapter Summary

- Use the `id` attribute (`id="value"`) to define bookmarks in a page
- Use the `href` attribute (`href="#value"`) to link to the bookmark

HTML Link Tags

Tag	Description
<code><a></code>	Defines a hyperlink

Html Images

Images

HTML Images

Images can improve the design and the appearance of a web page.



These examples show how to use the `<img>` tag to display images on a web page.

Examples:

```
  
  

```

- `src` specifies the **image file name or path**.

- `alt` gives **alternative text** that shows if the image can't load, and helps screen readers.

HTML Images Syntax

The HTML `<img>` tag is used to **show an image on a web page**.

- It **doesn't actually insert the image** inside the page.
It just **links to the image file** and displays it in a reserved space.
- The `<img>` tag is **empty** — it doesn't have a closing tag.

It has two main attributes:

- `src` → tells the browser **where the image file is** (path or URL).
- `alt` → gives **alternative text** if the image can't load (also good for screen readers).

Syntax:

```

```

Example:

```

```

The src Attribute

When someone opens your web page, the browser goes to get the image from the server **at that moment**.

So make sure the image **is still in the same location**.

If it's missing or moved, visitors will see:

- a **broken image icon**, and
- the **alt text** you provided.

Example:

```

```

The alt Attribute

The **alt attribute** gives a **text description of the image**, wch shows up if:

- the image can't load (maybe because of a broken link or slow internet), or
- someone uses a **screen reader** (like blind users).

It helps with **accessibility** and gives a backup message.

Example:

```

```

If a browser cannot find an image, it will display the value of the `alt` attribute:

```

```

A **screen reader** is a program that reads out the content of a web page.

It **reads the HTML code aloud**, so users can **hear** what's on the page instead of seeing it.

It is very helpful for people who are **visually impaired** or have **learning disabilities**, because it lets them understand and use websites.

Image Size - Width and Height

You can set an image's size in two ways:

1 Using the `style` attribute:

```

```

2 Or using the `width` and `height` attributes directly:

```

```

These sizes are always in **pixels**.

Tip:

Always set the width and height. If you don't, the page might **jump or flicker** while loading, because the browser won't know how much space to keep for the image.

Width and Height, or Style?

- The first image uses `width` and `height` attributes.
- The second image uses the `style` attribute, so it keeps its size even if a CSS rule tries to change it.

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
img {  
    width: 100%; /* ts will try to make all images full width */  
}  
</style>  
</head>  
<body>  
  

```

```


</body>
</html>
```

As you see, the second image keeps its 128px size because it uses **style**.

Images in Another Folder

If your image is inside a folder, you need to tell the browser where to find it by including the folder name in the **src**.

Example:

```

```

Here, the image **html5.gif** is inside the **images** folder.
So you must write **/images/html5.gif** in the **src**.

Images on Another Server/Website

You can also show an image that is stored on another website by giving the full URL (called an absolute URL) in the **src** attribute.

Example:

```

```

Note:

- That image belongs to another site. If you use it without permission, it might break copyright rules.
- Also, because it's on someone else's server, it could be removed or changed anytime, and then your image will not show.

Animated Images

You can use animated GIFs in HTML just like normal images. They will automatically play their animation.

Example:

```

```

Image as a Link

You can turn an image into a clickable link by placing the `<img>` tag inside an `<a>` tag.

Example:

```
<a href="default.asp">  
    
</a>
```

When someone clicks the image, it will open the link.

Image Floating

You can use the CSS `float` property to make an image move to the left or right side of the text. Ts way, the text wraps around the image.

Example:

```
<p>  
    
  The image will float to the right of the text.  
</p>
```

```
<p>  
    
  The image will float to the left of the text.  
</p>
```

Common Image Formats

These are the most common image file types that work in all browsers:

Abbreviation	File Format	File Extension
APNG	Animated Portable Network Graphics	.apng
GIF	Graphics Interchange Format	.gif
ICO	Microsoft Icon	.ico, .cur
JPEG	Joint Photographic Expert Group image	.jpg, .jpeg, .jif, .jpeg, .jpg
PNG	Portable Network Graphics	.png
SVG	Scalable Vector Graphics	.svg

Chapter Summary

- Use the `<img>` tag to show an image.
- Use the `src` attribute to set the image URL.
- Use the `alt` attribute for alternative text (shown if image fails).
- Use `width` and `height` (HTML or CSS) to set image size.
- Use `float` (CSS) to make the image float left or right next to text.

HTML Image Tags

Tag	Description
<u></u>	Defines an image
<u><map></u>	Defines an image map
<u><area></u>	Defines a clickable area inside an image map
<u><picture></u>	Defines a container for multiple image resources

Image Map

Image Maps

The HTML `<map>` tag is used to make an **image map**, wch means an image with **clickable areas** (hotspots).

Each clickable area is defined using the `<area>` tag.

For example, you can click on parts of a picture (like a computer, phone, or coffee cup) and go to different links.



Here is the HTML code for the image map above:

```

```

```
<map name="workmap">
  <area shape="rect" coords="34,44,270,350" alt="Computer"
href="computer.htm">
  <area shape="rect" coords="290,172,333,250" alt="Phone"
href="phone.htm">
  <area shape="circle" coords="337,300,44" alt="Coffee"
href="coffee.htm">
</map>
```

How Does it Work?

The idea of an image map is that you can do different things when you click on different parts of the image.

To make an image map, you need an **image**, and some **HTML code** that shows the clickable areas.

The Image

The image is added with the `<img>` tag. The only difference from a normal image is that you add a `usemap` attribute:

```

```

The use map value starts with a `#` and the name of the image map. This connects the image to the image map.

Create Image Map

Then, add a **<map>** element.

The **<map>** element is used to define the image map, and it links to the image using the required **name** attribute:

```
<map name="workmap">
```

The **name** must be the same as the **use map** value (without the #) in the **** tag.

The Areas

Then, add the clickable areas.

A clickable area is made using an **<area>** element.

Shape

You have to set the **shape** of the clickable area, and you can use one of these:

- **rect** – a rectangle
- **circle** – a circle

- **poly** – a polygon (many sides)
- **default** – the whole image

You also need to give **coordinates** to position the clickable area on the image.

Shape="rect"

The coordinates for **shape="rect"** are given in pairs:
one value for the **x-axis (left to right)** and one for the **y-axis (top to bottom)**.

For example, 34, 44 means:



The coordinates **270, 350** is located 270 pixels from the left margin and 350 pixels from the top:



Now we have enough data to create a clickable rectangular area:

Example

```
<area shape="rect" coords="34,44,270,350" href="computer.htm">
```

It makes a **rectangle area** on the image clickable.

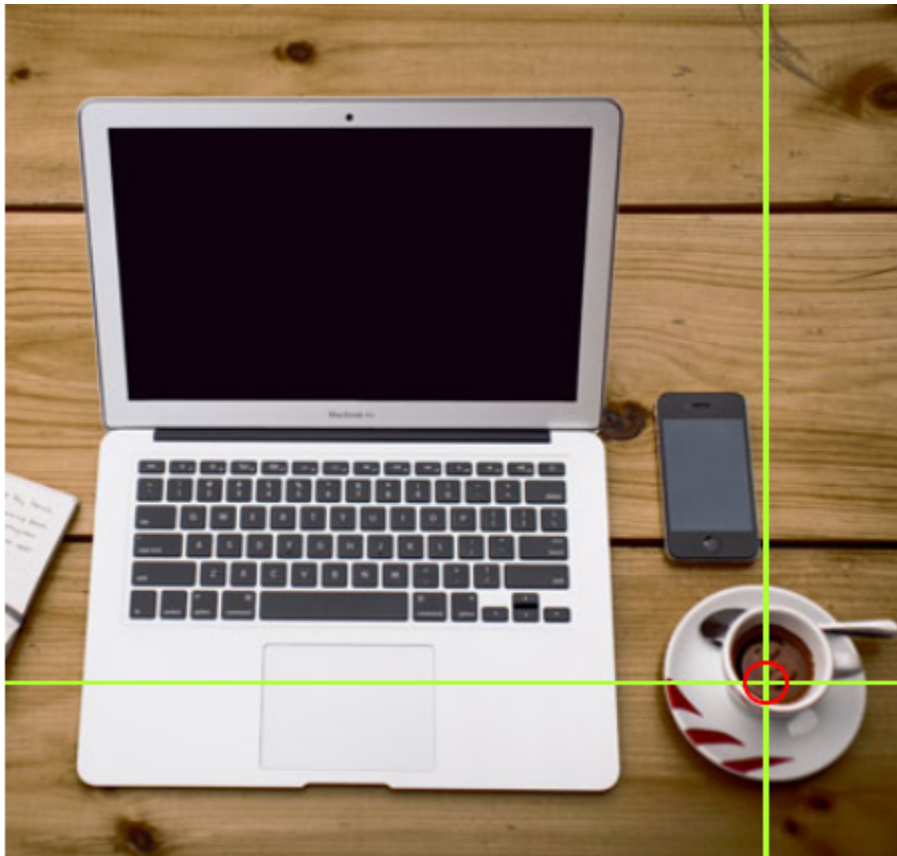
When the user clicks inside its area, they will go to "**computer.html**".



Shape="circle"

To add a circle area, first locate the coordinates of the center of the circle:

337,300



Then specify the radius of the circle:

44 pixels



Now you have enough data to create a clickable circular area:

Example

```
<area shape="circle" coords="337,300,44" href="coffee.htm">
```

Ts makes a **circular area** on the image clickable.

When the user clicks inside ts circle, they will be taken to **"coffee.htm"**.



Shape="poly"

The `shape="poly"` lets you define **many points**, connected by straight lines, to create a **polygonal area**.

Ts means you can outline **any custom shape**, like a croissant.

For example:

```
<area shape="poly"
coords="140,220,170,250,200,240,210,210,180,180,150,190"
href="croissant.htm">
```

Ts makes the drawn shape clickable, and takes the user to "**croissant.htm**" when clicked.

You can adjust the coordinates to match the outline of the croissant exactly.





We have to find the x and y coordinates for all edges of the croissant:



The coordinates come in pairs, one for the x-axis and one for the y-axis:

```
<area shape="poly"
```

```
coords="140,121,181,116,204,160,204,222,191,270,140,329,85,355,58,352,37,322,40,259,103,161,128,147" href="croissant.htm">
```

That creates a polygon area around the croissant.

When a user clicks inside the shape on the image, they will be taken to "croissant.htm".



Image Map and JavaScript

Example:

```
<map name="workmap">

    <area shape="circle" coords="337,300,44" href="coffee.htm"
onclick="myFunction()">

</map>


<script>

function myFunction() {

    alert("You clicked the coffee cup!");

}

</script>
```

It means when someone clicks on the **circle area**, it will:

- Go to "**coffee.htm**"
- Also show an **alert message** saying: *"You clicked the coffee cup!"*

Chapter Summary

- Use the `<map>` element to create an **image map**.
- Use the `<area>` element to mark **clickable areas** inside that map.
- Use the `use map` attribute on the `<img>` tag to **connect the image** to the image map.

HTML Image Tags

Tag	Description
<code></code>	Defines an image
<code><map></code>	Defines an image map
<code><area></code>	Defines a clickable area inside an image map
<code><picture></code>	Defines a container for multiple image resources

Background Images

Background Image on an HTML element

To add a background image on an HTML element, use the **style attribute** with the CSS **background-image property**.

Example

Add a background image on a `<p>` element:

```
<p style="background-image: url('img_girl.jpg');">  
Ts is a paragraph with a background image.  
</p>
```

You can also set the background image using the `<style>` element inside the `<head>`.

Example

Specify the background image in the `<style>` element:

```
<style>  
p {  
  background-image: url('img_girl.jpg');  
}  
</style>
```

Ts way, all `<p>` elements will have the background image.

Background Image on a Page

If you want the **whole page** to have a background image, you should set the background image on the `<body>` element.

Example

Add a background image for the entire page:

```
<style>

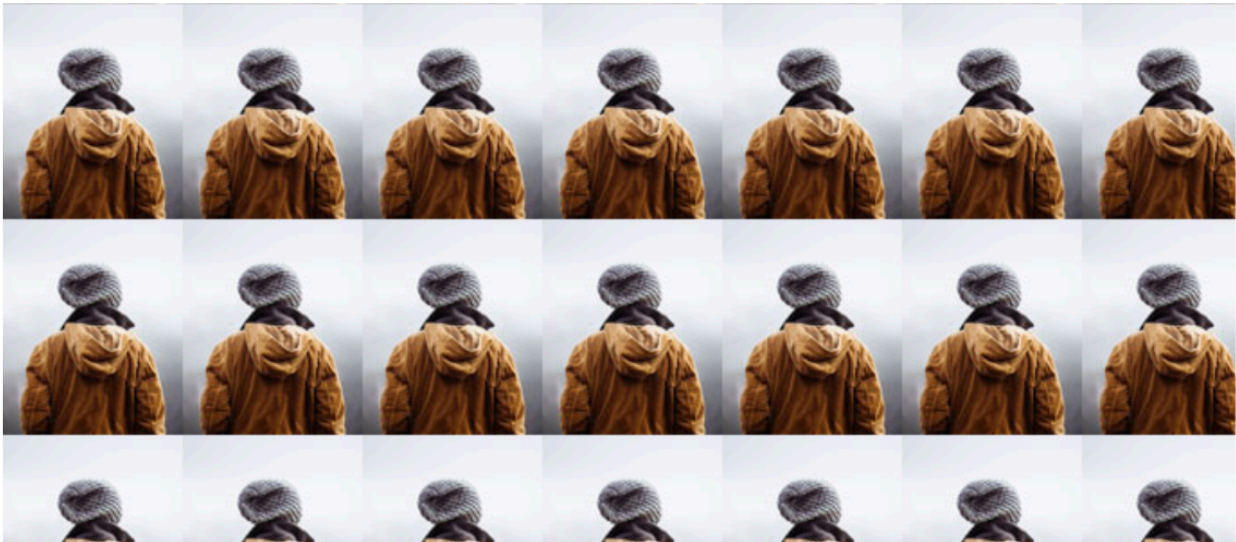
body {
  background-image: url('img_girl.jpg');
}

</style>
```

This will make the image appear as the background for the entire webpage.

Background Repeat

If the background image is **smaller than the element**, it will **repeat automatically** in both directions until it fills the element.



Example

```
<style>
body {
  background-image: url('example_img_girl.jpg');
}
</style>
```

To **stop the image from repeating**, use `background-repeat: no-repeat;`.

Example

```
<style>

body {
  background-image: url('example_img_girl.jpg');
  background-repeat: no-repeat;
```

```
}
```

```
</style>
```

Ths way, the image will **only appear once**.

Background Cover

If you want the **background image to fill the entire element**, use `background-size: cover;`.

To keep the image fixed when you scroll, add `background-attachment: fixed;`.

Ths way, the image will **cover the whole element without stretchng**, and keep its original shape.

Example

```
<style>
```

```
body {  
  background-image: url('img_girl.jpg');  
  background-repeat: no-repeat;  
  background-attachment: fixed;  
  background-size: cover;  
}
```

```
</style>
```

Background Stretch

If you want the **background image to stretch and exactly fit the whole element**, use:

```
background-size: 100% 100%;
```



It will **stretch the image** to fill the element completely (it may not keep original proportions).

Example

```
<style>

body {
  background-image: url('img_girl.jpg');
  background-size: 100% 100%;
}
```


</style>

Learn More CSS

From the examples above, you saw that you can **change how background images look** by using CSS background properties.

To learn more, check out the **CSS Background Tutorial**.

The Picture Images

HTML <picture> Element



The `<picture>` element in HTML lets developers have **more control over images**. It helps you show different images based on the **screen size** or **device type**.

How it works

- The `<picture>` element has one or more `<source>` tags.
- Each `<source>` tag points to a different image using `srcset`.
- Each `<source>` also has a `media` rule that says **when to use that image**.

The browser looks at these rules and **picks the best image for the screen**.

Example

Show different images for different screen sizes:

```
<picture>
  <source media="(min-width: 650px)" srcset="img_food.jpg">
  <source media="(min-width: 465px)" srcset="img_car.jpg">
  
</picture>
```

- If the screen is **650px or wider**, it shows `img_food.jpg`.
- If the screen is **465px or wider**, it shows `img_car.jpg`.
- If nothing matches, it shows `img_girl.jpg`.

Why always add `<img>` at the end?

- The `<img>` tag is a **backup**.
- Older browsers that do not understand `<picture>` will still show the image inside `<img>`.

When to use the Picture Element

1. Bandwidth :

Small devices get small images, big devices get big images. This means faster loading.

2. Format Support

Some browsers might not support a certain image format.
The browser uses the first image it understands.

Example with formats

```
<picture>
  <source srcset="img_avatar.png">
  <source srcset="img_girl.jpg">
  
</picture>
```

- The browser will try to use PNG first.
- If it does not support PNG, it will try JPG.
- If that fails too, it shows the GIF.

HTML Image Tags

Tag	Description
<u></u>	Defines an image
<u><map></u>	Defines an image map
<u><area></u>	Defines a clickable area inside an image map
<u><picture></u>	Defines a container for multiple image resources

HTML Favicon

How To Add a Favicon in HTML

You can use any image you like as your favicon. You can even make one on sites like <https://www.favicon.cc>.

Tip:

A favicon is very small, so keep it simple and high contrast.

A favicon shows up next to your page title in the browser tab.

How to add a favicon:

1 Save your favicon image in your website's root folder (or in a folder like `/images`).

A common name is `favicon.ico`.

2 Add a `<link>` tag inside your `<head>` section, after `<title>`, like this:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page Title</title>
  <link rel="icon" type="image/x-icon" href="/images/favicon.ico">
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>
```

```
</body>  
</html>
```

Save and reload in your browser.
Now you'll see your favicon in the browser tab!

Favicon File Format Support

The following table shows the file format support for a favicon image:

Browser	ICO	PNG	GIF	JPEG	SVG
Edge	Yes	Yes	Yes	Yes	Yes
Chrome	Yes	Yes	Yes	Yes	Yes
Firefox	Yes	Yes	Yes	Yes	Yes
Opera	Yes	Yes	Yes	Yes	Yes
Safari	Yes	Yes	Yes	Yes	Yes

Chapter Summary

- Use the HTML `<link>` element to insert a favicon

HTML Link Tag

Tag	Description
<u><link></u>	Defines the relationship between a document and an external resource

HTML Page Title

HTML Page Title

Every web page should have a page title to describe the meaning of the page.

The Title Element

The `<title>` element adds a title to your page

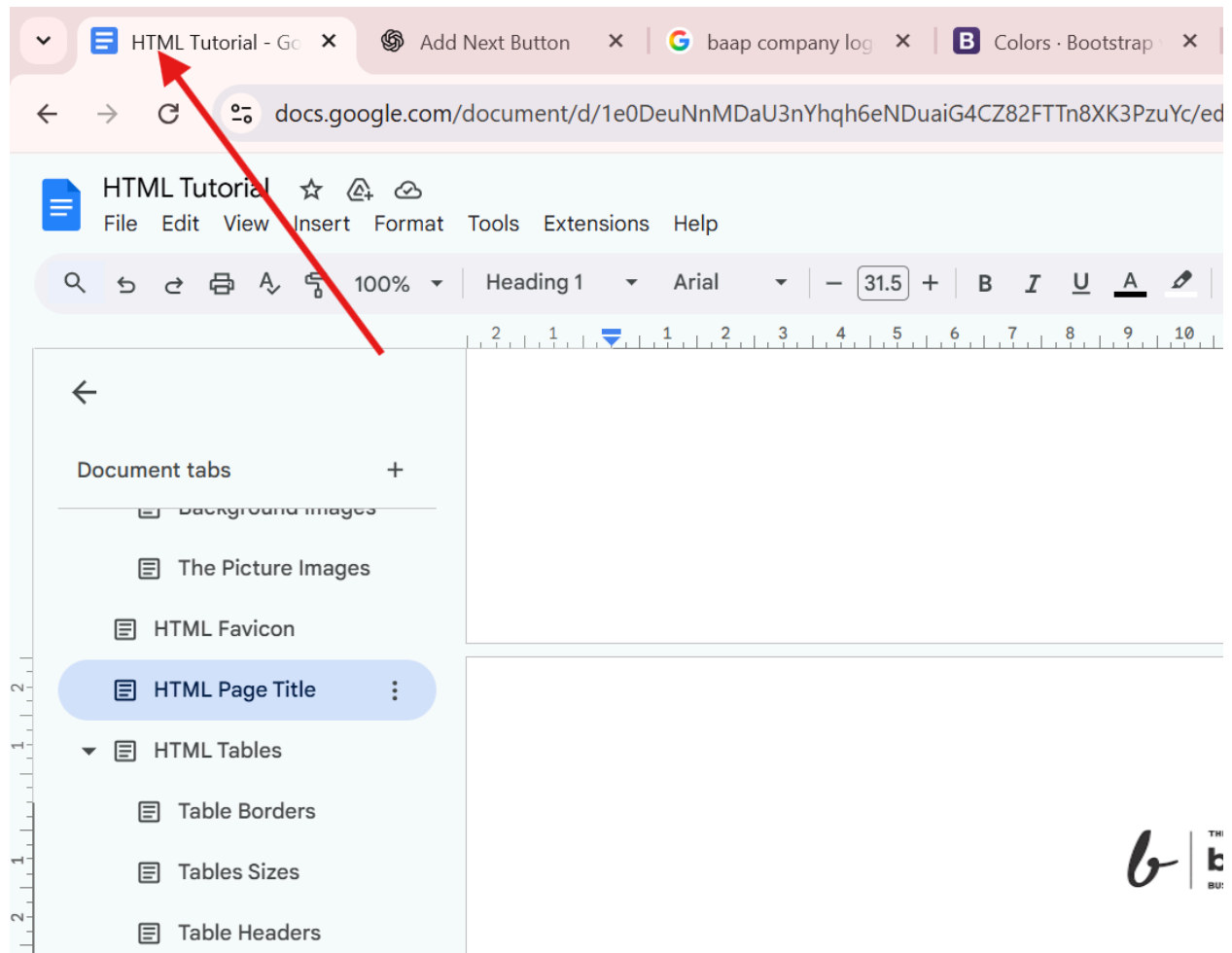
Example :

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Tutorial</title>
</head>
<body>
```

The content of the document.....

```
</body>
</html>
```

The title is shown in the browser's title bar:



What is a Good Title?

- It **describes** the page content clearly.
- It is **important for SEO** (Search Engine Optimization).
- Search engines use the title to rank pages.

The **<title>** Element:

- Shows in the **browser tab**
- Used when **bookmarking** a page
- Appears in **search results**

HTML Title Tag

Tag	Description
<u><code><title></code></u>	Defines the title of the document

HTML Tables

HTML Tables

HTML tables help you organize data neatly into rows and columns, like in a spreadsheet.

Example of a table:

Company	Contact	Country
Alfreds Futterkiste	Maria Anders	Germany
Centro comercial Moctezuma	Francisco Chang	Mexico
Ernst Handel	Roland Mendel	Austria
Island Trading	Helen Bennett	UK
Laugng Bacchus Winecellars	Yos Tannamuri	Canada
Magazzini Alimentari Riuniti	Giovanni Rovelli	Italy

It makes it easy to display information in a clear, grid-like format.

Define an HTML Table

A table in HTML is made of rows (`<tr>`), which contain cells.

There are header cells (`<th>`) for titles, and data cells (`<td>`) for content.

Example of a simple HTML table:

```
<table>
  <tr>
    <th>Company</th>
    <th>Contact</th>
    <th>Country</th>
  </tr>
  <tr>
    <td>Alfreds Futterkiste</td>
    <td>Maria Anders</td>
    <td>Germany</td>
  </tr>
  <tr>
    <td>Centro comercial Moctezuma</td>
    <td>Francisco Chang</td>
    <td>Mexico</td>
  </tr>
</table>
```

Ts builds a neat grid of data with headings on top.

Table Cells

Each table cell is made using the `<td>` tag.

`<td>` stands for **table data**.

Everything inside `<td> ... </td>` is the content of that cell.

Example:

```
<table>
```



```
<tr>
  <td>Emil</td>
  <td>Tobias</td>
  <td>Linus</td>
</tr>
</table>
```

Ts makes a row with three cells: Emil, Tobias, Linus.

Table Rows

Each table row is made with the `<tr>` tag.

`<tr>` stands for **table row**.

Example:

```
<table>
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
  <tr>
    <td>16</td>
    <td>14</td>
    <td>10</td>
  </tr>
</table>
```

You can add as many rows as you want.

Just try to keep the same number of cells in each row (most of the time).

Table Headers

Sometimes you want your cells to be **headings**, not just regular data.

For that, use the `<th>` tag instead of `<td>`.

`<th>` stands for **table header**.

Example:

```
<table border="1">
  <caption>Student Info</caption>
  <thead>
    <tr><th>Name</th><th>Marks</th></tr>
  </thead>
  <tbody>
    <tr><td>Amit</td><td>80</td></tr>
    <tr><td>Riya</td><td>90</td></tr>
  </tbody>
  <tfoot>
    <tr><td colspan="2">Total: 2 Students</td></tr>
  </tfoot>
</table>
```

Make the first row a header:

```
<table>
  <tr>
    <th>Person 1</th>
```

```
<th>Person 2</th>
  <th>Person 3</th>
</tr>
<tr>
  <td>Emil</td>
  <td>Tobias</td>
  <td>Linus</td>
</tr>
<tr>
  <td>16</td>
  <td>14</td>
  <td>10</td>
</tr>
</table>
```

By default, `<th>` text is **bold** and **centered**, but you can style it with CSS.

HTML Table Tags

Tag	Description
<u><table></u>	Defines a table
<u><th></u>	Defines a header cell in a table
<u><tr></u>	Defines a row in a table
<u><td></u>	Defines a cell in a table
<u><caption></u>	Defines a table caption
<u><colgroup></u>	Specifies a group of one or more columns in a table for formatting
<u><col></u>	Specifies column properties for each column within a <colgroup> element
<u><thead></u>	Groups the header content in a table
<u><tbody></u>	Groups the body content in a table
<u><tfoot></u>	Groups the footer content in a table

Table Borders

How To Add a Border

To add a border, use the CSS `border` property on `table`, `th`, and `td` elements:


```
table, th, td {  
  
  border: 1px solid black;  
  
}
```

Collapsed Table Borders

Ts CSS makes sure the table borders merge into a single border, so there are no double lines:


```
table, th, td {  
  border: 1px solid black;  
  border-collapse: collapse;  
}
```

`border-collapse: collapse;`

joins the adjacent borders together into a single border, giving a cleaner look.

Style Table Borders

TS CSS styles the table so that the cell borders blend with the page background, making them look invisible, while still separating cells with a background color:


```
table, th, td {  
  border: 1px solid wte;  
  border-collapse: collapse;  
}  
th, td {  
  background-color: #96D4D4;  
}
```

Thus way, the `wte` borders match the background of the page, and the cells are visually separated only by their `background-color`.

Round Table Borders

With rounded borders on the whole table and cells:


```
table, th, td {  
  border: 1px solid black;  
  border-radius: 10px;  
}
```

This makes the outer table border and each cell's border have rounded corners.

Rounded borders only on cells (not around the table):

```
th, td {  
  border: 1px solid black;  
  border-radius: 10px;  
}
```

This applies rounded corners just to each `<th>` and `<td>`, skipping the outer table border.

Dotted Table Borders

Example using `border-style`


```
th, td {  
  border-style: dotted;  
}
```

Allowed values for **border-style**:

- **dotted** → a series of dots
- **dashed** → a series of dashes
- **solid** → a solid line
- **double** → two solid lines
- **groove** → looks carved into the page
- **ridge** → looks like it is coming out of the page
- **inset** → looks embedded
- **outset** → looks like it is coming out
- **none** → no border
- **dden** → des the border (similar to **none**)

Try it with different styles:

```
th, td {  
  border-style: double;  
}
```

or

```
th, td {  
  border-style: ridge;  
}
```

It lets you control exactly how the borders look in your table.

Border Color

With the `border-color` property, you can set the color of the border.

Example

```
th, td {  
  border-color: #96D4D4;  
}
```

Tables Sizes

HTML Table Sizes

HTML tables can have different sizes for each column, row or the entire table.

Use the `style` attribute with the `width` or `height` properties to specify the size of a table, row or column.

HTML Table Width

To change how wide the table is, use the **style attribute** on the `<table>` tag.

Example

It makes the table take up **100% of the page width**:

```
<table style="width:100%">
  <tr>
    <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
```

```
</tr>
<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>
```

HTML Table Column Width

To change the width of a **specific column**, use the **style attribute** on the `<th>` or `<td>` tag.

Example

It makes the **first column 70% wide**:

```
<table style="width:100%">
  <tr>
    <th style="width:70%">Firstname</th>
```

```
<th>Lastname</th>
  <th>Age</th>
</tr>
<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>
```

HTML Table Row Height

To change the **height of a row**, use the **style attribute** on the `<tr>` tag.

Example

That makes the **second row 200 pixels tall**:

```
<table style="width:100%">
  <tr>
    <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr style="height:200px">
    <td>Jill</td>
    <td>Smith</td>
    <td>50</td>
  </tr>
  <tr>
    <td>Eve</td>
    <td>Jackson</td>
    <td>94</td>
  </tr>
</table>
```

Table Headers

HTML Table Headers

HTML tables can have headers for each column or row, or for many columns/rows.

EMIL	TOBIAS	LINUS

8:00		
9:00		
10:00		
11:00		
12:00		
13:00		

	MON	TUE	WED	THU	FRI
8:00					
9:00					
10:00					
11:00					
12:00					

DECEMBER		

HTML Table Headers

Table headers are written with **<th>** tags.

Each **<th>** is like a table cell, but it is used for **headings**.

Example

This table has a header row with three headings:

```
<table>
  <tr>
    <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
```

```
<td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>
```

Vertical Table Headers

If you want to make the **first column** as table headers, use the **<th>** tag in the first cell of each row.

Example

Here, the first column has the headings:

```
<table>
  <tr>
    <th>Firstname</th>
    <td>Jill</td>
    <td>Eve</td>
  </tr>
  <tr>
    <th>Lastname</th>
    <td>Smith</td>
    <td>Jackson</td>
  </tr>
</table>
```

```
<th>Age</th>
  <td>94</td>
  <td>50</td>
</tr>
</table>
```

Align Table Headers

By default, table headers are bold and centered:

Firstname	Lastname	Age
Jill	Smith	50
Eve	Jackson	94

If you want your **table headers** to be **left aligned**, use **CSS** with `text-align: left;`.

Example

```
th {
  text-align: left;
}
```

Header for Multiple Columns

You can make a table header that **covers two or more columns**.

Name		Age
Jill	Smith	50
Eve	Jackson	94

To do this, use the `colspan` attribute on the `<th>` element:

To make a header cover **two or more columns**, use the `colspan` attribute in the `<th>` tag.

Example:

```
<table>
  <tr>
    <th colspan="2">Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
    <td>Smith</td>
    <td>50</td>
  </tr>
  <tr>
    <td>Eve</td>
    <td>Jackson</td>
    <td>94</td>
  </tr>
</table>
```

Table Caption

You can add a **caption** to give a heading to the whole table.

Month	Savings
January	\$100
February	\$50

Example:

```
<table style="width:100%">
  <caption>Monthly savings</caption>
  <tr>
    <th>Month</th>
    <th>Savings</th>
  </tr>
  <tr>
    <td>January</td>
    <td>$100</td>
  </tr>
  <tr>
    <td>February</td>
    <td>$50</td>
  </tr>
</table>
```

Padding & Spacing

HTML Table Padding & Spacing

HTML tables can adjust the padding inside the cells, and also the space between the cells.

With Padding

hello	hello	hello
hello	hello	hello
hello	hello	hello

With Spacing

hello	hello	hello
hello	hello	hello
hello	hello	hello

HTML Table - Cell Padding

Cell padding means space inside the cell, between the cell border and the text.

By default, this space is **0**.

Add padding to all sides:

```
th, td {  
    padding: 15px;  
}
```

Ts adds **15px space** inside each cell.

Add padding to specific sides:

```
th, td {  
  padding-top: 10px;      /* space on top */  
  padding-bottom: 20px;   /* space at bottom */  
  padding-left: 30px;     /* space on left */  
  padding-right: 40px;    /* space on right */  
}
```

Colspan & Rowspan

HTML Table Colspan & Rowspan

HTML tables can have cells that span over multiple rows and/or columns.

NAME		

APRIL		

2022		
FIESTA		

HTML Table - Colspan

The **colspan** attribute makes a table cell stretch across multiple columns.

Example:

```
<table>
  <tr>
    <th colspan="2">Name</th> <!-- spans over 2 columns -->
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
```

```
<td>Smith</td>
  <td>43</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>57</td>
</tr>
</table>
```

HTML Table - Rowspan

The **rowspan** attribute lets you make a **table cell stretch over multiple rows** (go down

Example:

```
<table>
  <tr>
    <th>Name</th>
    <td>Jill</td>
  </tr>
  <tr>
    <th rowspan="2">Phone</th>
    <td>555-1234</td>
  </tr>
  <tr>
    <td>555-8745</td>
  </tr>
```

</table>

Table Styling

HTML Table - Zebra Stripes

If you add a background color on **every other table row**, you get a nice **zebra stripe effect**.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20

To style **every other row**, use the `:nth-child(even)` selector like ts:

Example

```
tr:nth-child(even) {  
  background-color: #D6EEEE;  
}
```

HTML Table - Vertical Zebra Stripes

To make **vertical zebra stripes**, style **every other column** instead of every other row.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16
17	18	19	20

Set `:nth-cld(even)` for the table cells like ts:

Example

```
td:nth-cld(even), th:nth-cld(even) {  
  background-color: #D6EEEE;  
}
```

Combine Vertical and Horizontal Zebra Stripes

You can **combine both styles** from above to get **stripes on every other row and every other column**.

If you use a **transparent color**, the stripes **overlap**, giving a nice pattern.

(table with overlapping stripes)

Use `rgba()` to set **transparent colors**.

Example

```
tr:nth-cld(even) {  
  background-color: rgba(150, 212, 212, 0.4);  
}  
  
th:nth-cld(even),  
td:nth-cld(even) {  
  background-color: rgba(150, 212, 212, 0.4);  
}
```

Horizontal Dividers

First Name	Last Name	Savings
Peter	Griffin	\$100
Lois	Griffin	\$150
Joe	Swanson	\$300

If you add a border **only at the bottom of each table row**, you'll get a table with **horizontal lines** separating the rows.

Use the `border-bottom` property on all `tr` elements to do ts.

Example

```
tr {  
  border-bottom: 1px solid #ddd;  
}
```

Hoverable Table

Use the `:hover` selector on `tr` to ghlight table rows on mouse over:

First Name	Last Name	Savings
Peter	Griffin	\$100
Lois	Griffin	\$150
Joe	Swanson	\$300

Example

```
tr:hover {
```

```
background-color: #D6EEEE;  
}
```

Table Colgroup

HTML Table Colgroup

If you want to **style the first two columns** of a table, use the `<colgroup>` and `<col>` elements.

MON	TUE	WED	THU	FRI	SAT	SUN
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28

-
- The `<colgroup>` tag is like a **container** for your column styles.
 - Each `<col>` inside it specifies **wch columns to style**.
 - The `span` attribute tells **how many columns** to style.
 - The `style` attribute says **what style to apply**.
-

Example

```
<table>
  <colgroup>
    <col span="2" style="background-color: #D6EEEE">
  </colgroup>
```

```
<tr>
  <th>MON</th>
  <th>TUE</th>
  <th>WED</th>
  <th>THU</th>

</tr>
</table>
```

Ts means:

- The **first two columns** will have a **light blue background**.

Note:

- `<colgroup>` must go **inside** `<table>`, before any `<thead>`, `<tbody>`, `<tr>`, etc.
- If there's a `<caption>`, it goes **before** `<colgroup>`.
- Only **some CSS properties** (like `background-color`, `width`) work on `<col>`

Legal CSS Properties

Allowed properties are:

- `width` → to set **column width**

- **visibility** → to **show or de** columns
- **background** → to set **background color or image**
- **border** → to set **borders on columns**

All other CSS properties will have no effect on your tables.

Multiple Col Elements

If you want to **style multiple columns with different styles**, just use **more than one** `<col>` inside the `<colgroup>`.

Example

```
<table>
  <colgroup>
    <col span="2" style="background-color: #D6EEEE">
    <col span="3" style="background-color: pink">
  </colgroup>
  <tr>
    <th>MON</th>
    <th>TUE</th>
    <th>WED</th>
    <th>THU</th>
    <!-- etc. -->
  </tr>
  <!-- more rows -->
</table>
```

Empty Colgroups

If you want to **style columns in the middle of a table**, add **empty <col> elements** (with **no style**) for the columns **before them**.

Example

```
<table>

  <colgroup>

    <col span="3"> <!-- first 3 columns, no style -->

    <col span="2" style="background-color: pink"> <!-- next 2 columns
    styled -->

  </colgroup>

  <tr>

    <th>MON</th>

    <th>TUE</th>

    <th>WED</th>

    <th>THU</th>

    <th>FRI</th>

  </tr>

</table>
```

Hide Columns

You can **de columns** in a table by using the **visibility: collapse** property.

Example

```
<table>

  <colgroup>

    <col span="2">

    <col span="3" style="visibility: collapse">

  </colgroup>

  <tr>

    <th>MON</th>

    <th>TUE</th>

    <th>WED</th>

    <th>THU</th>

    <th>FRI</th>

  </tr>

</table>
```

HTML Lists

Lists

HTML Lists:-

Lists in HTML

Ordered List	Unordered List	Description List
1. Information Gathering 2. Planning 3. Design 4. Development 5. Testing & Deployment 6. Maintenance	▪ HTML ▪ CSS ◦ SQL ◦ PHP • JavaScript • Python	HTML HTML stands for Hyper Text Markup Language. HTML is the standard markup language for creating Web pages. HTML describes the structure of a Web page. CSS CSS is the acronym of "Cascading Style Sheets". CSS is the language use to style an HTML document. CSS describes how HTML elements should be displayed. PHP PHP is an acronym for "PHP: Hypertext Preprocessor". PHP is a server scripting language, and a powerful tool for making dynamic and interactive Web pages.

HTML **lists** let web developers **group related items together** on a web page.

Example

An unordered HTML list:

- Item
- Item
- Item
- Item

An ordered HTML list:

1. First item
2. Second item
3. Third item
4. Fourth item

Unordered HTML List

An **unordered list** uses the `<ul>` tag.

Each item inside the list uses the `<li>` tag.

By default, the items show up with **bullets (•)**.

Example

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Ordered HTML List

An **ordered list** starts with the `<ol>` tag.

Each item inside the list uses the `<li>` tag.

By default, the items are shown with **numbers**.

Example

```
<ol>
  <li>Coffee</li>
  <li>Tea</li>
```

```
<li>Milk</li>
</ol>
```

HTML Description Lists

HTML also supports **description lists**.

A **description list** is a list of **terms and their descriptions**.

- The **<dl>** tag starts the description list.
- The **<dt>** tag is for the **term (name)**.
- The **<dd>** tag is for the **description** of that term

Example

```
<dl>
<dt>Coffee</dt>
<dd>- black hot drink</dd>
<dt>Milk</dt>
<dd>- wte cold drink</dd>
</dl>
```

HTML List Tags

Tag	Description
<u></u>	Defines an unordered list
<u></u>	Defines an ordered list
<u></u>	Defines a list item
<u><dl></u>	Defines a description list
<u><dt></u>	Defines a term in a description list
<u><dd></u>	Describes the term in a description list

Unordered Lists

Unordered HTML List

An **unordered list** starts with the `<ul>` tag.

Each item inside it uses the `<li>` tag.

By default, the items will show with **bullets (•)**.

Example

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Unordered HTML List - Choose List Item Marker

The CSS `list-style-type` property is used to **change the style of the list bullets or numbers**.

Value	Description
disc	Sets the list item marker to a bullet (default)
circle	Sets the list item marker to a circle
square	Sets the list item marker to a square
none	The list items will not be marked

Disc

The **disc** is the **default bullet style** — a **solid round dot**.

Example

```
<ul style="list-style-type:disc;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

Circle

Circle Example

```
<ul style="list-style-type:circle;">
  <li>Coffee</li>
```

```
<li>Tea</li>

<li>Milk</li>

</ul>
```

Square

Example - Square

```
<ul style="list-style-type:square;">

  <li>Coffee</li>

  <li>Tea</li>

  <li>Milk</li>

</ul>
```

No List Marker

Example - None

```
<ul style="list-style-type:none;">

  <li>Coffee</li>

  <li>Tea</li>

  <li>Milk</li>

</ul>
```

Nested HTML Lists

Lists can be nested (list inside list):

Example

```
<ul>

  <li>Coffee</li>

  <li>Tea

    <ul>

      <li>Black tea</li>

      <li>Green tea</li>

    </ul>

  </li>

  <li>Milk</li>

</ul>
```

Horizontal List with CSS

HTML lists can be **styled in many ways using CSS**.

One popular use is to **make a horizontal list** for a **navigation menu**.

Example

```
<!DOCTYPE html>

<html>

<head>

<style>

ul {

    list-style-type: none;    /* remove bullets */

    margin: 0;

    padding: 0;

    overflow: hidden;

    background-color: #333333; /* dark background */

}

li {

    float: left; /* place list items side by side */

}


```

```
li a {  
    display: block;  
    color: wte;  
    text-align: center;  
    padding: 16px;  
    text-decoration: none; /* remove underline */  
}  
  
li a:hover {  
    background-color: #111111; /* darker on hover */  
}  
  
</style>  
</head>  
<body>  
  
<ul>  
    <li><a href="#home">Home</a></li>  
    <li><a href="#news">News</a></li>  
    <li><a href="#contact">Contact</a></li>
```



```
<li><a href="#about">About</a></li>

</ul>

</body>

</html>
```

Chapter Summary

- Use the HTML `<ul>` element to define an unordered list
- Use the CSS `list-style-type` property to define the list item marker
- Use the HTML `<li>` element to define a list item
- Lists can be nested
- List items can contain other HTML elements
- Use the CSS property `float:left` to display a list horizontally

HTML List Tags

Tag	Description
<u></u>	Defines an unordered list
<u></u>	Defines an ordered list
<u></u>	Defines a list item
<u><dl></u>	Defines a description list
<u><dt></u>	Defines a term in a description list
<u><dd></u>	Describes the term in a description list

Ordered Lists

Ordered HTML List

An **ordered list** uses the `<ol>` tag.

Each item inside it uses the `<li>` tag.

By default, the list items are shown with **numbers**.

Example

```
<ol>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Ordered HTML List - The Type Attribute

The `type` attribute of the `<ol>` tag, defines the type of the list item marker:

Type	Description
type="1"	The list items will be numbered with numbers (default)
type="A"	The list items will be numbered with uppercase letters
type="a"	The list items will be numbered with lowercase letters
type="I"	The list items will be numbered with uppercase roman numbers
type="i"	The list items will be numbered with lowercase roman numbers

Numbers

```
<ol type="1">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Uppercase Letters

Example

```
<ol type="A">  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

Lowercase Letters

Example

```
<ol type="a">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Roman Numbers - Uppercase

Example

```
<ol type="I">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Roman Numbers - Lowercase

Lowercase Roman Numbers

Example

```
<ol type="i">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Control List Counting

By default, an **ordered list** will **start from 1**.

If you want it to start from a different number, use the **start** attribute.

Example

```
<ol start="50">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>
```

Nested HTML Lists

Lists can be nested (list inside list):

Example

```
<ol>
  <li>Coffee</li>
  <li>Tea
    <ol>
      <li>Black tea</li>
      <li>Green tea</li>
    </ol>
  </li>
  <li>Milk</li>
</ol>
```

Chapter Summary

- Use the HTML `<ol>` element to define an ordered list
- Use the HTML `type` attribute to define the numbering type

- Use the HTML `<li>` element to define a list item
- Lists can be nested
- List items can contain other HTML elements

HTML List Tags

Tag	Description
<u><code></code></u>	Defines an unordered list
<u><code></code></u>	Defines an ordered list
<u><code></code></u>	Defines a list item
<u><code><dl></code></u>	Defines a description list
<u><code><dt></code></u>	Defines a term in a description list
<u><code><dd></code></u>	Describes the term in a description list

Other Lists

Other Lists

A **description list** is a list that has **terms** and a **description for each term**.

- Use the **<dl>** tag to start the description list.
- Use the **<dt>** tag for the **term (name)**.
- Use the **<dd>** tag for the **description**.

Example

```
<dl>
  <dt>Coffee</dt>
  <dd>- black hot drink</dd>
  <dt>Milk</dt>
  <dd>- wte cold drink</dd>
</dl>
```

Chapter Summary

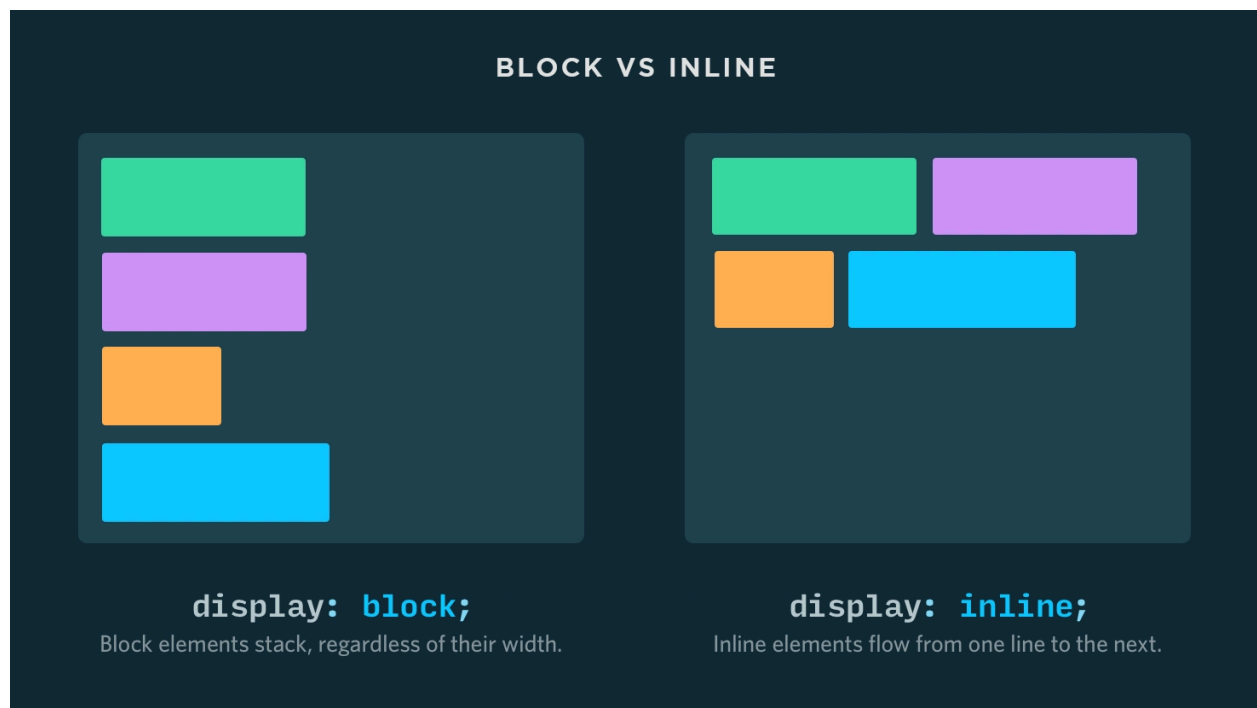
- Use the HTML **<dl>** element to define a description list
- Use the HTML **<dt>** element to define the description term
- Use the HTML **<dd>** element to describe the term in a description list

HTML List Tags

Tag	Description
<u></u>	Defines an unordered list
<u></u>	Defines an ordered list
<u></u>	Defines a list item
<u><dl></u>	Defines a description list
<u><dt></u>	Defines a term in a description list
<u><dd></u>	Describes the term in a description list

HTML Block & Inline

HTML Block and Inline Elements



Every HTML element has a **default display type**, based on what kind of element it is.

The two most common types are:

- **block**
- **inline**

Block-level Elements

- A **block-level element** always starts on a **new line**.
- The browser automatically adds **some space (margin)** before and after it.
- It always takes up the **full width available**, stretching from left to right.

Some common **block-level elements** are:

- `<p>` — defines a **paragraph**.
- `<div>` — defines a **division or section**.

Both are **block-level elements**.

The `<p>` element is a block-level element.

The `<div>` element is a block-level element.

Example

```
<p>Hello World</p>
<div>Hello World</div>
```

List of block-level elements

```
<address> <article> <aside> <blockquote> <canvas> <dd> <div>
<dl> <dt> <fieldset> <figcaption> <figure> <footer> <form>
<h1>-<h6> <header> <hr> <li> <main> <nav> <noscript>
<ol> <p> <pre> <section> <table> <tfoot> <ul> <video>
```

Inline Elements

- An inline element does not start on a new line.
- It only takes up as much width as it needs.

Example

```
<span>Hello World</span>
```

Its `<span>` is inside a paragraph, and it stays on the same line.

List of inline elements

```
<a> <abbr> <acronym> <b> <bdo> <big> <br> <button> <cite>  
<code> <dfn> <em> <i> <img> <input> <kbd> <label> <map>  
<object> <output> <q> <samp> <script> <select> <small>  
<span> <strong> <sub> <sup> <textarea> <time> <tt> <var>
```

The `<div>` Element

The `<div>` element is often used as a container to hold other HTML elements.

- The `<div>` tag does not need any required attributes, but it is commonly used with `style`, `class`, or `id`.

When you use `<div>` with CSS, you can style blocks of content easily.

Example

```
<div style="background-color:black;color:white;padding:20px;">
  <h2>London</h2>
  <p>London is the capital city of England. It is the most populous
city in the United Kingdom, with a metropolitan area of over 13
million inhabitants.</p>
</div>
```

It makes a **black box with white text and padding**.

The `<span>` Element

The `<span>` element is an **inline container**.

It is used to **mark or style a part of text**, or a small part of a document.

The `<span>` tag does **not need any required attributes**,
but it is **commonly used with `style`, `class`, or `id`**.

When you use `<span>` with CSS, you can **style only small parts of the text**.

Example

```
<p>My mother has <span
style="color:blue;font-weight:bold;">blue</span> eyes and my father
has <span style="color:darkolivegreen;font-weight:bold;">dark
green</span> eyes.</p>
```

Chapter Summary

- A block-level element always starts on a new line and takes up the full width available
- An inline element does not start on a new line and it only takes up as much width as necessary
- The `<div>` element is a block-level element and is often used as a container for other HTML elements
- The `<span>` element is an inline container used to mark up a part of a text, or a part of a document

```
<!DOCTYPE html>
<html>
<head>
  <title>Block vs Inline</title>
</head>
<body>

  <!-- Block Elements -->
  <h2>Block Elements</h2>
  <div>This is a div (block)</div>
  <p>This is a paragraph (block)</p>
  <ul>
    <li>List Item 1 (block)</li>
    <li>List Item 2 (block)</li>
  </ul>

  <!-- Inline Elements -->
  <h2>Inline Elements</h2>
  <p>This is a <span>span</span> and an <a href="#">anchor</a> inside a paragraph.</p>
  <p><strong>Bold</strong> and <em>Italic</em> text are inline too.</p>

</body>
</html>
```

HTML Tags

Tag	Description
<u><div></u>	Defines a section in a document (block-level)
<u></u>	Defines a section in a document (inline)

HTML Div

The <div> Element

The `<div>` element is by default a block element, meaning that it takes all available width, and starts on a new line, with line breaks before and after.

Example

A `<div>` element takes up all available width:

```
Lorem Ipsum <div>I am a div</div> dolor sit amet.
```

Result

```
Lorem Ipsum
I am a div
dolor sit amet.
```

The `<div>` element does not need any special attributes, but `style`, `class`, and `id` are commonly used

<div> as a container

The `<div>` element is often used to **group parts of a web page together**.

Example

A `<div>` element with some HTML elements inside:

```
<div>
  <h2>London</h2>
  <p>London is the capital city of England.</p>
  <p>London has over 9 million inhabitants.</p>
</div>
```

Result

London

London is the capital city of England.

London has over 9 million inhabitants.

Center align a <div> element

If you have a `<div>` element that is **not full width**, and you want to **center it**, set the CSS `margin` property to `auto`.

Example

```
<style>
div {
  width: 300px;
  margin: auto;
}
</style>
```

Result

London

London is the capital city of England.

London has over 9 million inhabitants.

Multiple <div> elements

You can have many **<div>** containers on the same page.

Example

```
<div>
  <h2>London</h2>
  <p>London is the capital city of England.</p>
  <p>London has over 9 million inhabitants.</p>
</div>
```

```
<div>
  <h2>Oslo</h2>
  <p>Oslo is the capital city of Norway.</p>
  <p>Oslo has over 700,000 inhabitants.</p>
</div>
```

```
<div>
```

```
<h2>Rome</h2>
  <p>Rome is the capital city of Italy.</p>
  <p>Rome has over 4 million inhabitants.</p>
</div>
```

Result

London

London is the capital city of England.

London has over 9 million inhabitants.

Oslo

Oslo is the capital city of Norway.

Oslo has over 700,000 inhabitants.

Rome

Rome is the capital city of Italy.

Rome has over 4 million inhabitants.

Aligning <div> elements side by side

When making web pages, you often want to put **two or more `<div>` elements next to each other**, like ts:

Result

London	Oslo	Rome
London is the capital city of England.	Oslo is the capital city of Norway.	Rome is the capital city of Italy.
London has over 9 million inhabitants.	Oslo has over 700,000 inhabitants.	Rome has over 4 million inhabitants.

There are **different ways** to place elements **side by side**.

All of them use **some CSS styling**.

We will now look at the **most common methods**

Float

The CSS `float` property was **not originally made to place `<div>` elements side by side**, but people have used it like that for many years.

The CSS `float` property is used for **positioning and formatting content**, and it lets elements be placed **horizontally (next to each other)** instead of vertically (on top of each other).

Example

How to use `float` to align `<div>` elements side by side:


```
<style>
.mycontainer {
  width: 100%;
  overflow: auto;
}
.mycontainer div {
  width: 33%;
  float: left;
}
</style>
```

London

London is the capital city of England.

London has over 9 million inhabitants.

Oslo

Oslo is the capital city of Norway.

Oslo has over 700,000 inhabitants.

Rome

Rome is the capital city of Italy.

Rome has over 4 million inhabitants.

Inline-block

If you change the `<div>` element's `display` property from `block` to `inline-block`, the `<div>` elements will **not add line breaks before and after anymore**.

They will be shown **side by side**, instead of on top of each other.

Example

How to use `display: inline-block` to align `<div>` elements side by side:

```
<style>
div {
  width: 30%;
  display: inline-block;
}
</style>
```

Result



Flex

The CSS **Flexbox Layout Module** was created to make it **easier to build flexible, responsive layouts** without using `float` or `position`.

To make the CSS flex method work, you **wrap the `<div>` elements inside another `<div>`** and set that outer `<div>` as a **flex container**.

Example

How to use `flex` to align `<div>` elements side by side:

```
<style>
.mycontainer {
  display: flex;
}
.mycontainer > div {
  width: 33%;
}
</style>
```

Result

London	Oslo	Rome
London is the capital city of England.	Oslo is the capital city of Norway.	Rome is the capital city of Italy.
London has over 9 million inhabitants.	Oslo has over 700,000 inhabitants.	Rome has over 4 million inhabitants.

Grid

The CSS **Grid Layout Module** gives a **grid-based system** with rows and columns. It makes it easier to design web pages without using `float` or `position`.

It sounds almost the same as flex, but **grid lets you create many rows and place each row where you want.**

The CSS grid method needs you to **wrap the `<div>` elements inside another `<div>`** and set that outer `<div>` as a **grid container**.

You also have to **set the width of each column**.

Example

How to use `grid` to align `<div>` elements side by side:

```
<style>
.grid-container {
  display: grid;
  grid-template-columns: 33% 33% 33%;
}
</style>
```

Result

London	Oslo	Rome
London is the capital city of England.	Oslo is the capital city of Norway.	Rome is the capital city of Italy.
London has over 9 million inhabitants.	Oslo has over 700,000 inhabitants.	Rome has over 4 million inhabitants.

HTML Tags

Tag	Description
<u><div></u>	Defines a section in a document (block-level)

HTML classes

The class Attribute

The `class` attribute is often used to **connect to a class name in a style sheet**.

It can also be used by **JavaScript to find and change elements** with that class name.

In the following example, we have **three `<div>` elements** with a `class` attribute set to `"city"`.

All three `<div>` elements will be styled the same way, because of the `.city` style in the `<head>` section.

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
.city {
  background-color: tomato;
  color: wte;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
}
</style>
</head>
<body>

<div class="city">
  <h2>London</h2>
  <p>London is the capital of England.</p>
</div>
```

```
<div class="city">
  <h2>Paris</h2>
  <p>Paris is the capital of France.</p>
</div>
```

```
<div class="city">
  <h2>Tokyo</h2>
  <p>Tokyo is the capital of Japan.</p>
</div>
```

```
</body>
</html>
```

In the next example, we have **two** **** elements with a **class** attribute set to "note".

Both **** elements will be styled the same way, because of the **.note** style in the **<head>** section.

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
.note {
  font-size: 120%;
  color: red;
}
</style>
</head>
<body>

<h1>My <span class="note">Important</span> Heading</h1>
```



```
<p>Ts is some <span class="note">important</span> text.</p>
```

```
</body>  
</html>
```

Tip: The `class` attribute can be used on **any HTML element**.

Note: The class name is **case sensitive**!

Tip: You can learn much more about CSS in our **CSS Tutorial**.

The Syntax For Class

To create a class, write a **dot (.)**, then the **class name**, and define the CSS properties inside **curly braces {}**.

Example

Create a class named "`city`":

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<style>  
  
.city {  
  
    background-color: tomato;
```

```
color: wte;

padding: 10px;
}

</style>

</head>

<body>

<h2 class="city">London</h2>

<p>London is the capital of England.</p>

<h2 class="city">Paris</h2>

<p>Paris is the capital of France.</p>

<h2 class="city">Tokyo</h2>

<p>Tokyo is the capital of Japan.</p>

</body>

</html>
```

Multiple Classes

HTML elements can belong to **more than one class**.

To set multiple classes, write the class names **with a space between them**, like ts:

```
<div class="city main">
```

The element will get the **styles from all the classes** listed.

In the following example, the first `<h2>` element belongs to both the `city` class and the `main` class, so it will get the CSS styles from **both classes**.

Example

```
<h2 class="city main">London</h2>
```

```
<h2 class="city">Paris</h2>
```

```
<h2 class="city">Tokyo</h2>
```

Different Elements Can Share Same Class

Different HTML elements can use the **same class name**.

In the following example, both `<h2>` and `<p>` use the `"city"` class and will get the **same style**.

Example

```
<h2 class="city">Paris</h2>
```

```
<p class="city">Paris is the capital of France</p>
```

Use of the class Attribute in JavaScript

The class name can also be used by **JavaScript** to do **certain actions** on specific elements.

JavaScript can find elements with a specific class name using the `getElementsByClassName()` method.

Example

Click on a button to de all elements with the class name "city":

```
<script>

function myFunction() {

    var x = document.getElementsByClassName("city");

    for (var i = 0; i < x.length; i++) {

        x[i].style.display = "none";

    }

}

</script>
```

Don't worry if you don't understand the code above.

You will learn more about JavaScript in our **HTML JavaScript chapter**, or you can check our **JavaScript Tutorial**.

Chapter Summary

- The HTML `class` attribute sets **one or more class names** for an element.
- Classes are used by **CSS and JavaScript** to find and work with specific elements.
- The `class` attribute can be used on **any HTML element**.
- The class name is **case sensitive** (capital and small letters matter).
- Different HTML elements can use the **same class name**.
- JavaScript can find elements with a specific class name using the `getElementsByClassName()` method.

HTML Id

The id Attribute

The **id** attribute sets a **unique id** for an HTML element.

The value of the **id** must be **unique in the whole HTML document**.

The **id** attribute is used to point to a **specific style** in a style sheet.

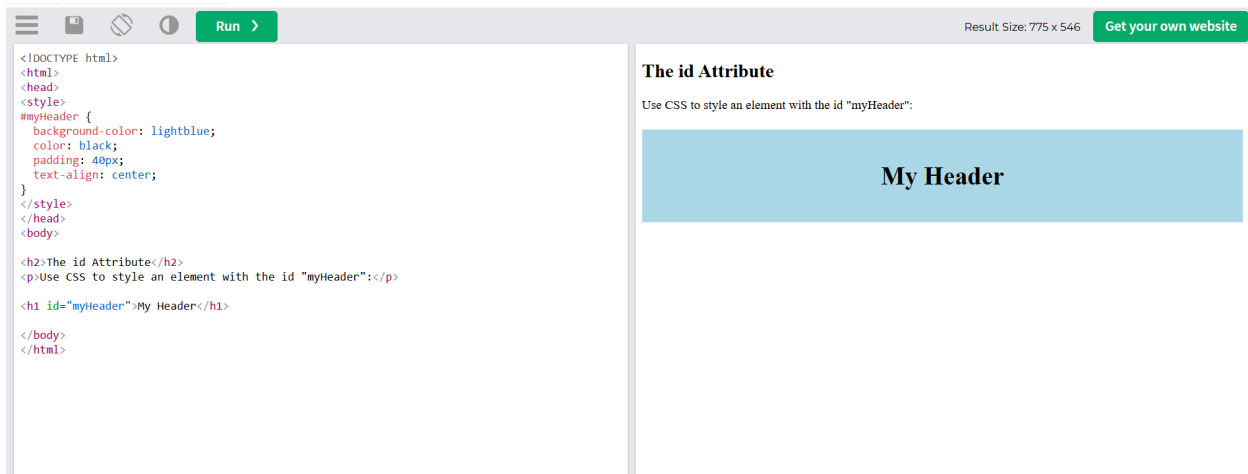
It is also used by **JavaScript** to **find and work with the element** that has that id.

The syntax for **id** is: write a **hash #**, followed by the id name.

Then, set the CSS properties inside **curly braces {}**.

In the following example, we have an **<h1>** element that uses the id name **"myHeader"**.

Its **<h1>** will be styled by the **#myHeader** style in the **<head>** section.



```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>

<h2>The id Attribute</h2>
<p>Use CSS to style an element with the id "myHeader":</p>

<h1 id="myHeader">My Header</h1>

</body>
</html>
```

The id Attribute

Use CSS to style an element with the id "myHeader":

My Header

Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    #main-title {
      color: blue;
      font-size: 30px;
    }
  </style>
</head>
<body>

  <h1 id="main-title">Welcome to My Website</h1>
  <p id="intro">This is the introduction paragraph.</p>
  <a href="#main-title">Go to Title</a>

  <script>
    document.getElementById('main-title').style.backgroundColor = 'lightyellow';
  </script>

</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
<style>
#myHeader {
  background-color: lightblue;
  color: black;
  padding: 40px;
  text-align: center;
}
</style>
</head>
<body>
```



```
<h1 id="myHeader">My Header</h1>
```

```
</body>
```

```
</html>
```

Difference Between Class and ID

A **class name** can be used by **many HTML elements**,
while an **id name** must be used by **only one HTML element** on the page.

Example

```
<style>
/* Style the element with the id "myHeader" */
#myHeader {
    background-color: lightblue;
    color: black;
    padding: 40px;
    text-align: center;
}

/* Style all elements with the class name "city" */
.city {
    background-color: tomato;
    color: white;
    padding: 10px;
}
</style>
```

```
<!-- An element with a unique id -->
<h1 id="myHeader">My Cities</h1>

<!-- Multiple elements with same class -->
<h2 class="city">London</h2>
<p>London is the capital of England.</p>

<h2 class="city">Paris</h2>
<p>Paris is the capital of France.</p>

<h2 class="city">Tokyo</h2>
<p>Tokyo is the capital of Japan.</p>
```

HTML Bookmarks with ID and Links

HTML **bookmarks** let readers **jump to specific parts of a webpage**.

Bookmarks are helpful if your page is **very long**.

To use a bookmark, you must first **create it**, then add a **link to it**.

When the link is clicked, the page will **scroll to the place with the bookmark**.

Example

First, create a bookmark using the **id** attribute:

```
<h2 id="C4">Chapter 4</h2>
```

Then, add a link to the bookmark ("Jump to Chapter 4") from the **same page**:

Example

```
<a href="#C4">Jump to Chapter 4</a>
```

Or, add a link to the bookmark ("Jump to Chapter 4") from **another page**:

```
<a href="html_demo.html#C4">Jump to Chapter 4</a>
```

Using the id Attribute in JavaScript

The `id` attribute can also be used by JavaScript to do something with that specific element.

JavaScript can find an element with a specific id using the `getElementById()` method.

Example

Use the `id` attribute to change text with JavaScript:

```
<script>
function displayResult() {
    document.getElementById("myHeader").innerHTML = "Have a nice day!";
}
</script>
```

Chapter Summary

The **id** attribute is used to **set a unique id** for an HTML element.

The value of the **id** must be **unique in the whole HTML document**.

The **id** is used by **CSS and JavaScript** to **style or select a specific element**.

The value of the **id** is **case sensitive**.

The **id** is also used to **create HTML bookmarks**.

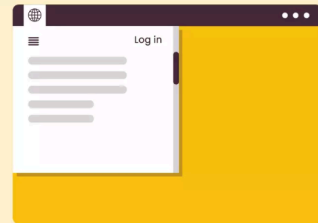
JavaScript can find an element with a specific id using the **getElementById()** method.

HTML Iframes

HTML Iframes

iFrame HTML

Explore what is HTML Iframes. Learn about the attributes associated with Iframes to integrate dynamic content into your website.



An HTML iframe is used to display a web page with a web page.

HTML Iframe Syntax

The HTML `<iframe>` tag defines an **inline frame**.

An inline frame is used to **show another document inside the current HTML page**.

Syntax

```
<iframe src="url" title="description"></iframe>
```

Tip: It is a good practice to always add a **title attribute** to the `<iframe>`.
It helps **screen readers** tell what the iframe content is.

Iframe - Set Height and Width

Use the **height** and **width** attributes to set the size of the iframe.

The height and width are by default given in **pixels**.

Example

```
<iframe src="demo_iframe.htm" height="200" width="300" title="Iframe Example"></iframe>
```

Or you can use the **style** attribute with CSS **height** and **width** properties:

Example

```
<iframe src="demo_iframe.htm" style="height:200px;width:300px;" title="Iframe Example"></iframe>
```

Iframe - Remove the Border

By default, an iframe has a **border around it**.

To **remove the border**, add the **style** attribute and use the CSS **border** property:

Example

```
<iframe src="demo_iframe.htm" style="border:none;" title="Iframe Example"></iframe>
```

With CSS, you can also **change the size, style, and color** of the iframe's border:

Example

```
<iframe src="demo_iframe.htm" style="border:2px solid red;" title="Iframe Example"></iframe>
```

Iframe - Target for a Link

An iframe can be used as the **target frame for a link**.

The **target** attribute of the link must match the **name** attribute of the iframe.

Example

html

```
<iframe src="demo_iframe.htm" name="iframe_a" title="Iframe Example"></iframe>
```

```
<p><a href="https://www.w3schools.com" target="iframe_a">W3Schools.com</a></p>
```

Chapter Summary

- The HTML `<iframe>` tag defines an **inline frame**.
- The `src` attribute sets the **URL of the page to show**.
- Always include a `title` attribute (**for screen readers**).
- The `height` and `width` attributes set the **size of the iframe**.
- Use `border:none;` to **remove the border around the iframe**.

HTML JavaScript

HTML JavaScript

JavaScript makes HTML pages more **dynamic and interactive**.

Example

My First JavaScript

Click me to display Date and Time

Wed Jul 16 2025 11:38:31 GMT+0530 (India Standard Time)

The HTML `<script>` Tag

The HTML `<script>` tag is used to write a **client-side script (JavaScript)**.

The `<script>` element either has **script statements inside it**, or points to an **external script file** using the `src` attribute.

Common uses for JavaScript are **image changes, checking forms, and updating content dynamically**.

To select an HTML element, JavaScript most often uses the `document.getElementById()` method.

Js JavaScript example writes "Hello JavaScript!" into an HTML element with `id="demo"`:

Example

```
<script>  
document.getElementById("demo").innerHTML = "Hello JavaScript!";  
</script>
```

A Taste of JavaScript

Here are some examples of what **JavaScript** can do:

Example

JavaScript can **change content**:

```
document.getElementById("demo").innerHTML = "Hello JavaScript!";
```

Example

JavaScript can **change styles**:

```
document.getElementById("demo").style.fontSize = "25px";  
document.getElementById("demo").style.color = "red";  
document.getElementById("demo").style.backgroundColor = "yellow";
```

Example

JavaScript can **change attributes**:

```
document.getElementById("image").src = "picture.gif";
```

The HTML < noscript> Tag

The HTML `<noscript >` tag defines **alternate content** for users who have **disabled scripts** in their browser or use a browser that **doesn't support scripts**.

Example

```
<script>
document.getElementById("demo").innerHTML = "Hello JavaScript!";
</script>
<noscript/>Sorry, your browser does not support JavaScript!</noscript>
```

HTML Script Tags

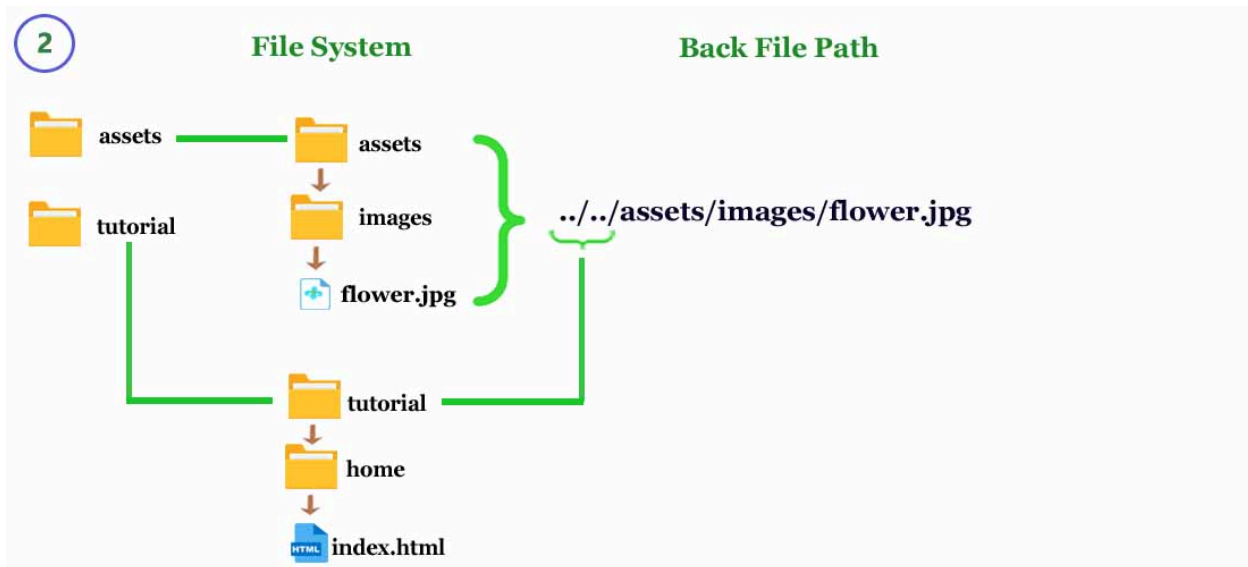
Tag	Description
<u><code><script></code></u>	Defines a client-side script
<u><code><noscript></code></u>	Defines an alternate content for users that do not support client-side scripts

HTML File Paths

File Path Examples

Path	Description
<code></code>	The "picture.jpg" file is located in the same folder as the current page
<code></code>	The "picture.jpg" file is located in the images folder in the current folder
<code></code>	The "picture.jpg" file is located in the images folder at the root of the current web
<code></code>	The "picture.jpg" file is located in the folder one level up from the current folder

HTML File Paths



A **file path** shows the **location of a file** in a website's folder structure.

File paths are used when linking to external files, like:

- Web pages
- Images
- Style sheets
- JavaScripts

Absolute File Paths

An **absolute file path** is the **full URL** to a file.

Example

```

```

Relative File Paths

A **relative file path** points to a file **relative to the current page**.

In the following example, the file path points to a file in the **images** folder at the **root of the website**:

Example

```

```

In the following example, the file path points to a file in the **images** folder in the **current folder**:

Example

```

```

In the following example, the file path points to a file in the **images** folder **one level up from the current folder**:

Example

```

```

Best Practice

It is **best practice to use relative file paths (if possible)**.

When you use relative file paths, your web pages will **not be tied to your current base URL**.

All links will work on your **own computer (localhost)**, on your **current public domain**, and on **future public domains**.

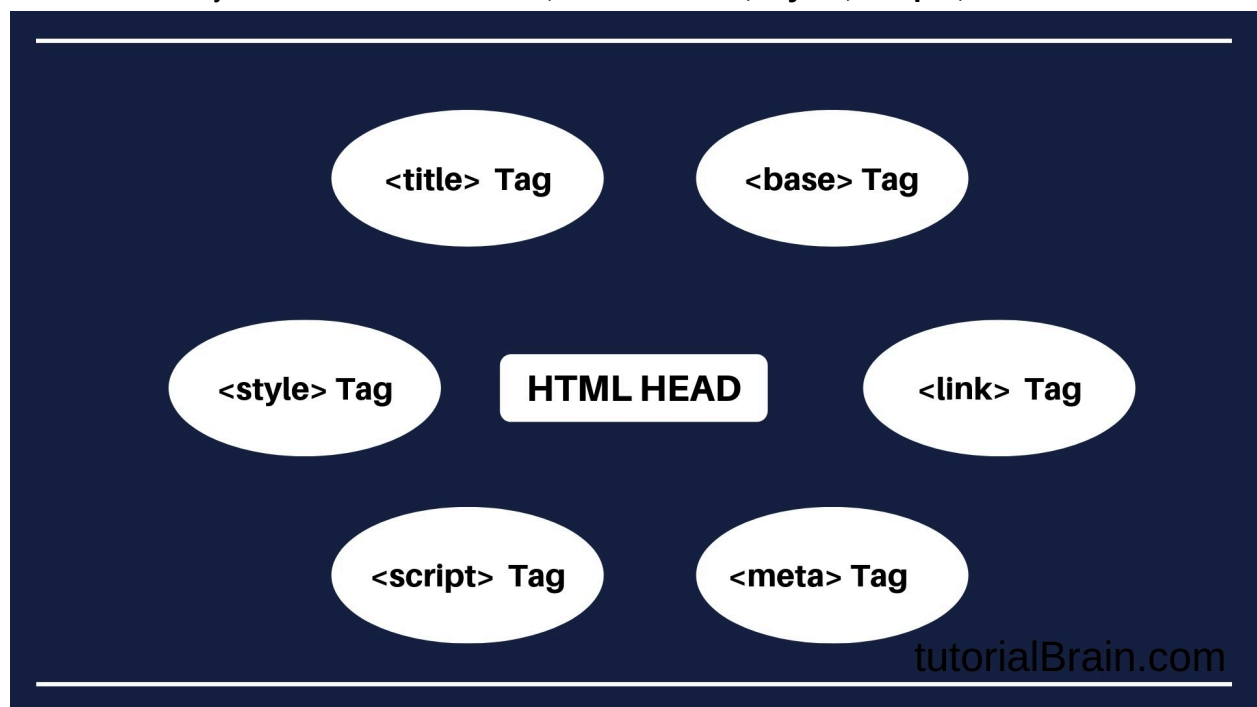
HTML head

The HTML <head> Element

The **<head>** element is a **container for metadata (data about data)** and is placed between the **<html>** tag and the **<body>** tag.

HTML **metadata** is data **about the HTML document**.
Metadata is **not shown on the page**.

Metadata usually sets the **document title, character set, styles, scripts, and other meta info**.



The HTML <title> Element

The `<title>` element sets the **title of the document**.

The title must be **text-only**, and it shows up in the **browser's title bar or tab**.

The `<title>` element is **required in HTML documents!**

The page title is very important for **search engine optimization (SEO)**.

Search engines use the title to help decide how to list pages in search results.

The `<title>` element:

- sets a title in the **browser toolbar**
- gives a title when the page is **added to favorites**
- shows a title for the page in **search engine results**

So, try to make the title **accurate and meaningful!**

A simple HTML document:

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>A Meaningful Page Title</title>
</head>
<body>
```

The content of the document.....

```
</body>
</html>
```

The HTML `<style>` Element

The `<style>` element is used to **set style information for a single HTML page**.

Example

```
<style>
  body {background-color: powderblue;}
  h1 {color: red;}
  p {color: blue;}
</style>
```

The HTML `<link>` Element

The `<link>` element sets the **relationsp between the current document and an external resource**.

The `<link>` tag is most often used to **link to external style sheets**.

Example

```
<link rel="stylesheet" href="mystyle.css">
```

Tip: To learn all about CSS, visit our **CSS Tutorial**.

The HTML `<meta>` Element

The `<meta>` element is usually used to set the **character set, page description, keywords, author of the document, and viewport settings**.

The metadata is **not shown on the page**, but is used by:

- **browsers** (to display content or reload the page)
- **search engines** (for keywords)
- **other web services**

Examples

Define the **character set** used:

```
<meta charset="UTF-8">
```

Define **keywords for search engines**:

```
<meta name="keywords" content="HTML, CSS, JavaScript">
```

Define a **description of your web page**:

```
<meta name="description" content="Free Web tutorials">
```

Define the **author of a page**:

```
<meta name="author" content="John Doe">
```

Refresh the document every 30 seconds:

```
<meta http-equiv="refresh" content="30">
```

Set the **viewport** to make your website look good on all devices:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Example of <meta> tags:

```
<meta charset="UTF-8">
<meta name="description" content="Free Web tutorials">
<meta name="keywords" content="HTML, CSS, JavaScript">
<meta name="author" content="John Doe">
```

Setting The Viewport

The **viewport** is the **visible area of a web page on the user's screen**.

It changes with the device — it is **smaller on a mobile phone** than on a computer screen.

You should include the **<meta>** element on **all your web pages**:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

It tells the browser **how to control the page's size and zoom**.

- The **width=device-width** part sets the width of the page to **match the screen-width of the device** (so it fits phones, tablets, desktops).
- The **initial-scale=1.0** part sets the **initial zoom level** when the page first loads.

Here is an example of a web page **without** the viewport meta tag, and the **same page with** the viewport meta tag:

Tip: If you are looking at this page on a **phone or tablet**, you can click on the two links below to **see the difference**.

The HTML `<script>` Element

The `<script>` element is used to **write client-side JavaScripts**.

The following JavaScript writes **"Hello JavaScript!"** into an HTML element with `id="demo"`:

Example

```
<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Hello JavaScript!";
}
</script>
```

Tip: To learn all about JavaScript, visit our **JavaScript Tutorial**.

The HTML `<base>` Element

The `<base>` element sets the **base URL and/or target** for all **relative URLs on a page**.

The `<base>` tag must have an **href** or **target attribute**, or both.

There can only be **one `<base>` element in a document!**

Example

Set a **default URL and default target** for all links on a page:

```
<head>
<base href="https://www.w3schools.com/" target="_blank">
</head>

<body>

<a href="tags/tag_base.asp">HTML base Tag</a>
</body>
```

Chapter Summary

- The `<head>` element is a **container for metadata (data about data)**.
- The `<head>` element is placed **between the `<html>` tag and the `<body>` tag**.
- The `<title>` element is **required** and it sets the **title of the document**.
- The `<style>` element is used to **set style information for a single document**.
- The `<link>` tag is most often used to **link to external style sheets**.
- The `<meta>` element is usually used to set the **character set, page description, keywords, author, and viewport settings**.
- The `<script>` element is used to **write client-side JavaScripts**.

- The `<base>` element sets the **base URL and/or target for all relative URLs on a page**.

HTML head Elements

Tag	Description
<u><code><head></code></u>	Defines information about the document
<u><code><title></code></u>	Defines the title of a document
<u><code><base></code></u>	Defines a default address or a default target for all links on a page
<u><code><link></code></u>	Defines the relationship between a document and an external resource
<u><code><meta></code></u>	Defines metadata about an HTML document
<u><code><script></code></u>	Defines a client-side script
<u><code><style></code></u>	Defines style information for a document

HTML Layout

HTML Layout Elements

HTML has several **semantic elements** that define **different parts of a web page**:

HTML5 Semantic Elements



- `<header>` - Defines a header for a document or a section
- `<nav>` - Defines a set of navigation links
- `<section>` - Defines a section in a document
- `<article>` - Defines independent, self-contained content
- `<aside>` - Defines content aside from the content (like a sidebar)
- `<footer>` - Defines a footer for a document or a section
- `<details>` - Defines additional details that the user can open and close on demand
- `<summary>` - Defines a heading for the `<details>` element

You can read more about **semantic elements** in our **HTML Semantics chapter**.

HTML Layout Techniques

There are **four different ways** to create **multi-column layouts**.
Each method has its **own pros and cons**:

- **CSS framework**
- **CSS float property**
- **CSS flexbox**
- **CSS grid**

CSS Frameworks

If you want to **create your layout quickly**, you can use a **CSS framework**, like **W3.CSS** or **Bootstrap**.

CSS Float Layout

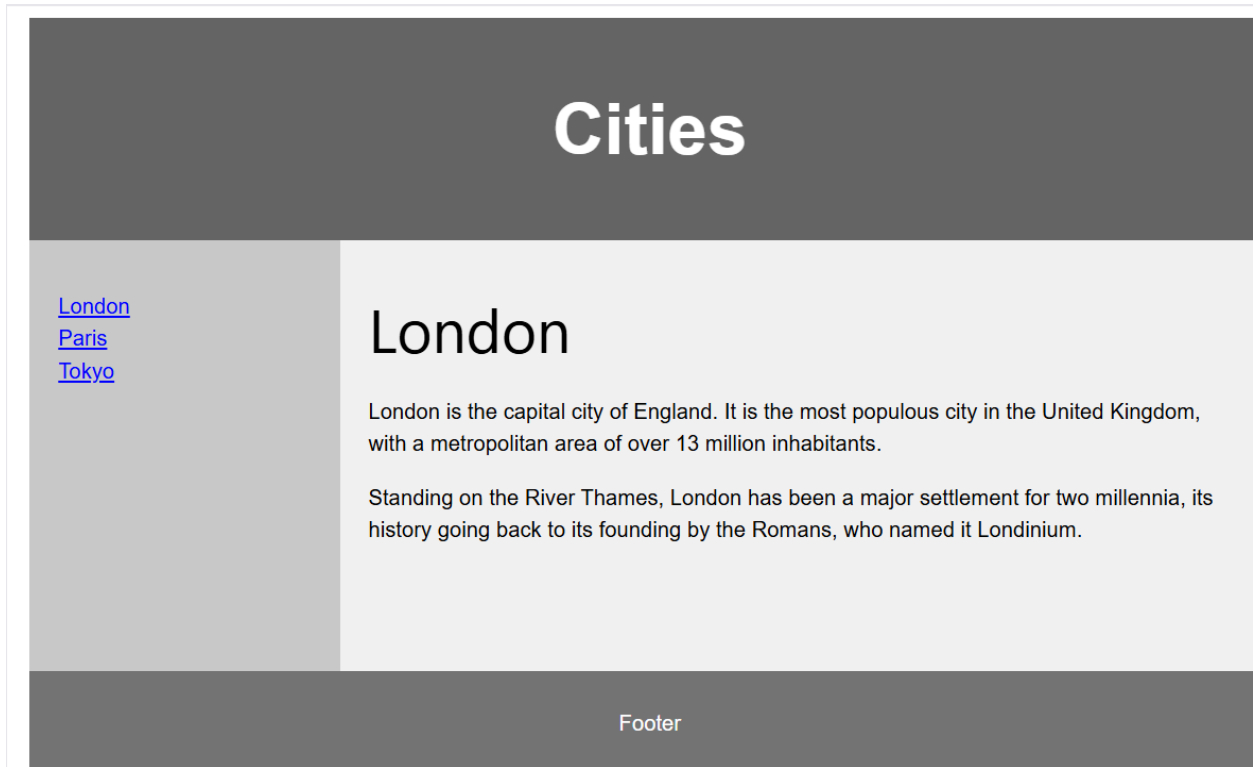
It is common to build **entire web layouts** using the **CSS float property**.

Float is **easy to learn** — you just need to remember how the **float** and **clear** properties work.

Disadvantages:

Floating elements are **tied to the document flow**, wch can **reduce flexibility**.

Learn more about float in our **CSS Float and Clear chapter**.



CSS Flexbox Layout

Using **flexbox** makes sure that elements **behave predictably** when the page layout needs to fit **different screen sizes and devices**.

Learn more about flexbox in our **CSS Flexbox chapter**.

Cities

[London](#)
[Paris](#)
[Tokyo](#)

London

London is the capital city of England. It is the most populous city in the United Kingdom, with a metropolitan area of over 13 million inhabitants.

Standing on the River Thames, London has been a major settlement for two millennia, its history going back to its founding by the Romans, who named it Londinium.

Footer

CSS Grid Layout

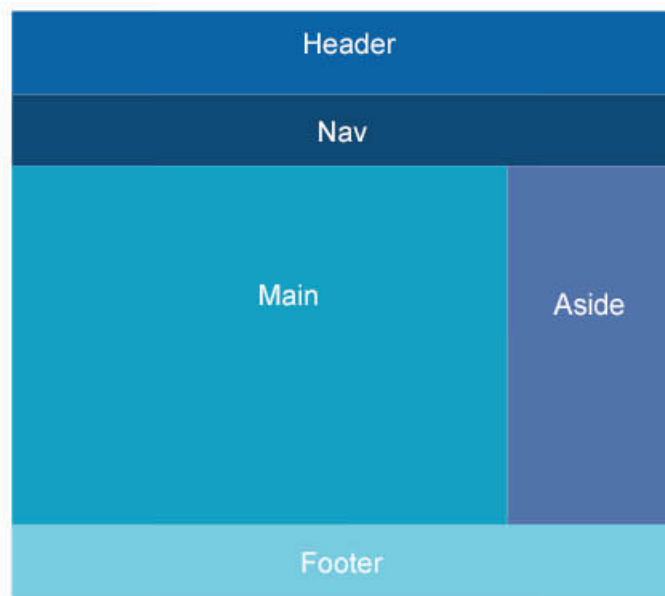
The **CSS Grid Layout Module** gives a **grid-based layout system** with **rows and columns**, making it easier to design web pages **without using floats and positioning**.

Learn more about CSS grids in our **CSS Grid Intro chapter**.

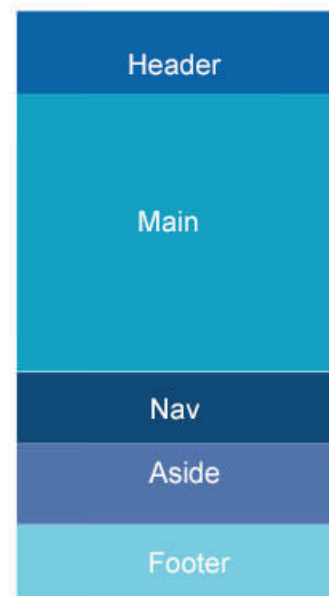
HTML Responsive

What is Responsive Web Design?

Responsive Web Design is about using **HTML** and **CSS** to **automatically resize, de, shrink, or enlarge** a website, so it looks good on **all devices** (desktops, tablets, and phones).



Desktop



Mobile

Setting The Viewport

To create a **responsive website**, add the following `<meta>` tag to **all your web pages**:

Example

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

It sets the **viewport of your page**, telling the browser **how to control the page's size and zoom**.

Here is an example of a web page **without** the viewport meta tag, and the **same web page with** the viewport meta tag:

Without the viewport meta tag:



With the viewport meta tag:



Responsive Images

Responsive images are images that **scale nicely to fit any browser size**.

Using the width Property

If the CSS **width** property is set to **100%**, the image will be **responsive** and **scale up and down**.



Example

```

```



Notice that in the example above, the image can be **scaled up bigger than its original size**. A better way, in many cases, is to use the **max-width** property instead.

Using the max-width Property

If the **max-width** property is set to **100%**, the image will **scale down if needed**, but will **never scale up bigger than its original size**.



Example

```

```

Show Different Images Depending on Browser Width

The HTML `<picture>` element lets you **set different images for different browser window sizes**.

Resize the browser window to see how the image below changes depending on the width:

Flowers



Example

`<picture>`

```
<source srcset="img_smallflower.jpg" media="(max-width: 600px)">
  <source srcset="img_flowers.jpg" media="(max-width: 1500px)">
  <source srcset="flowers.jpg">
  
</picture>
```

Responsive Text Size

The **text size** can be set with a "**vw**" unit, wch means "**viewport width**".

Ts way, the **text size changes** to match the **size of the browser window**.

Hello World

Resize the browser window to see how the text size scales.

Example

```
<h1 style="font-size:10vw">Hello World</h1>
```

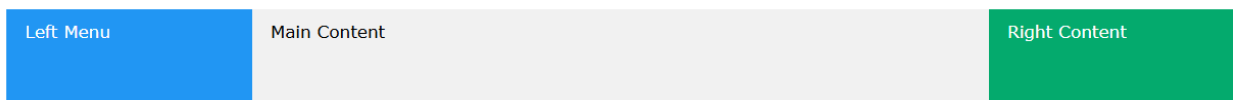
Media Queries

In addition to **resizing text and images**, it is also common to use **media queries** in responsive web pages.

With **media queries**, you can set **completely different styles** for **different browser sizes**.

Example:

Resize the browser window to see that the three `<div>` elements below will **display side by side on large screens**, and **stack on top of each other on small screens**.



Example

```
<style>
.left, .right {
  float: left;
  width: 20%; /* The width is 20%, by default */
}

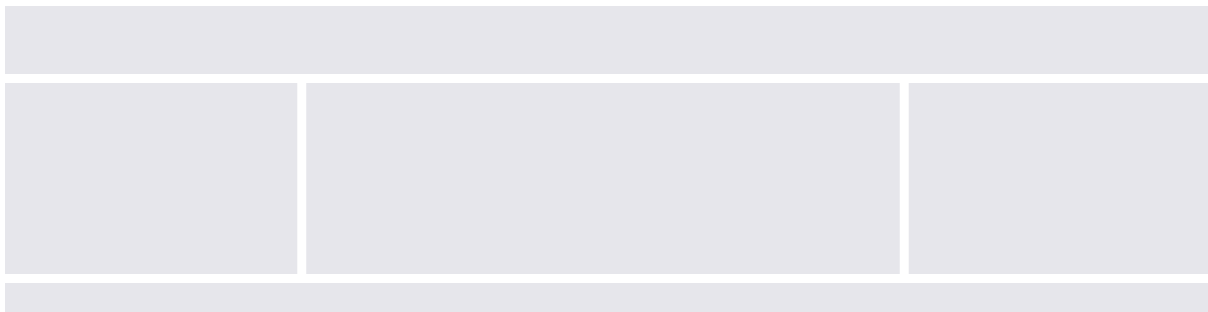
.main {
  float: left;
  width: 60%; /* The width is 60%, by default */
}

/* Use a media query to add a breakpoint at 800px: */
@media screen and (max-width: 800px) {
  .left, .main, .right {
    width: 100%; /* The width is 100%, when the viewport is 800px or
smaller */
  }
}
```

```
}  
</style>
```

Responsive Web Page - Full Example

A responsive web page should look good on large desktop screens and on small mobile phones.



Responsive Web Design - Frameworks

All popular **CSS Frameworks** offer **responsive design**.

They are **free**, and **easy to use**.

W3.CSS

- W3.CSS is a **modern CSS framework** with support for **desktop, tablet, and mobile design by default**.

- W3.CSS is **smaller and faster** than similar CSS frameworks.
- W3.CSS is made to work **without jQuery or any other JavaScript library**.

W3.CSS Demo

Resize the page to see the responsiveness!

London

London is the capital city of England.

It is the most populous city in the United Kingdom, with a metropolitan area of over 13 million inhabitants.

Paris

Paris is the capital of France.

The Paris area is one of the largest population centers in Europe, with more than 12 million inhabitants.

Tokyo

Tokyo is the capital of Japan.

It is the center of the Greater Tokyo Area, and the most populous metropolitan area in the world.

Example (W3.CSS)

```
<!DOCTYPE html>
<html>
<head>
<title>W3.CSS</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet"
href="https://www.w3schools.com/w3css/4/w3.css">
</head>
<body>

<div class="w3-container w3-green">
  <h1>W3Schools Demo</h1>
  <p>Resize ts responsive page!</p>
</div>
```

```
<div class="w3-row-padding">
  <div class="w3-trd">
    <h2>London</h2>
    <p>London is the capital city of England.</p>
    <p>It is the most populous city in the UK, with a metro area of
over 13 million people.</p>
  </div>

  <div class="w3-trd">
    <h2>Paris</h2>
    <p>Paris is the capital of France.</p>
    <p>The Paris area is one of the largest in Europe, with more than
12 million people.</p>
  </div>

  <div class="w3-trd">
    <h2>Tokyo</h2>
    <p>Tokyo is the capital of Japan.</p>
    <p>It is the center of the Greater Tokyo Area, the most populated
area in the world.</p>
  </div>
</div>

</body>
</html>
```

To learn more about W3.CSS, read our **W3.CSS Tutorial**.

Bootstrap

Another popular CSS framework is **Bootstrap**:

Example (Bootstrap)

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Bootstrap 5 Example</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.
min.css" rel="stylesheet">
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.bu
ndle.min.js"></script>
</head>
<body>

<div class="container-fluid p-5 bg-primary text-white text-center">
  <h1>My First Bootstrap Page</h1>
  <p>Resize this responsive page to see the effect!</p>
</div>

<div class="container mt-5">
  <div class="row">
    <div class="col-sm-4">
      <h3>Column 1</h3>
      <p>Lorem ipsum...</p>
    </div>
    <div class="col-sm-4">
      <h3>Column 2</h3>
      <p>Lorem ipsum...</p>
    </div>
    <div class="col-sm-4">
      <h3>Column 3</h3>
      <p>Lorem ipsum...</p>
    </div>
  </div>
</div>
```



```
</div>  
  </div>  
</div>  
  
</body>  
</html>
```

To learn more about Bootstrap, go to our **Bootstrap Tutorial**.

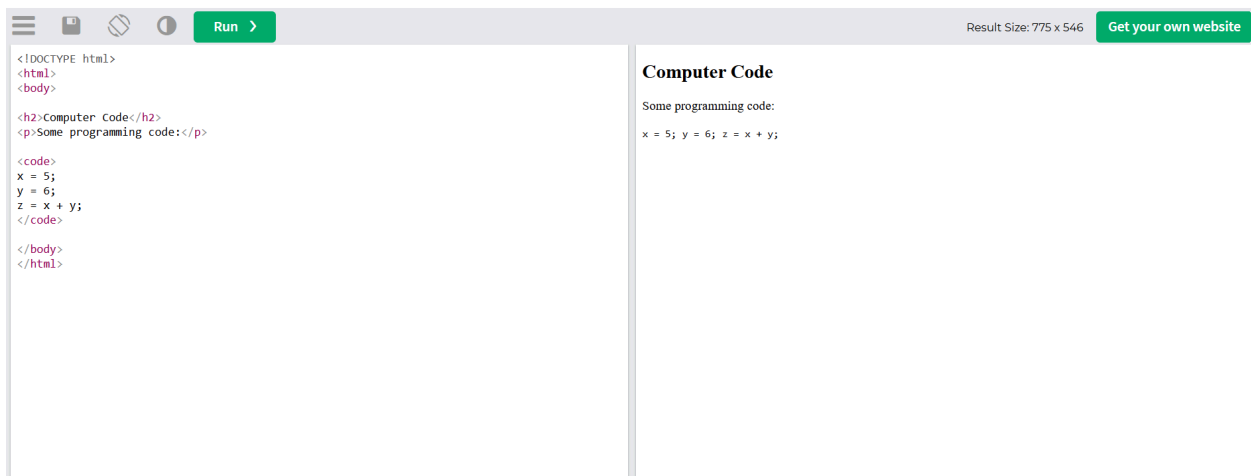
HTML Computer Code

HTML Computer Code Elements

HTML has several elements for showing **user input** and **computer code**.

Example

```
<code>
x = 5;
y = 6;
z = x + y;
</code>
```



HTML <kbd> For Keyboard Input

The HTML `<kbd>` element is used to show **keyboard input**.
The text inside shows up in the browser's **monospace font**.

Example

Define some text as keyboard input:

```
<p>Save the document by pressing <kbd>Ctrl + S</kbd></p>
```

Result:

Save the document by pressing Ctrl + S

HTML `<samp>` For Program Output

The HTML `<samp>` element is used to show **sample output** from a program.
The text inside shows up in the browser's **monospace font**.

Example

Show some text as output from a program:

```
<p>Message from my computer:</p>  
<p><samp>File not found.<br>Press F1 to continue</samp></p>
```

Result:

Message from my computer:
File not found.
Press F1 to continue

HTML `<code>` For Computer Code

The HTML `<code>` element is used to show **computer code**.
The text inside shows up in the browser's **monospace font**.

Example

Show some text as computer code:

```
<code>  
x = 5;  
y = 6;
```

```
z = x + y;  
</code>
```

Result:

x = 5; y = 6; z = x + y;

Preserve Line-Breaks

The `<code>` element does **NOT** keep spaces or line-breaks.

To keep them, use `<pre>` around `<code>`.

Example

```
<pre>  
<code>  
x = 5;  
y = 6;  
z = x + y;  
</code>  
</pre>
```

Result:

x = 5;
y = 6;
z = x + y;

HTML `<var>` For Variables

The HTML `<var>` element is used to show a **variable** in programming or math.
The text inside usually shows up in **italic**.

Example

Show some text as variables:

```
<p>The area of a triangle is: 1/2 x <var>b</var> x <var>h</var>, where  
<var>b</var> is the base, and <var>h</var> is the height.</p>
```

Result:

The area of a triangle is: 1/2 x b x h, where b is the base, and h is the height.

Chapter Summary

- The `<kbd>` element shows **keyboard input**.
- The `<samp>` element shows **sample program output**.
- The `<code>` element shows **computer code**.
- The `<var>` element shows a **variable**.
- The `<pre>` element shows **preformatted text** (keeps spaces & line breaks).

HTML Computer Code Elements

Tag	Description
<u><code></u>	Defines programming code
<u><kbd></u>	Defines keyboard input
<u><samp></u>	Defines computer output
<u><var></u>	Defines a variable
<u><pre></u>	Defines preformatted text

HTML Semantic

HTML Semantic Elements

What are Semantic Elements?

Semantic elements = elements with a meaning.

A semantic element clearly tells **what it is**, to both the **browser** and the **developer**.

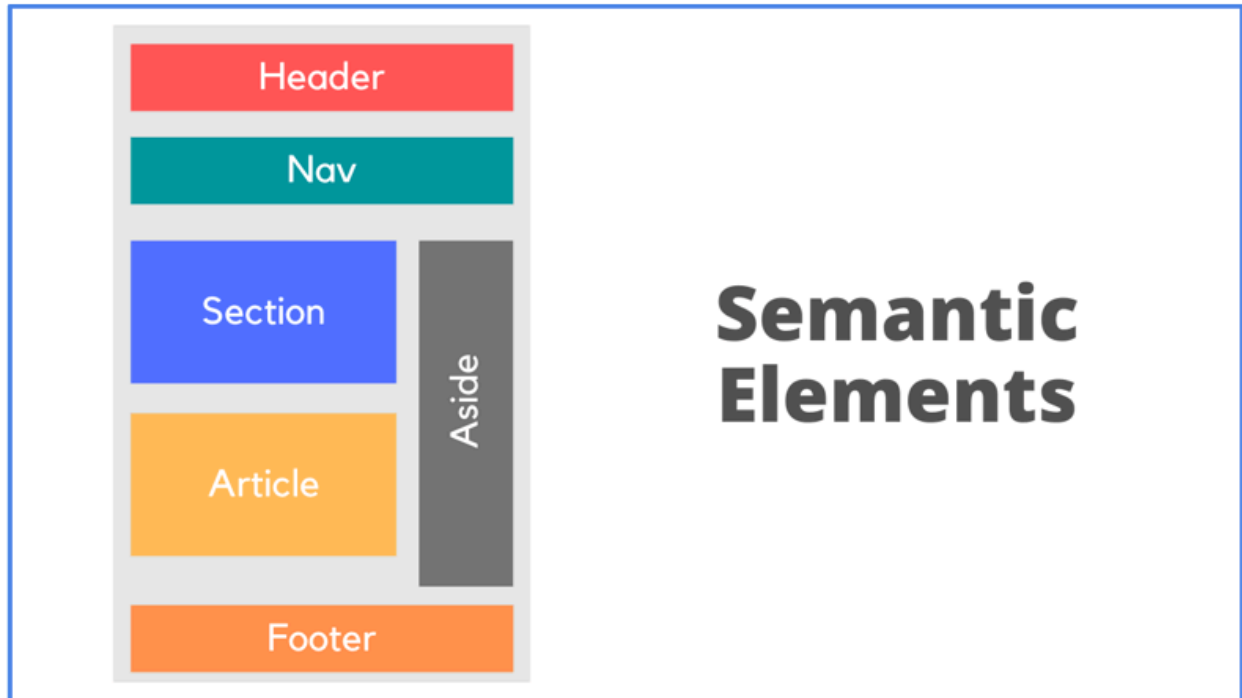
- Examples of **non-semantic elements**:
`<div>`, `<span>` — they **do not tell** what their content is.
- Examples of **semantic elements**:
`<img>`, `<table>`, `<article>` — they **clearly describe** their content.

Semantic Elements in HTML

Many websites use code like:

```
<div id="nav"></div>  
<div class="header"></div>
```

```
<div id="footer"></div>
```



to show **navigation, header, footer** — but these are **not meaningful** to the browser.

HTML gives us **semantic elements** that clearly define parts of a page, such as:

- `<article>`
- `<aside>`
- `<details>`
- `<figcaption>`
- `<figure>`
- `<footer>`
- `<header>`
- `<main>`
- `<mark>`
- `<nav>`
- `<section>`
- `<summary>`
- `<time>`



HTML `<section>` Element

The `<section>` element defines a **section** in a document.

From W3C:

"A section is a grouping of content, usually with a heading."

Examples of using `<section>`:

- Chapters
- Introduction
- News items
- Contact info

Example

Two sections in a document:

```
<section>
  <h1>WWF</h1>
  <p>The World Wide Fund for Nature (WWF) is an international
organization working on conservation, research, and restoration of the
environment.</p>
</section>

<section>
  <h1>WWF's Panda Symbol</h1>
  <p>The panda logo of WWF came from a panda named C C that moved from
Beijing Zoo to London Zoo in 1961.</p>
</section>
```

HTML **<article>** Element

The **<article>** element is for **independent, self-contained content**.

Examples:

- Blog posts
- Forum posts
- Comments
- News articles
- Product cards

Example

Three articles:

```
<article>
  <h2>Google Chrome</h2>
  <p>Google Chrome is a web browser by Google, released in 2008.</p>
</article>
```

```
<article>
  <h2>Mozilla Firefox</h2>
  <p>Mozilla Firefox is an open-source browser, by Mozilla. It's been
the second most popular browser since 2018.</p>
</article>
```

```
<article>
  <h2>Microsoft Edge</h2>
  <p>Microsoft Edge is a browser by Microsoft, released in 2015. It
replaced Internet Explorer.</p>
</article>
```

Example 2

Use CSS to style the <article> element:

```
<html>
<head>
<style>
.all-browsers {
  background: lightgray;
  padding: 5px;
}
.browser {
  background: wte;
  margin: 10px;
```

```
padding: 5px;
}
.browser > h2, p {
margin: 4px;
font-size: 90%;
}
</style>
</head>
<body>

<article class="all-browsers">
  <h1>Most Popular Browsers</h1>
  <article class="browser">
    <h2>Google Chrome</h2>
    <p>World's most popular browser.</p>
  </article>
  <article class="browser">
    <h2>Mozilla Firefox</h2>
    <p>Second most popular browser.</p>
  </article>
  <article class="browser">
    <h2>Microsoft Edge</h2>
    <p>Replaced Internet Explorer.</p>
  </article>
</article>

</body>
</html>
```

Should you nest **<section>** inside **<article>** or vice versa?

- `<article>` is for **independent content**.
- `<section>` is for **grouping content**.

You will often see both ways. It depends on how you organize your content.

HTML `<header>` Element

The `<header>` element is for **intro content or navigation links**.

It often contains:

- Headings (`<h1>` to `<h6>`)
- Logos
- Author info

You can have **many `<header>` elements**, but not inside `<footer>`, `<address>`, or another `<header>`.

Example

A header for an article:

```
<article>
  <header>
    <h1>What Does WWF Do?</h1>
    <p>WWF's mission:</p>
  </header>
  <p>WWF wants a world where people live in harmony with nature.</p>
</article>
```

HTML<footer> Element

The <footer> element is for a **footer** of a page or a section.

It often has:

- Author info
- Copyright info
- Contact info
- Sitemap
- “Back to top” links

You can have **many** <footer> elements.

Example

A footer:

```
<footer>
  <p>Author: Hege Refsnes</p>
  <p><a href="mailto:hege@example.com">hege@example.com</a></p>
</footer>
```

HTML<nav> Element

The <nav> element is for **major navigation links**.

Note:

Not **all links** belong inside <nav>. Use it only for **main navigation**.

Screen readers use <nav> to **skip repetitive navigation**.

Example

Navigation links:

```
<nav>
  <a href="/html/">HTML</a> |
  <a href="/css/">CSS</a> |
  <a href="/js/">JavaScript</a> |
  <a href="/jquery/">jQuery</a>
</nav>
```

HTML<aside> Element

The <aside> element is for content **slightly related** to the main content (like a sidebar).

Example

A sidebar about Epcot:

```
<p>We visited Epcot. It was great!</p>

<aside>
  <h4>Epcot Center</h4>
  <p>Epcot is a theme park with attractions, international pavilions,
and fireworks.</p>
</aside>
```

Example 2

Use CSS to style the <aside> element

```
<style>
```

```
aside {  
  width: 30%;  
  margin-left: 15px;  
  float: right;  
  background: lightgray;  
  padding-left: 15px;  
  font-style: italic;  
}  
</style>
```

HTML<figure> and <figcaption>

- <figure> is for **self-contained content** (images, diagrams, etc).
- <figcaption> is for a **caption** of the figure.

Example

```
<figure>  
    
  <figcaption>Fig1. - Trulli, Italy</figcaption>  
</figure>
```

Why Use Semantic Elements?

From W3C:

"A semantic web allows data to be shared and reused across apps, businesses, and communities."

It also makes pages **easier to read**, **better for SEO**, and **more accessible**.

List of some semantic elements in HTML:

`<article>`, `<aside>`, `<details>`, `<figcaption>`, `<figure>`,
`<footer>`, `<header>`, `<main>`, `<mark>`, `<nav>`, `<section>`,
`<summary>`, `<time>`

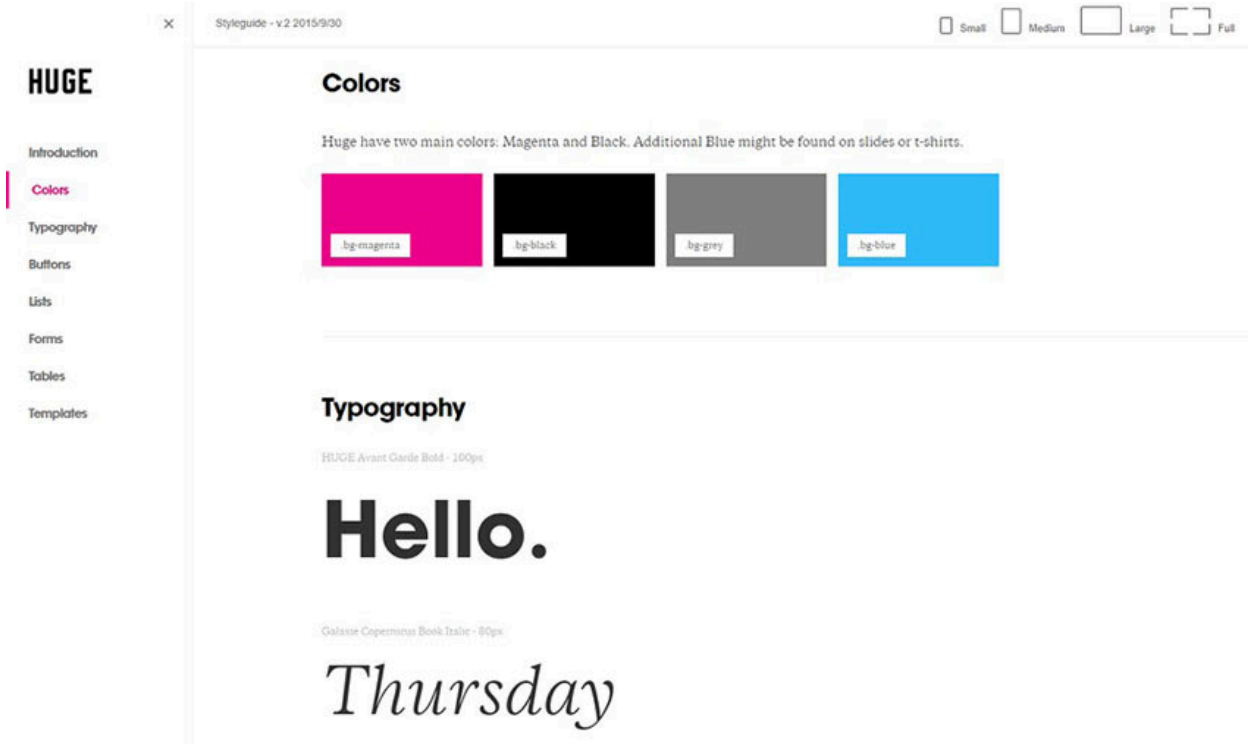
Semantic Elements in HTML

Below is a list of some of the semantic elements in HTML.

Tag	Description
<code><article></code>	Defines independent, self-contained content
<code><aside></code>	Defines content aside from the page content
<code><details></code>	Defines additional details that the user can view or hide
<code><figcaption></code>	Defines a caption for a <code><figure></code> element
<code><figure></code>	Specifies self-contained content, like illustrations, diagrams, photos, code listings, etc.
<code><footer></code>	Defines a footer for a document or section
<code><header></code>	Specifies a header for a document or section
<code><main></code>	Specifies the main content of a document
<code><mark></code>	Defines marked/highlighted text
<code><nav></code>	Defines navigation links
<code><section></code>	Defines a section in a document
<code><summary></code>	Defines a visible heading for a <code><details></code> element
<code><time></code>	Defines a date/time

HTML style Guide

HTML Style Guide & Best Practices



1 Always Declare Document Type

Always start with:

```
<!DOCTYPE html>
```

It tells the browser it's HTML5.

2 Use Lowercase Element Names

Good:

```
<body>
<p>Ts is a paragraph.</p>
</body>
```

Bad:

```
<BODY>
<P>Ts is a paragraph.</P>
</BODY>
```

3 Close All HTML Elements

In HTML, you do not have to close all elements (for example the `<p>` element).

However, we strongly recommend closing all HTML elements, like ts:

Good:

```
<section>
  <p>Ts is a paragraph.</p>
  <p>Ts is a paragraph.</p>
</section>
```

Bad:

```
<section>
  <p>Ts is a paragraph.
  <p>Ts is a paragraph.
</section>
```

4 Use Lowercase Attribute Names

HTML allows mixing uppercase and lowercase letters in attribute names.

However, we recommend using lowercase attribute names, because:

- Mixing uppercase and lowercase names looks bad
- Developers normally use lowercase names
- Lowercase looks cleaner
- Lowercase is easier to type

Good:

```
<a href="https://example.com">Link</a>
```

Bad:

```
<a HREF="https://example.com">Link</a>
```

5 Always Quote Attribute Values

HTML allows attribute values without quotes.

However, we recommend quoting attribute values, because:

- Developers normally quote attribute values
- Quoted values are easier to read
- You MUST use quotes if the value contains spaces

Good:

```
<table class="striped">
```

Bad:

```
<table class=striped>
```

(And very bad with spaces!)

```
<table class=table striped>
```

6 Always Use **alt**, **width**, **height** for Images

The **alt** attribute is **important for accessibility and SEO**. It shows alternative text if the image fails to load and helps screen readers describe the image.

Specifying **width** and **height** allows the browser to **reserve space before the image loads**, preventing layout shifts (page flickering).

Good:

```

```

Bad:

```

```

Spaces and Equal Signs

Using **no spaces around equal signs** makes the code cleaner and groups each attribute-value pair tightly, improving readability.

Good (preferred):

```
<link rel="stylesheet" href="styles.css">
```

Bad:

```
<link rel = "stylesheet" href = "styles.css">
```

Avoid Long Code Lines

When using an HTML editor, it is NOT convenient to scroll right and left to read the HTML code.

Try to avoid too long code lines.

Blank Lines and Indentation

- Do not add extra blank lines or spaces just anywhere.
- Only use blank lines to **separate big sections**, so it looks clean.

- It makes your code **easy to read**.

Indentation (spaces before lines)

- Use **2 spaces** for indentation (not tabs).
- It keeps the code **neat and lined up**.

Good example

```
<body>
```

```
  <h1>Famous Cities</h1>
```

```
    <h2>Tokyo</h2>
```

```
      <p>Tokyo is the capital of Japan...</p>
```

```
    <h2>London</h2>
```

```
      <p>London is the capital city of England...</p>
```

```
    <h2>Paris</h2>
```

```
<p>Paris is the capital of France...</p>
```

```
</body>
```

It is **easy to read**, because there are blank lines between sections and 2 spaces used inside `<body>`.

Bad example

```
<body>
```

```
<h1>Famous Cities</h1>
```

```
<h2>Tokyo</h2><p>Tokyo is the capital of Japan...</p>
```

```
<h2>London</h2><p>London is the capital city of England...</p>
```

```
<h2>Paris</h2><p>Paris is the capital of France...</p>
```

```
</body>
```

It is **hard to read**, everytng is packed together, no spaces, no indentation.

A table that is **neat and easy to read**:

```
<table>
```



```
<tr>

  <th>Name</th>

  <th>Description</th>

</tr>

<tr>

  <td>A</td>

  <td>Description of A</td>

</tr>

<tr>

  <td>B</td>

  <td>Description of B</td>

</tr>

</table>
```

Ths table is **easy to read** because:

- Each row (<tr>) is on its own line.
- Columns (<th>, <td>) are nicely indented.

A list that is **clean and clear**:

```
<ul>

  <li>London</li>

  <li>Paris</li>

  <li>Tokyo</li>

</ul>
```

This list is **easy to understand** because:

- Each item () is on its own line.
- The items are inside with proper spacing.

Never Skip the <title> Element

The **<title>** tag is very important in HTML.

- It is **required**, every web page must have it.
- It sets the **name of the page** that shows at the top of your browser tab.
- It also shows up when someone **saves your page as a favorite/bookmark**.
- Search engines (like Google) use it to decide **how to list your page in search results**.

Make sure your title is clear and matches what your page is about.

Example:

```
<title>HTML Style Guide and Coding Conventions</title>
```

Omitting <html> and <body>?

An HTML page will validate without the <html> and <body> tags:

Example

```
<!DOCTYPE html>
```

```
<head>
```

```
  <title>Page Title</title>
```

```
</head>
```

```
<h1>Ts is a heading</h1>
```

```
<p>Ts is a paragraph.</p>
```

However, we strongly recommend to always add the <html> and <body> tags!

Omitting <body> can produce errors in older browsers.

Omitting <html> and <body> can also crash DOM and XML software.

Omitting <head>?

```
<!DOCTYPE html>

<html>

<title>Page Title</title>

<body>

  <h1>Ts is a heading</h1>

  <p>Ts is a paragraph.</p>

</body>

</html>
```

Browsers will treat the <title> as if it is inside a default <head>.

You're right — I didn't give code that time because you just wanted it explained **in simple**

Example without <head> tag (browser still understands it)

```
<!DOCTYPE html>

<html>

<title>Page Title</title>

<body>

  <h1>Ts is a heading</h1>
```

```
<p>Ts is a paragraph.</p>
```

```
</body>
```

```
</html>
```

Browsers will treat the `<title>` as if it is inside a default `<head>`.

Close Empty HTML Elements?

In HTML, you **don't have to** close empty tags.

Both of these are fine:

```
html
```

CopyEdit

```
<meta charset="utf-8">
```

```
<meta charset="utf-8" />
```

But if you want your page to work with **XML/XHTML**, you **should include** the `/` at the end.

Add the lang Attribute

You should **always add** the `lang` attribute in the `<html>` tag.

Ts helps **search engines and browsers** know what language your page is in.

Example:

```
<!DOCTYPE html>

<html lang="en-us">

<head>

  <title>Page Title</title>

</head>

<body>

  <h1>Ts is a heading</h1>

  <p>Ts is a paragraph.</p>

</body>

</html>
```

Meta Data

You should also set the **character encoding** at the top, like ts:

```
<!DOCTYPE html>

<html lang="en-us">

<head>

  <meta charset="UTF-8">

  <title>Page Title</title>
```

</head>

It helps browsers display your text properly and helps with SEO.

Setting The Viewport

The viewport tag makes your website **responsive** (looks good on phones, tablets, and computers).

Add it inside your <head>:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- `width=device-width` → matches the screen's width.
 - `initial-scale=1.0` → sets the zoom level when the page loads.
-

With viewport:

```
<!DOCTYPE html>
```

```
<html lang="en-us">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width,  
  initial-scale=1.0">
```

```
<title>Page Title</title>

</head>

<body>

    <h1>Ts is a heading</h1>

    <p>Ts is a paragraph.</p>

</body>

</html>
```



Without the viewport meta tag



With the viewport meta tag

HTML Comments

Write **short comments** on one line:

html
CopyEdit
`<!-- Ts is a comment -->`

-

Write **long comments** over multiple lines, indented by two spaces:

html
CopyEdit
`<!--
 Ts is a long comment.
 Ts is another line of the comment.
-->`

-

Using Style Sheets

Link to your CSS file like ts (no **type** needed):

html
CopyEdit
`<link rel="stylesheet" href="styles.css">`

-

Short CSS rules can be on one line:

css

CopyEdit

```
p.intro {font-family:Verdana; font-size:16em;}
```

-

Long CSS rules should be written like ts:

CSS

CopyEdit

```
body {  
  
    background-color: lightgrey;  
  
    font-family: "Arial Black", Helvetica, sans-serif;  
  
    font-size: 16em;  
  
    color: black;  
  
}
```

-

✓ Tips for writing CSS:

- Put the **opening {** on the same line as the selector.
 - Leave **one space** before **{**.
 - Use **two spaces for indentation**.
 - Always put a **semicolon ;**, even after the last property.
 - Only use quotes around values if they contain spaces.
 - Put the **closing }** on a new line, no extra spaces.
-

Loading JavaScript in HTML

Load external JS files like ts (no `type` needed):

html

CopyEdit

```
<script src="myscript.js"></script>
```

Accessing HTML Elements with JavaScript

- HTML is **case-sensitive** with IDs in JavaScript.

For example, ts will **NOT work** if your HTML id is `demo`:

js

CopyEdit

```
getElementById("Demo").innerHTML = "Hello";
```

Use the exact case:

js

CopyEdit

```
getElementById("demo").innerHTML = "Hello";
```

Use Lower Case File Names

- Some servers (like Apache/Linux) are **case-sensitive**.
So `London.jpg` and `london.jpg` are **different files**.

✓ Always use **lowercase file names**.

For example:

- `index.html`
- `style.css`
- `script.js`

File Extensions

- HTML → `.html` (or `.htm`, both work)
- CSS → `.css`
- JavaScript → `.js`

There's **no difference** between `.html` and `.htm`. Both are treated the same.

Differences Between `.htm` and `.html`?

There is no difference between the .htm and .html file extensions!

Both will be treated as HTML by any web browser and web server.

Default Filenames

When you visit a URL like <https://example.com/>, the server tries to load a **default file**, such as:

- [index.html](#)
- [index.htm](#)
- [default.html](#)
- [default.htm](#)

Make sure your file uses the right default name (like [index.html](#)) for it to load automatically.

HTML Entities

HTML Entities

What is it?

- Some characters are **special in HTML**, like `<` and `>`.
 - These are normally used for HTML tags.
 - If you use them directly in your text, the browser might get **confused**.
-

How to fix it?

HTML Character Entities

There are two types:

Named entity:

`<` `<!--` means `<` `-->`

`>` `<!--` means `>` `-->`

Number entity:

`<` `<!--` means `<` `-->`

`>` `<!--` means `>` `-->`

Examples

If you write:

```
<p>5 < 10</p>
```

the browser tnks `< 10` is a tag and it may break.

Instead write:

```
<p>5 &lt; 10</p>
```

Ts will show:

```
5 < 10
```

Non-breaking Space

- Normally, the browser **collapses multiple spaces** into a single space.
- Example:
 - Writing `Hello World` will show as `Hello World` (only one space).

Use ` ` for spaces that don't break or get removed.

```
Hello&nbsp;&nbsp;&nbsp;World
```

Ts will show **three spaces**.

Where is it useful?

To keep words together:

10 km/h

- so they don't break into two lines.
 - To add **extra spaces**.
-

Non-breaking hyphen

- You can also keep words with a hyphen together using:

‑

Example:

well‑known

will stay on the **same line**.

Some Useful HTML Character Entities

Result	Description	Name	Number	
	non-breaking space	 	 	Try it »
<	less than	<	<	Try it »
>	greater than	>	>	Try it »
&	ampersand	&	&	Try it »
"	double quotation mark	"	"	Try it »
'	single quotation mark	'	'	Try it »
¢	cent	¢	¢	Try it »
£	pound	£	£	Try it »
¥	yen	¥	¥	Try it »
€	euro	€	€	Try it »
©	copyright	©	©	Try it »
®	registered trademark	®	®	Try it »
™	trademark	™	™	Try it »

Combining Diacritical Marks

What is a diacritical mark?

- It's a small **symbol** added to a letter.
- Example:
 - á has an **acute accent** (´)
 - è has a **grave accent** (`)

Why use them?

- To create **special letters** that are not normally in the basic character set.
- Useful for writing words in **different languages**.

Mark	Character	Construct	Result	
`	a	à	à	Try it »
´	a	á	á	Try it »
^	a	â	â	Try it »
~	a	ã	ã	Try it »
`	O	Ò	Ò	Try it »
´	O	Ó	Ó	Try it »
^	O	Ô	Ô	Try it »
~	O	Õ	Õ	Try it »

HTML Symbols

HTML Symbol Entities

In HTML, we can show special symbols (like €, ©, ™) that are not on your keyboard.

How to do it?

You can write them in 3 ways:

1 Entity name

- Example: `€`
- Easier to remember.

2 Entity number (decimal)

- Example: `€`

3 Entity number (hexadecimal)

- Example: `€`

&	'	(	)	*	+	,
<code>&amp; &AMP;</code>	<code>&apos;</code>	<code>&lpar;</code>	<code>&rpar;</code>	<code>&ast; &midast;</code>	<code>&plus;</code>	<code>&comma;</code>
.	/	:	;	<	=	>
<code>&period;</code>	<code>&sol;</code>	<code>&colon;</code>	<code>&semi;</code>	<code>&lt; &LT;</code>	<code>&equals;</code>	<code>&gt; &GT;</code>
?	@	[	\	]	^	—
<code>&quest;</code>	<code>&commat;</code>	<code>&lqb; &lbrack;</code>	<code>&bsol;</code>	<code>&rqb; &rbrack;</code>	<code>&hat;</code>	<code>&lowbar;</code>
`	{		}		!	¢
<code>&grave;</code> <code>&DiacriticalGrave;</code>	<code>&lcb; &lbrace;</code>	<code>&verbar; &vert;</code> <code>&VerticalLine;</code>	<code>&rcub; &rbrace;</code>	<code>&nbsp;</code> <code>&NonBreakingSpace;</code>	<code>&excl;</code>	<code>&cent;</code>
£	¤	¥		§	¨	©

Example in HTML

`<p>I will display €</p>`

`<p>I will display €</p>`

`<p>I will display €</p>`

All of these will show:

I will display €

I will display €

I will display €

Some Mathematical Symbols Supported by HTML

Char	Number	Entity	Description	
∀	∀	∀	For all	Try it »
∂	∂	∂	Partial differential	Try it »
∃	∃	∃	There exists	Try it »
∅	∅	∅	Empty sets	Try it »
∇	∇	∇	Nabla	Try it »
∈	∈	∈	Element of	Try it »
∉	∉	∉	Not an element of	Try it »
∋	∋	∋	Contains as member	Try it »
∏	∏	∏	N-ary product	Try it »
∑	∑	∑	N-ary summation	Try it »

Some Greek Letters Supported by HTML

Char	Number	Entity	Description	
A	Α	Α	GREEK ALPHA	Try it »
B	Β	Β	GREEK BETA	Try it »
Γ	Γ	Γ	GREEK GAMMA	Try it »
Δ	Δ	Δ	GREEK DELTA	Try it »
E	Ε	Ε	GREEK EPSILON	Try it »
Z	Ζ	Ζ	GREEK ZETA	Try it »

Some Other Entities Supported by HTML

Char	Number	Entity	Description	
©	©	©	COPYRIGHT	Try it »
®	®	®	REGISTERED TRADEMARK	Try it »
€	€	€	EURO SIGN	Try it »
™	™	™	TRADEMARK	Try it »
←	←	←	LEFT ARROW	Try it »
↑	↑	↑	UP ARROW	Try it »
→	→	→	RIGHT ARROW	Try it »
↓	↓	↓	DOWN ARROW	Try it »
♠	♠	♠	SPADE	Try it »
♣	♣	♣	CLUB	Try it »
♥	♥	♥	HEART	Try it »
♦	♦	♦	DIAMOND	Try it »

HTML Emojis

Using Emojis in HTML

Emojis are characters from the UTF-8 character set: 😊 😍 💕

Emoji	Value	
🗿	🗻	Try it »
🗼	🗼	Try it »
🗽	🗽	Try it »
🇯🇵	🗾	Try it »
🗿	🗿	Try it »
😊	😀	Try it »
😄	😁	Try it »
😂	😂	Try it »
😁	😃	Try it »
😃	😄	Try it »
😆	😅	Try it »

HTML Emojis Examples



[HTML Emoji Transport Symbols](#)



[HTML Emoji Office Symbols](#)



[HTML Emoji People Symbols](#)



[HTML Emoji Animals Symbols](#)

What are Emojis?

- Emojis **look like pictures or icons**, but they are not.
- They are **characters (letters) from the UTF-8 set**, just like A, B, C.

The HTML charset Attribute

- UTF-8 is a **character set**.
- It includes **almost every letter and symbol in the world**.
- That's why we can show all kinds of symbols and emojis.

UTF-8 Characters

- The tag tells the browser **which character set to use**.
- Example:

```
html  
CopyEdit  
<meta charset="UTF-8">
```

- If you don't write it, HTML will still use UTF-8 by default.

Example Explained

- Some characters are shown using **entity numbers**.
- Example:

Character	Number
-----------	--------

A	65
---	----

B	66
---	----

C	67
---	----

So you can write:

```
<p>I will display A B C</p>
```

```
<p>I will display &#65; &#66; &#67;</p>
```

Both lines will show:

```
I will display A B C
```

```
I will display A B C
```

Emoji Characters

- Emojis are also **UTF-8 characters**.

Emoji	Number
-------	--------

😊	128516
---	--------

😍	128525
---	--------



128151

Example

```
<p>❤️</p>
```

Because they are just text, you can style them like ts:

Example

```
<p style="font-size:48px">
  ❤️💎💰💵;
</p>
```

HTML Charsets

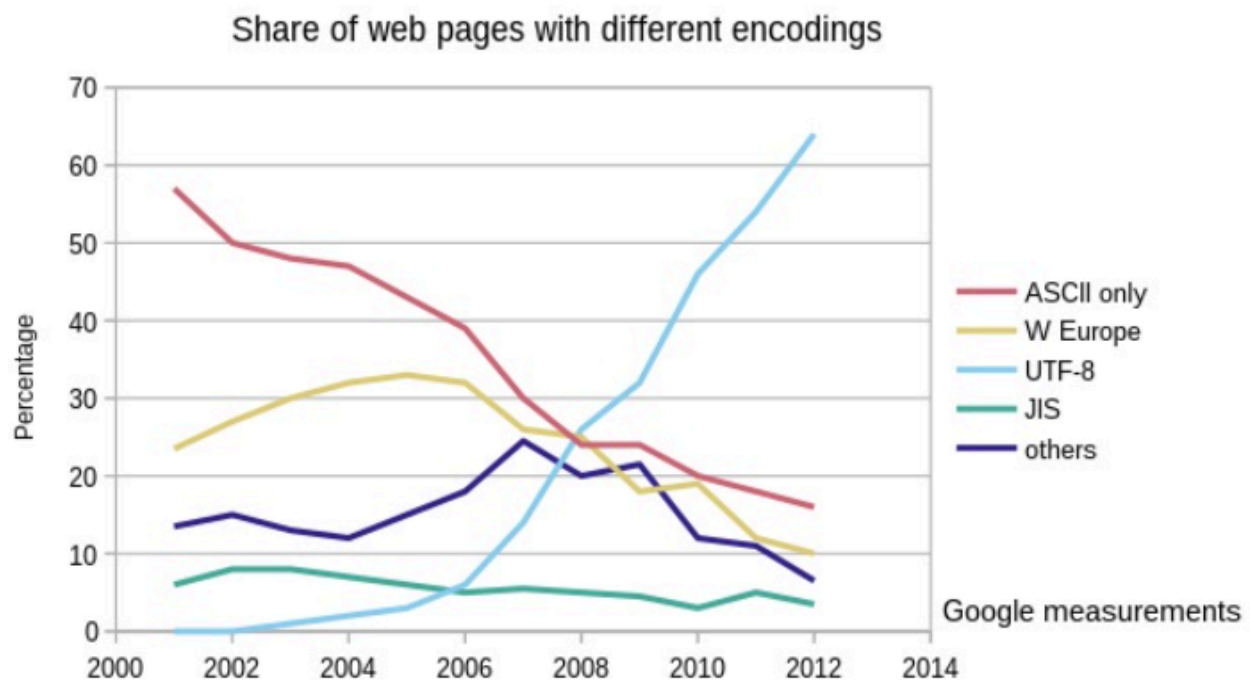
What is a Character Set in HTML?

When a browser loads an HTML page, it needs to know **which characters to display** and **how to display them**. This is done using a **character set** (also called "charset").

How to Set the Character Set in HTML?

You can tell the browser which character set to use by adding this line in the `<head>` of your HTML page:

```
<meta charset="UTF-8">
```



Ts tells the browser: “Use UTF-8 to read ts page.”

Why Use UTF-8?

UTF-8 is the **most popular** character set used in websites today.

- It can show **almost every character and symbol** used in all languages around the world.
 - It is recommended by **HTML5**.
-

Older Character Sets

Before UTF-8, different character sets were used. Here are some examples:

1. ASCII (American Standard Code for Information Interchange)

- It was the first standard.
- Supports 128 characters only.
- Includes English letters (A-Z, a-z), numbers (0–9), and common symbols like @, !, #, etc.

2. ANSI (Windows-1252)

- Used in old Windows systems.
- Same as ASCII for the first 127 characters.
- Adds extra symbols and letters from 128–255.

You can set it like ts:

```
<meta charset="Windows-1252">
```

3. ISO-8859-1

- Used in HTML4 as the default.
- Supports 256 characters.
- Like ANSI but does **not** use characters from 128 to 159.

You can set it like ts:

```
<meta charset="ISO-8859-1">
```

How is UTF-8 Better?

UTF-8:

- Supports all characters in ASCII.
- Adds **thousands more** from other languages (like Hindi, Japanese, Arabic, etc.).
- It's the **most flexible and future-proof** option.

Character Encoding Table (Sample)

Here are a few characters and how they appear in different sets:

Number	ASCII	ANSI	ISO	UTF-8	Character Name
33	!	!	!	!	Exclamation mark
64	@	@	@	@	At symbol
65	A	A	A	A	Capital A
97	a	a	a	a	Small a
169		©	©	©	Copyright symbol
174		®	®	®	Registered symbol
8364		€		€	Euro sign (UTF-8 only)

You can find a **full list** of characters and how they appear in each character set on character reference websites like W3Schools.

Summary

- Always use `<meta charset="UTF-8">` at the top of your HTML.
- It ensures your website supports all kinds of letters, symbols, and languages.
- UTF-8 is modern, widely supported, and safe to use for all web pages.

HTML URL Encode

What is a URL?

URL stands for **Uniform Resource Locator**. It's the technical name for a **web address**.

For example:

- <https://www.google.com> is a URL
- 192.68.20.50 (an IP address) can also be a URL

Most people prefer using names like [w3schools.com](https://www.w3schools.com) because they're easier to remember than numbers.

How Does a URL Work?

A **URL** tells your web browser where to find a web page or a file on the internet.

Example URL:

Bash:

<https://www.w3schools.com/html/default.asp>

This URL is made up of different parts:

Bash:

[scheme://prefix.domain:port/path/filename](#)

Let's break it down:

Part	Meaning
scheme	Type of service (like http or https)
prefix	Commonly www , but can be different or even skipped
domain	Website name (like w3schools.com)
port	Port number on the server (default is 80 for http , 443 for https)
path	Folder or location on the server
filename	The actual file name being requested

Common URL Types (Schemes)

Scheme	Stands For	Use
http	HyperText Transfer Protocol	Regular websites (not secure)
https	Secure HTTP	Secure websites (encrypted)
ftp	File Transfer Protocol	For downloading or uploading files
file	—	Refers to a file on your computer

What is URL Encoding?

URLs can only use **ASCII characters** (letters, numbers, and a few symbols).

If a URL contains **special characters** (like spaces or symbols), they need to be **encoded** before being sent online.

Example:

- A space becomes %20 or sometimes +
- Special letters like ä, é, or symbols like © are replaced with special codes

URL Encoding Example:

- The character © becomes %C2%A9 in a URL

That's how web browsers send information safely using standard characters only.

Try It Yourself Example

If you type your name as **Günter** into a form and submit it:

- The browser will **convert special letters** like ü into URL-friendly format
- The server will receive something like G%C3%BCnter

That's how websites handle names and text from many different languages properly.

Character Encoding Table

Character	URL-encoded (UTF-8)
€ (Euro)	%E2%82%AC
£ (Pound)	%C2%A3
© (Copyright)	%C2%A9
® (Registered)	%C2%AE
Ä (A with umlaut)	%C3%84

Summary

- A **URL** is a web address that tells browsers where to find a page or file.
- URLs use specific formats and may include encoding for special characters.
- **URL encoding** ensures that any unusual characters (like symbols or foreign letters) are safely sent over the Internet.

HTML vs XHTML

What is XHTML?

XHTML stands for **Extensible HyperText Markup Language**.

It is a **stricter version of HTML**, wch follows the rules of **XML (Extensible Markup Language)**.

You can tnk of XHTML as **HTML that follows more rules**, so that the code is cleaner and works better with other technologies like XML.

DIFFERENCE BETWEEN HTML AND XHTML

	HTML	XHTML
Introduction (from Wikipedia)	HTML- HyperText Markup Language	XHTML-Extensible HyperText Markup Language
Developed by	W3C & WHATWG	World Wide Web Consortium
Versions	HTML2, HTML3.2, HTML4.0, HTML5.	XHTML1, XHTML1.1, XHTML2, XHTML5.
Application	Standard Generalized Markup Language (SGML)	Application of XML
Filename extension	.html, .htm	.xhtml, .xht, .xml, .html, .htm
Extended from	SGML	XML, HTML
Type	text/html	application/xhtml+xml
Stands for	HyperText Markup Language	Extensible HyperText Markup Language
Type of format	Document file format	Markup language
Function	Here Web pages are written in HTML.	It is a very strict version of HTML and based on XML.
Nature	Flexible framework requiring lenient HTML specific parser.	A Restrictive subset of XML and needs to be parsed with standard XML parsers.
Origin	HTML was created by Tim Berners-Lee	(W3C)World Wide Web Consortium Recommendation in 2000.

Why Use XHTML?

- In **regular HTML**, browsers often **ignore small mistakes** in code.
- But in **XHTML**, everything must be **written correctly** (well-formed).
- XHTML helps you write better code that is easier to understand and share.
- It is also **compatible with all major web browsers**.

Main Differences Between XHTML and HTML

To use XHTML, you must follow some extra rules:

Rule	Explanation
<code><!DOCTYPE></code> is required	Every XHTML document must start with a DOCTYPE declaration
<code><html></code> tag must include <code>xmlns</code>	It defines the XML namespace
Some tags are mandatory	You must use <code><html></code> , <code><head></code> , <code><title></code> , and <code><body></code>
Tags must be nested correctly	Example: <code><i>text</i></code> is correct
Tags must always be closed	Example: <code><p>Text</p></code> not just <code><p></code>
Use lowercase for all tag names	Example: <code><p></code> , not <code><P></code>
Attribute names must be lowercase	Example: <code>href</code> , not <code>HREF</code>
Attribute values must be in quotes	Example: <code>href="link.html"</code>
No shortcut attributes	Example: use <code>checked="checked"</code> , not just <code>checked</code>

Example of a Simple XHTML Page

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>My XHTML Page</title>
  </head>
  <body>
    <p>Ts is some content.</p>
  </body>
</html>
```

Correct vs Wrong Examples

Proper Nesting

Correct:

```
<b><i>Text</i></b>
```

Wrong:

```
<b><i>Text</b></i>
```

All Tags Must Be Closed

Correct:

```
<p>Ts is a paragraph.</p>
```

Wrong:

```
<p>Ts is a paragraph.
```

Empty Tags Must Be Closed

Correct:

```
<br />

```

Wrong:

```
<br>

```

Lowercase Tags and Attributes

Correct:

```
<a href="page.html">Click here</a>
```

Wrong:

```
<A HREF="page.html">Click here</A>
```

Quoted Attribute Values

Correct:

```
<input type="text" name="username" />
```

Wrong:

```
<input type=text name=username>
```

No Attribute Shortcuts

Correct:

```
<input type="checkbox" checked="checked" />
```

Wrong:

```
<input type="checkbox" checked />
```

Validate Your XHTML Code

To make sure your code is correct, you can check it using the **W3C Validator**:

Link: https://www.w3schools.com/html/html_validate.html

HTML Forms

What is an HTML Form?

An **HTML form** is used to **collect user input**, like their name, email, or choices. The information is usually **sent to a server** to be saved or processed.

HTML Form

First Name :

Last Name :

Date of Birth :

Email id :

Mobile Number :

SUBMIT

RESET

tutorialBrain.com

The **<form>** Element

The **<form>** tag is like a **container**. Inside it, you put all the fields (like text boxes, buttons, etc.) that you want the user to fill out.

Example:

```
<form>  
  <!-- Your input elements go here -->
```

```
</form>
```

The `<input>` Element

The `<input>` tag is used to create different types of fields in a form. The type of field depends on the `type` you set.

Input Type	What It Does
<code>type="text"</code>	Creates a box for single-line text
<code>type="radio"</code>	Creates a round button (choose one)
<code>type="checkbox"</code>	Creates a square checkbox (choose many)
<code>type="submit"</code>	Creates a button to submit the form
<code>type="button"</code>	Just a clickable button (no submit)

Text Fields

Use `<input type="text">` to let the user type something.

Example:

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname"><br>
  <label for="lname">Last name:</label><br>
```

```
<input type="text" id="lname" name="lname">
</form>
```

Note: The form is invisible. Only the input boxes and labels will show.

The `<label>` Element

The `<label>` tag describes what the input is for. It's helpful because:

- Screen readers can read the label out loud.
- Clicking on the label selects the input (like a checkbox).

Always use the `for` attribute with `<label>` and match it to the `id` of the input.

Radio Buttons

Use `<input type="radio">` when users must **choose only one option**.

Example:

```
<p>Choose your favorite Web language:</p>
<form>
  <input type="radio" id="html" name="fav_language" value="HTML">
  <label for="html">HTML</label><br>
  <input type="radio" id="css" name="fav_language" value="CSS">
  <label for="css">CSS</label><br>
  <input type="radio" id="javascript" name="fav_language"
value="JavaScript">
```

```
<label for="javascript">JavaScript</label>
</form>
```

Tip: Use the **same name** for all radio buttons to group them together.

Checkboxes

Use `<input type="checkbox">` when users can **select multiple options**.

Example:

```
<form>
  <input type="checkbox" id="vecle1" name="vecle1" value="Bike">
  <label for="vecle1"> I have a bike</label><br>
  <input type="checkbox" id="vecle2" name="vecle2" value="Car">
  <label for="vecle2"> I have a car</label><br>
  <input type="checkbox" id="vecle3" name="vecle3" value="Boat">
  <label for="vecle3"> I have a boat</label>
</form>
```

Submit Button

Use `<input type="submit">` to **send form data** to a server.

Example:

```
<form action="/action_page.php">
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe"><br><br>
  <input type="submit" value="Submit">
```

```
</form>
```

Important: The `action` attribute tells where to send the form data (like a PHP file).

The `name` Attribute

Every input should have a `name`. Without it, the value will **not be sent** to the server.

Wrong Example:

```
<form action="/action_page.php">
  <input type="text" id="fname" value="John">
  <input type="submit" value="Submit">
</form>
```

In this case, "John" won't be submitted because there's no `name="fname"`.

Summary for Beginners

- Use `<form>` to create a form.
- Use `<input>` for fields like text, radio buttons, checkboxes, etc.
- Always use `name` in input fields if you want the data to be submitted.
- Use `<label>` to describe inputs for better accessibility.
- Use `type="submit"` to send the form data.

HTML Form Attributes

HTML **<form>** Attributes – Explained Simply

When creating a form in HTML, we can use different **attributes** to control how the form behaves. Let's go through them one by one

1) **action** Attribute

- **What it does:** Tells the browser **where to send** the form data when you click the **Submit** button.
- **Usually points to:** A file on the server like **action_page.php**.

Example:

```
<form action="/action_page.php">
```

Tip:

If you **don't use action**, the form will be submitted to the **same page** it's on.

2) **target** Attribute

- **What it does:** Tells the browser **where to show the result** after submitting the form.

Value	Meaning
<code>_self</code>	Opens in the same tab (default)
<code>_blank</code>	Opens in a new tab or window
<code>_parent</code>	Opens in the parent frame
<code>_top</code>	Opens in the full body of the window
<code>framename</code>	Opens in a named iframe

Example (open in new tab):

```
<form action="/action_page.php" target="_blank">
```

3) **method** Attribute

- **What it does:** Decides **how the form data is sent** to the server.

You can use:

- `GET` (default)
- `POST`

Example:

```
<form action="/action_page.php" method="get">  
<form action="/action_page.php" method="post">
```

GET vs POST

GET

Adds data to the **URL** (visible)

Not secure – **don't use for passwords**

Can be **bookmarked**

Limited data size

POST

Sends data in the **background** (hidden)

Safer for sensitive data

Cannot be bookmarked

Can send **large amounts** of data

Use **POST** for things like **login forms**, **contact forms**, etc.

4) **autocomplete** Attribute

- **What it does:** Helps the browser **remember what users typed before** in the form.

Values:

- **"on"** = Enable autocomplete
- **"off"** = Disable autocomplete

Example:

```
<form action="/action_page.php" autocomplete="on">
```

5) **novalidate** Attribute

- **What it does:** Tells the browser **not to check** if the form inputs are valid before submitting.

Example:

```
<form action="/action_page.php" novalidate>
```

Use ts only if you want to **manually validate** data with JavaScript or on the server.

List of All `<form>` Attributes (Simple Summary)

Attribute	Description
<u>accept-charset</u>	Specifies the character encodings used for form submission
<u>action</u>	Specifies where to send the form-data when a form is submitted
<u>autocomplete</u>	Specifies whether a form should have autocomplete on or off
<u>enctype</u>	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
<u>method</u>	Specifies the HTTP method to use when sending form-data
<u>name</u>	Specifies the name of the form
<u>novalidate</u>	Specifies that the form should not be validated when submitted
<u>rel</u>	Specifies the relationship between a linked resource and the current document
<u>target</u>	Specifies where to display the response that is received after submitting the form

HTML Form Elements

HTML Form Elements – Explained for Beginners

An HTML **form** is used to **collect user input** like names, emails, passwords, etc.

A form is created using the `<form>` tag. Inside ts tag, we can use different **form elements** to get information from users.

Common HTML Form Elements

Here are the elements you can use inside a form:

- `<input>` – to take text, number, or other inputs
- `<label>` – to name the input fields
- `<select>` – to create drop-down menus
- `<textarea>` – to let users type long messages
- `<button>` – to make clickable buttons
- `<fieldset>` – to group related fields
- `<legend>` – to give a title to the field group
- `<datalist>` – to show suggestions wle typing
- `<output>` – to show results of calculations
- `<option>` – items inside `<select>`
- `<optgroup>` – groups inside dropdowns

1) **<input>** – Take User Input

Used to create fields where users can type or select something.

Example:

```
<label for="fname">First name:</label>  
<input type="text" id="fname" name="fname">
```

`type="text"` creates a text input. There are many other types (like password, email, etc.).

2) **<label>** – Name for Inputs

Makes input fields easier to understand and click.

Helps screen readers and makes the form more accessible.

Example:

```
<label for="email">Email:</label>  
<input type="text" id="email" name="email">
```

3) **<select>** – Drop-down Menu

Creates a list of choices the user can pick from.

Example:

```
<label for="cars">Choose a car:</label>  
<select id="cars" name="cars">  
  <option value="volvo">Volvo</option>
```

```
<option value="audi">Audi</option>
</select>
```

The first option is selected by default. You can pre-select using `selected`.

Show more options at once:

```
<select size="3">
```

Allow multiple selections:

```
<select multiple>
```

4) `<textarea>` – Multi-line Text Input

Let users write longer messages (like comments or feedback).

Example:

```
<textarea name="message" rows="5" cols="30">
Your message here...
</textarea>
```

You can also style it using CSS:

```
<textarea style="width:200px; height:100px;"></textarea>
```

5) `<button>` – Clickable Button

Creates a button that users can click.

Example:

```
<button type="button" onclick="alert('Hello!')">Click Me</button>
```

Always add the `type` (like `type="submit"` or `type="button"`).

6) `<fieldset>` and `<legend>` – Group Related Fields

Use `<fieldset>` to group form elements and `<legend>` to give it a title.

Example:

```
<fieldset>
  <legend>Personal Info:</legend>
  <input type="text" name="fname">
</fieldset>
```

It adds a border around the group with a title.

7) `<datalist>` – Show Suggestions While Typing

Lets you suggest values for an `<input>`.

Example:

```
<input list="browsers">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
</datalist>
```

As the user types, suggestions appear.

8) **<output>** – Show Calculated Result

Displays a result (like from a script).

Example:

```
<form oninput="x.value=parseInt(a.value)+parseInt(b.value)">
  <input type="range" id="a" value="50">
  +
  <input type="number" id="b" value="50">
  =
  <output name="x" for="a b"></output>
</form>
```

The result of the addition is shown live.

Quick Summary: Form Tags

Tag	Description
<u><form></u>	Defines an HTML form for user input
<u><input></u>	Defines an input control
<u><textarea></u>	Defines a multiline input control (text area)
<u><label></u>	Defines a label for an <input> element
<u><fieldset></u>	Groups related elements in a form
<u><legend></u>	Defines a caption for a <fieldset> element
<u><select></u>	Defines a drop-down list
<u><optgroup></u>	Defines a group of related options in a drop-down list
<u><option></u>	Defines an option in a drop-down list
<u><button></u>	Defines a clickable button
<u><datalist></u>	Specifies a list of pre-defined options for input controls
<u><output></u>	Defines the result of a calculation

HTML Input Types

Common Input Type Examples (with explanation)

1. Text Input

```
<input type="text">
```

Is the **default** input for short text like name or city.

2. Password Field

```
<input type="password">
```

Characters are hidden (shows dots or stars). Great for passwords.

3. Submit Button

```
<input type="submit" value="Submit">
```

Sends the form data to the server.

4. Reset Button

```
<input type="reset" value="Reset">
```

Clears everything in the form and resets it to the default values.

5. Radio Button (Choose One Option)

```
<input type="radio" name="language" value="HTML"> HTML
```

Use **same name** so only one option can be selected.

6. Checkbox (Choose Multiple)

```
<input type="checkbox" value="Bike"> I have a bike
```

You can check one, two, or all of them.

7. Button

```
<input type="button" value="Click Me" onclick="alert('Hello!')">
```

A normal clickable button (you can control what it does using JavaScript).

8. Color Picker

```
<input type="color">
```

Opens a color selector.

9. Date Picker

```
<input type="date">
```

Shows a calendar to choose a date.

15. Number Field

```
<input type="number" min="1" max="5">
```

Only allows numbers. You can limit the values using `min`, `max`, and `step`.

16. Range (Slider)

```
<input type="range" min="0" max="100">
```

Lets users select a number by sliding.

17. Search Field

```
<input type="search">
```

Same as text input but styled for search (e.g., with a clear button on mobile).

18. Phone Number Field

```
<input type="tel" pattern="[0-9]{3}-[0-9]{3}-[0-9]{4}">
```

Input for telephone numbers, with pattern to control the format.

19. Time Picker

```
<input type="time">
```

Lets user choose only time (like 11:30 AM).

20. URL Field

```
<input type="url">
```

Validates if input is a website URL (like <https://example.com>).

21. Week Picker

```
<input type="week">
```

Lets the user select a week and year.

Input Restrictions You Can Use

Attribute	Description
checked	Specifies that an input field should be pre-selected when the page loads (for type="checkbox" or type="radio")
disabled	Specifies that an input field should be disabled
max	Specifies the maximum value for an input field
maxlength	Specifies the maximum number of character for an input field
min	Specifies the minimum value for an input field
pattern	Specifies a regular expression to check the input value against
readonly	Specifies that an input field is read only (cannot be changed)
required	Specifies that an input field is required (must be filled out)
size	Specifies the width (in characters) of an input field
step	Specifies the legal number intervals for an input field
value	Specifies the default value for an input field

HTML Input Attributes

HTML `<input>` Attributes — Easy Guide for Beginners

The `<input>` tag is used to create fields in a form. These fields can have different behaviors based on the **attributes** added to them.

1. `value` Attribute

Sets the **default value** in the input box.

```
<input type="text" value="John">
```

When the page loads, "John" will already appear in the box.

2. `readonly` Attribute

Makes the field **read-only** (users can see it but can't change it).

```
<input type="text" value="John" readonly>
```

Users can **copy** the text but **not edit** it.

3. `disabled` Attribute

Completely **disables** the input field.

```
<input type="text" value="John" disabled>
```

Users **cannot click, type, or submit** data from its field.

4. **size** Attribute

Sets the **visible width** of the input (in characters).

```
<input type="text" size="50">
```

Default size is **20** if not mentioned.

5. **maxlength** Attribute

Sets the **maximum number of characters** allowed.

```
<input type="text" maxlength="4">
```

Prevents typing more than 4 characters.

6. **min** and **max** Attributes

Used for inputs like number, date, range, etc.

```
<input type="number" min="1" max="5">
```

Allows only numbers between 1 and 5.

7. **multiple** Attribute

Lets users **select more than one file or email**.

```
<input type="file" multiple>
```

Useful for file uploads or multiple email addresses.

8. **pattern** Attribute

Sets a **rule** (like 3 letters only) using regular expressions.

```
<input type="text" pattern="[A-Za-z]{3}" title="Enter 3 letters only">
```

Shows an error if the rule is not followed when submitting.

9. **placeholder** Attribute

Shows a **light nt** inside the field.

```
<input type="tel" placeholder="123-45-678">
```

Helps users understand what to type.

10. **required** Attribute

Makes the field **mandatory**.

```
<input type="text" required>
```

You can't submit the form until ts field is filled.

11. **step** Attribute

Sets **step values** (e.g., steps of 3, 5, 10).

```
<input type="number" step="3">
```

Allows values like 0, 3, 6, 9, etc.

12. **autofocus** Attribute

Puts the **cursor in the field automatically** when the page loads.

```
<input type="text" autofocus>
```

No need to click the field manually.

13. **height** and **width** (For Image Inputs)

Set the **size** of an image used as a submit button.

```
<input type="image" src="submit.png" width="48" height="48">
```

Helps the browser reserve space wle loading images.

14. **list** Attribute

Connects an input with a **datalist** (suggested values).

```
<input list="browsers">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
</datalist>
```

Suggestions show up as the user types.

15. **autocomplete** Attribute

Tells the browser whether it should **suggest saved inputs**.

```
<input type="email" autocomplete="off">
```

If set to **on**, it remembers previous entries. If **off**, it won't suggest anything.

Summary Table: HTML Form Tags

Tag	Description
<code><form></code>	Creates a form
<code><input></code>	Creates input fields

Input Form Attributes

HTML `<input>` — Form Attributes for Beginners

HTML form attributes are used to control how form elements behave. Let's break them down with simple explanations and examples.

1. `form` Attribute

It tells the browser which `<form>` the input belongs to — even if it's outside the form. It connects the input to a form using the form's `id`.

```
<form id="form1">
  <input type="text" name="fname">
</form>
```

```
<!-- This input is outside the form but still belongs to it -->
<input type="text" name="lname" form="form1">
```

2. `formaction` Attribute

It sets a different URL to send the form data — only for this button. Works with `<input type="submit">` and `<input type="image">`.

```
<form action="/default_page.php">
  <input type="submit" value="Submit Normally">
  <input type="submit" formaction="/admin_page.php" value="Submit as Admin">
</form>
```

3. `formenctype` Attribute

Tells the browser how to send the form data — mostly used when uploading files.

Used only when `method="post"`.

```
<form method="post" action="/save.php">
  <input type="text" name="fname">
  <input type="submit" value="Submit Normally">
  <input type="submit" formenctype="multipart/form-data" value="Submit
as File">
</form>
```

Use `"multipart/form-data"` when uploading files.

4. `formmethod` Attribute

Sets the method used to send data:

- `"get"`: sends data in the URL
- `"post"`: sends data securely in the background

```
<form action="/submit.php" method="get">
  <input type="text" name="fname">
  <input type="submit" value="Send with GET">
  <input type="submit" formmethod="post" value="Send with POST">
</form>
```

- `get`: Fast, visible in the URL
 - `post`: Secure, better for forms with passwords or large data
-

5. **formtarget** Attribute

Tells where to show the response when the form is submitted.

```
<form action="/submit.php">
  <input type="text" name="fname">
  <input type="submit" value="Submit Here">
  <input type="submit" formtarget="_blank" value="Open in New Tab">
</form>
```

- **_blank** opens the result in a new tab
 - **_self** opens the result in the same tab (default)
-

6. **formnovalidate** Attribute

Stops input validation for that button.

Works with `<input type="submit">`.

```
<form action="/submit.php">
  <input type="email" name="email" required>
  <input type="submit" value="Validate Normally">
  <input type="submit" formnovalidate value="Submit Without
Validation">
</form>
```

Even if the email is invalid, the second button submits it anyway.

7. **novalidate** (form attribute)

novalidate is used on the `<form>` itself.

It turns off all input validation for that form.

```
<form action="/submit.php" novalidate>
  <input type="email" name="email" required>
  <input type="submit" value="Submit Without Checking">
</form>
```

Use novalidate if you don't want the browser to check if fields are filled out correctly.

Summary Table

Attribute	What it does
form	Connects an input to a form
formaction	Sends form to a different URL
formenctype	Changes how data is encoded (for file upload)
formmethod	Changes the method (GET or POST)
formtarget	Sets where the result opens (tab, window)
formnovalidate	Disables validation for its input
novalidate	Disables validation for the whole form

HTML Tags Recap

Tag	Description
<code><form></code>	Defines an HTML form for user input
<code><input></code>	Defines an input control

HTML Graphics

1. HTML Canvas Grapcs

HTML **<canvas>** Made Simple

The **HTML **<canvas>**** element lets you draw shapes, text, and images on a web page using **JavaScript**.

What is **<canvas>**?

- **<canvas>** is like a blank board where you can **draw** using code.
 - You **can't draw** directly with HTML. You need **JavaScript** to draw on it.
 - You can draw:
 - Lines
 - Circles
 - Rectangles
 - Text
 - Images
 - Gradients
-

Browser Support

All major web browsers like Chrome, Firefox, Safari, and Edge support `<canvas>`.

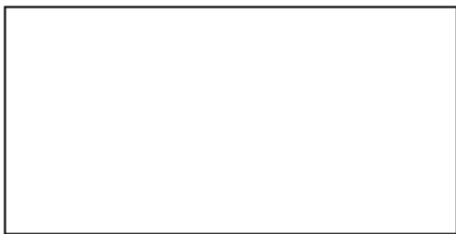
Basic `<canvas>` Setup

You must give:

- An `id` so JavaScript can find it.
- A `width` and `height` for size.
- A `style` (optional) for things like border.

Example:

```
<canvas id="myCanvas" width="200" height="100" style="border:1px solid #000000;"></canvas>
```



How to Draw on Canvas (with JavaScript)

Once the canvas is set, use JavaScript to draw.

Draw a Line

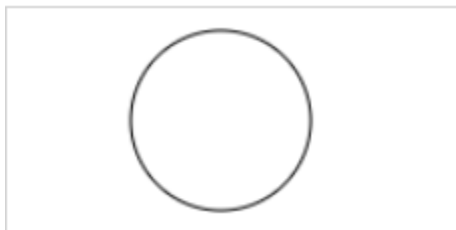
```
<script>  
var c = document.getElementById("myCanvas");
```

```
var ctx = c.getContext("2d");
ctx.moveTo(0, 0);           // start point
ctx.lineTo(200, 100);       // end point
ctx.stroke();                // draw the line
</script>
```



Draw a Circle

```
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
ctx.beginPath();
ctx.arc(95, 50, 40, 0, 2 * Math.PI); // center x, y, radius
ctx.stroke();                          // draw the circle
</script>
```

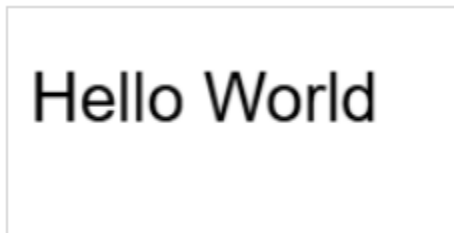


Draw Text

```
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
```



```
ctx.font = "30px Arial";  
ctx.fillText("Hello World", 10, 50);  
</script>
```



Draw Text Outline (Stroke Text)

```
<script>  
var c = document.getElementById("myCanvas");  
var ctx = c.getContext("2d");  
ctx.font = "30px Arial";  
ctx.strokeText("Hello World", 10, 50);  
</script>
```



Draw a Rectangle with Gradient (Left to Right)

```
<script>  
var c = document.getElementById("myCanvas");  
var ctx = c.getContext("2d");  
  
var grd = ctx.createLinearGradient(0, 0, 200, 0); // left to right  
grd.addColorStop(0, "red");
```

```
grd.addColorStop(1, "wte");  
  
ctx.fillStyle = grd;  
ctx.fillRect(10, 10, 150, 80);  
</script>
```



Draw a Circular Gradient

```
<script>  
var c = document.getElementById("myCanvas");  
var ctx = c.getContext("2d");  
  
var grd = ctx.createRadialGradient(75, 50, 5, 90, 60, 100);  
grd.addColorStop(0, "red");  
grd.addColorStop(1, "wte");  
  
ctx.fillStyle = grd;  
ctx.fillRect(10, 10, 150, 80);  
</script>
```



Draw an Image

You first need to add the image in HTML:

```

```

Then use ts JavaScript:

```
<script>
var c = document.getElementById("myCanvas");
var ctx = c.getContext("2d");
var img = document.getElementById("scream");
ctx.drawImage(img, 10, 10);
</script>
```

Summary for Beginners

What You Can Do	Method Used
Draw a line	<code>moveTo, lineTo, stroke()</code>
Draw a circle	<code>arc, stroke()</code>
Draw text	<code>fillText, strokeText</code>
Fill with color	<code>fillRect</code>
Use gradients	<code>createLinearGradient, createRadialGradient</code>
Draw images	<code>drawImage</code>

HTML SVG

2. HTML SVG Grapcs

What is SVG?

SVG stands for **Scalable Vector Grapcs**.

It is used to draw shapes like **circles**, **rectangles**, **lines**, and **text** on a webpage using **XML code**.

- SVG images **do not lose quality** when zoomed or resized.
 - They can be written directly inside HTML.
 - They work in **all modern browsers**.
-

Key Points about SVG

- SVG uses **XML format** (text-based code).
 - It is **vector-based**, meaning it's not made of pixels like a photo.
 - You can use **CSS** and **JavaScript** with SVG.
 - SVG shapes can be **animated**.
 - SVG is recommended by **W3C** (World Wide Web standards group).
-

SVG Example: Circle

Ts code draws a **yellow circle** with a **green border**.



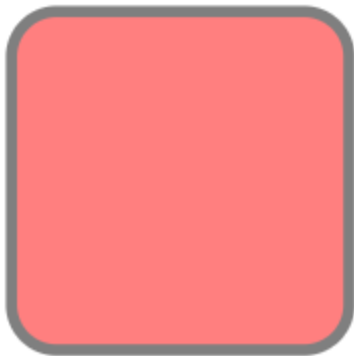
SVG Rectangle

Ts draws a **blue rectangle** with a **red border**.



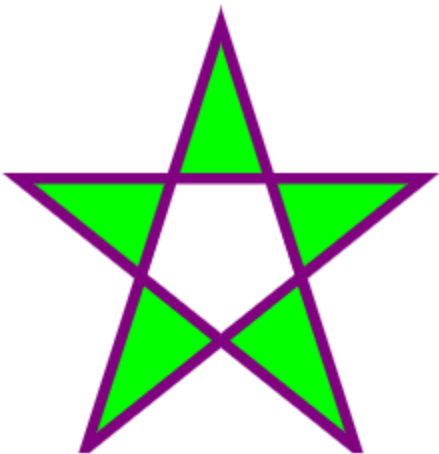
Rounded and Transparent Rectangle

Ts rectangle has **rounded corners** and is **50% transparent**.



SVG Star (Polygon)

Ts draws a **star shape** using points.



SVG with Gradient Fill and Text

Ts adds **gradient color** to a shape and shows text.



SVG vs. Canvas (Simple Comparison)

Feature	SVG	Canvas
Graphics Type	Vector (scalable)	Pixel-based (bitmap)
Zoom/Resize Quality	No quality loss	Can lose quality
Control with JS	Yes (individual objects)	No (drawn once, then forgotten)
Event Listeners	Yes (click, hover on shapes)	No direct event handling
Text Support	Very good	Basic text support
Use Case	Charts, diagrams, icons	Games, fast graphics, animations
Save as Image	Not direct	Can be saved as .png or .jpg

In Short

- Use **SVG** when you want **clean, scalable graphics** like icons, charts, or illustrations.
- Use **Canvas** when you want **fast drawing**, like for **games** or **real-time animation**.

HTML Media

Multimedia on the Web

Multimedia means content that includes **sound, music, videos, movies, and animations**.

What is Multimedia?

Multimedia can be anything you can **see or hear**, such as:

- Images
- Music
- Sounds
- Videos
- Movies
- Animations

Many websites include multimedia elements to make pages more interactive and interesting.

Browser Support

In the beginning, web browsers could only show **plain text** in one font and one color.

Now, modern browsers support:

- Colors and fonts
- Images and videos
- Sound and animations

Multimedia Formats

Multimedia content (like audio or video) is saved in **media files**.

You can recognize the type of media by the **file extension** at the end of the file name.
Examples:

- .mp3
- .mp4
- .wav
- .ogg

Common Video Formats

Here are some popular video file types and how they work:

Format	File Extension	Description
MPEG	.mpg, .mpeg	One of the first popular web video formats. No longer supported in HTML.
AVI	.avi	Developed by Microsoft. Works well on Windows but not in most web browsers.
WMV	.wmv	Another Microsoft format. Good for Windows devices, but not browser-friendly .
MOV (QuickTime)	.mov	Developed by Apple. Works on Apple devices, but not in all browsers .
RealVideo	.rm, .ram	Designed for slow internet, but not supported in modern browsers .
Flash	.swf, .flv	Used to need a browser plug-in. Now outdated and not supported.
Ogg	.ogg	Open format supported by HTML. Works in modern browsers.
WebM	.webm	Made by Google and others. Works in most browsers.

MP4 **.mp4** Very popular. Works on almost all devices and browsers.
Recommended by YouTube.

Important: Only **.mp4**, **.webm**, and **.ogg** are fully supported by modern HTML.

Common Audio Formats

If you're sharing music or sound on your website, you need to use supported audio file formats.

Format	File Extension	Description
MIDI	.mid , .midi	Mostly used in music hardware. Doesn't work in browsers.
RealAudio	.rm , .ram	Used to work with slow internet. Not used in modern browsers.
WMA	.wma	Made by Microsoft. Works on Windows but not in most browsers.
AAC	.aac	Made by Apple. Used in iTunes. Not supported in all browsers.
WAV	.wav	Works well on all systems (Windows, Mac, Linux). Supported in HTML.
Ogg	.ogg	Open format. Works in modern browsers.
MP3	.mp3	Very popular for music. Small file size with good quality. Supported by all browsers.
MP4	.mp4	Mostly used for video, but can also store audio. Supported by all browsers.

Summary

- Use **MP3** for audio/music (best for compatibility and quality).
- Use **MP4**, **WebM**, or **Ogg** for video (these work with HTML and modern browsers).
- Avoid old formats like Flash, RealAudio, or WMA for web use.

HTML Video

HTML `<video>` Element – Easy Guide for Beginners

The `<video>` tag in HTML is used to **show a video on your web page**. You can play videos like movies or clips directly in the browser.

```
<!DOCTYPE html>
<html>
<body>

<h1>The video element</h1>

<video width="320" height="240" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>

</body>
</html>
```

The video element



Basic Video Example

```
<video width="320" height="240" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>
```

What does ts do?

- `<video>` is the main tag used to display video.
 - `width` and `height` set the size of the video player.
 - `controls` shows buttons like play, pause, and volume.
 - `<source>` tells the browser where the video file is and what type it is.
 - The text inside the tag is shown only if the browser cannot play the video.
-

Autoplay a Video

If you want the video to **start automatically**, use the `autoplay` attribute:

```
<video width="320" height="240" autoplay>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>
```

Important:

Most browsers **do not autoplay videos with sound**. To make autoplay work, you must also add `muted`.

```
<video width="320" height="240" autoplay muted>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.ogg" type="video/ogg">
  Your browser does not support the video tag.
</video>
```

Supported Video Formats

Web browsers support these formats:

Format	File Extension	Supported in HTML
MP4	.mp4	Yes
WebM	.webm	Yes
Ogg	.ogg	Yes

Most recommended format: **MP4** (works in almost all browsers and on YouTube).

Media Types for Video

Each video format has a media type:

File Format	Media Type
MP4	video/mp4
WebM	video/webm
Ogg	video/ogg

Browser Support for **<video>** Tag

Browser	MP4	WebM	Ogg
Chrome	Yes	Yes	Yes
Firefox	Yes	Yes	Yes
Safari	Yes	Yes	No
Edge	Yes	Yes	Yes
Opera	Yes	Yes	Yes

Controlling Video with JavaScript

Using JavaScript, you can control the video (play, pause, change volume, etc.).

You can also track events like when the video plays, pauses, or ends.

Example of controlling a video:

```
var video = document.getElementById("myVideo");  
video.play(); // To play the video  
video.pause(); // To pause the video  
video.volume = 0.5; // Set volume (0.0 to 1.0)
```

Video-Related HTML Tags

Tag	Description
<code><video></code>	Defines a video on the web page
<code><source></code>	Adds multiple video files for browser choice
<code><track></code>	Adds subtitles or captions (optional)

HTML Audio

HTML `<audio>` Element – Beginner Guide

The HTML `<audio>` element is used to **play audio** (like music or sound effects) on a web page.

```
<!DOCTYPE html>
<html>
<body>

<h1>The audio element</h1>

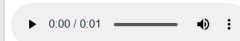
<p>Click on the play button to play a sound:</p>

<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>

</body>
</html>
```

The audio element

Click on the play button to play a sound:



Basic Audio Example

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

What does its code do?

- `<audio>` is used to add sound to a web page.
- `controls` adds play, pause, and volume buttons for users.

- `<source>` lets you add multiple audio formats. The browser will pick the one it supports.
 - The text inside the tag is shown only if the browser does not support audio.
-

Autoplay Audio

You can make audio **play automatically** when the page loads by using the `autoplay` attribute.

```
<audio controls autoplay>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Important:

Some browsers (like Chrome) do **not allow autoplay with sound**. But autoplay **with no sound** is allowed. You can do that by adding `muted`.

```
<audio controls autoplay muted>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Supported Audio Formats

HTML supports **three main audio file types**:

Format	File Extension	Media Type
MP3	.mp3	audio/mpeg
WAV	.wav	audio/wav
OGG	.ogg	audio/ogg

Browser Support

Here is wch browser supports wch audio formats:

Browser	MP3	WAV	OGG
Chrome	Yes	Yes	Yes
Firefox	Yes	Yes	Yes
Safari	Yes	Yes	No
Edge	Yes	Yes	Yes
Opera	Yes	Yes	Yes

Using JavaScript with `<audio>`

You can control the audio with JavaScript using special **methods** and **events**. For example:

```
var audio = document.getElementById("myAudio");
audio.play();    // Start playing
audio.pause();   // Pause the sound
audio.volume = 0.5; // Set volume (0.0 to 1.0)
```

You can also check when the audio **starts playing**, **pauses**, or **finishes**.

HTML Audio Tags

Tag	Description
<code><audio></code>	Used to play sound on a web page
<code><source></code>	Used to give multiple audio file options

HTML Plug-ins

Plug-ins in HTML

What are Plug-ins?

Plug-ins are small programs that help web browsers do extra tngs. They were used to:

- Run Java programs (Java Applets)
- Use Microsoft tools (ActiveX)
- Show Flash videos
- Display maps
- Scan for viruses
- Verify banking identity

Important:

Most modern web browsers **no longer support** plug-ins like Java, Flash, and ActiveX. These are outdated and not secure anymore.

```
<!DOCTYPE html>
<html>
<body>

<object data="audi.jpeg"></object>

</body>
</html>
```



<object> Element

The `<object>` element is used to **embed content** (like files, images, or HTML pages) into a webpage.

Example 1: Embed another HTML page

```
<object width="100%" height="500px" data="snippet.html"></object>
```

It shows another HTML page (called `snippet.html`) inside the current page.

Example 2: Embed an image

```
<object data="audi.jpeg"></object>
```

It shows an image on the page.

<embed> Element

The `<embed>` element is similar to `<object>`. It is also used to **embed content** like images, videos, or other files in a web page.

Example: Embed an image

```
<embed src="audi.jpeg">
```

It displays the image `audi.jpeg`.

Note: `<embed>` does **not have a closing tag** and it **cannot** contain fallback text if the file doesn't load.

Example: Embed another HTML page

```
<embed width="100%" height="500px" src="snippet.html">
```

It displays the content of `snippet.html` inside your web page.

Summary

Tag	Purpose
<code><object></code>	Embeds images, videos, or HTML files
<code><embed></code>	Also embeds files into the web page

HTML YouTube

Using YouTube Videos in HTML

The **easiest way to play videos** on your website is by using **YouTube**.

Why YouTube?

Converting videos into different file types (like MP4 or WebM) can be hard and take a lot of time.

Instead, you can upload your video to **YouTube** and let **YouTube handle the video playing** for you.

How to Show a YouTube Video on Your Web Page

Step-by-Step:

1. Upload your video to **YouTube**.
2. Note the **video ID** (a code like `tgBNymZ7vqY` found in the video URL).
3. Use an HTML `<iframe>` element to display the video.
4. Set the `src` (source) to the YouTube video link.
5. Add `width` and `height` to control the video size.

Example:

```
<iframe width="420" height="315"
```

```
src="https://www.youtube.com/embed/tgbNymZ7vqY">  
</iframe>
```

It will show the video in your webpage.

Autoplay and Mute Options

You can make the video start **automatically** when the page loads by adding:

- `autoplay=1` — starts video automatically
- `mute=1` — mutes the video (needed for autoplay to work in some browsers)

Example:

```
<iframe width="420" height="315"  
src="https://www.youtube.com/embed/tgbNymZ7vqY?autoplay=1&mute=1">  
</iframe>
```

Note: Most modern browsers only allow **muted videos** to autoplay.

Loop the Video

You can make the video play **again and again** using:

- `loop=1` — loops the video
- `playlist=videoID` — required for looping to work

Example:

```
<iframe width="420" height="315"  
src="https://www.youtube.com/embed/tgbNymZ7vqY?playlist=tgbNymZ7vqY&lo  
op=1">  
</iframe>
```

Hide the Video Controls

By default, YouTube videos show controls like play, pause, and volume.

You can **hide** the controls by adding:

- `controls=0` — hides controls
- `controls=1` — shows controls (default)

Example (no controls):

```
<iframe width="420" height="315"  
src="https://www.youtube.com/embed/tgbNymZ7vqY?controls=0">  
</iframe>
```

Summary of URL Options

Option	What It Does
autoplay=1	Starts video automatically
mute=1	Mutes the video
loop=1	Plays the video in a loop
controls=0	Hides the play/pause/volume controls
playlist=ID	Needed for looping

HTML APIs

What is a Web API?

A **Web API** is a tool that makes it easier for developers to build web applications. It helps you:

- Add extra features to your website
 - Perform complex tasks more easily
 - Use simple commands instead of long code
-

What Does API Mean?

API stands for **Application Programming Interface**.

It is a way for one program to **talk to** another program or service and use its features.

A **Web API** is an API that works over the **internet** — for websites.

Built-in HTML APIs (from the browser)

Modern web browsers come with many built-in APIs that you can use in your code.

Here are some popular ones:

1. Geolocation API

- Used to get the **current location** of the user (like latitude and longitude).

- Useful for maps, delivery services, and location-based apps.
-

2. Drag and Drop API

- Lets users **drag and drop** items on a web page.
 - Good for file uploads or rearranging elements.
-

3. Web Storage API

- Stores data **inside the browser**.
 - Works like cookies, but easier and with more space.
 - Example: Remembering a user's preferences or login info.
-

4. Web Workers API

- Allows JavaScript to run **in the background**.
 - Ts keeps your page fast and smooth wle doing heavy tasks.
-

5. Server-Sent Events (SSE) API

- Lets the web page get **automatic updates from the server**.

- Useful for live news feeds, stock updates, etc.
-

6. Canvas API

- Lets you **draw graphics** (like shapes, images, games) using JavaScript.
-

Things to Remember When Using APIs

1. **Check Browser Support**
Not all browsers support all APIs. Always check and add fallback options if needed.
 2. **Add Error Handling**
Write code that handles errors, so your website doesn't break if something goes wrong.
 3. **Ask for Permission**
Some APIs, like the Geolocation API, ask for the user's permission before accessing sensitive data.
-

Third-Party APIs

Some APIs are not built into the browser. These are called **third-party APIs**.

To use them, you usually need to:

- **Include a script or connect to their website**
- **Use their documentation** to know how to use their features

Examples:

- **YouTube API** – Show YouTube videos on your site
- **Twitter API** – Display Tweets
- **Facebook API** – Show Facebook posts or user info

HTML Geolocation

Geolocation API

The **Geolocation API** is used to **get the current location of the user** (like latitude and longitude).

For example, it can help show where the user is on a map or give directions.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Geolocation API</h2>
<p>Click the button to get your coordinates.</p>

<button onclick="getLocation()">Try It</button>

<p id="demo"></p>

<script>
const x = document.getElementById("demo");

function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition);
  } else {
    x.innerHTML = "Geolocation is not supported by this browser.";
  }
}

function showPosition(position) {
  x.innerHTML = "Latitude: " + position.coords.latitude +
    "<br>Longitude: " + position.coords.longitude;
}
</script>

</body>
</html>
```

JavaScript Geolocation API

Click the button to get your coordinates.

Try It

Important Points:

- The browser will **ask the user for permission** before accessing their location.
- It only works over **secure connections** (like <https://>).
- It works best on **devices with GPS**, like smartphones.

Browser Support

Most modern browsers support the Geolocation API (like Chrome, Firefox, Safari, etc.).

How to Use the Geolocation API

You use this line to access the location:

```
javascript
navigator.geolocation.getCurrentPosition()
```

Here's a simple example:

Example

```
<p id="demo"></p>

<button onclick="getLocation()">Get My Location</button>

<script>
const x = document.getElementById("demo");

function getLocation() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(showPosition, showError);
  } else {
    x.innerHTML = "Geolocation is not supported by this browser.";
  }
}

function showPosition(position) {
  x.innerHTML = "Latitude: " + position.coords.latitude +
    "<br>Longitude: " + position.coords.longitude;
```

```
}  
  
function showError() {  
    x.innerHTML = "Unable to get your location.";  
}  
</script>
```

What's Happening Here?

1. **Check** if the browser supports Geolocation.
 2. If yes, call `getCurrentPosition()`.
 3. If successful, the `showPosition()` function shows the **latitude and longitude**.
 4. If there's an error, the `showError()` function displays a message.
-

Better Error Messages

You can improve the error function like ts:

Javascript:

```
function showError(error) {  
    switch(error.code) {  
        case error.PERMISSION_DENIED:  
            x.innerHTML = "User did not allow location access.";  
            break;  
        case error.POSITION_UNAVAILABLE:  
            x.innerHTML = "Location information is not available.";  
            break;
```



```
case error.TIMEOUT:
    x.innerHTML = "Request to get location timed out.";
    break;
case error.UNKNOWN_ERROR:
    x.innerHTML = "An unknown error occurred.";
    break;
}
}
```

When to Use Geolocation

You can use Geolocation for:

- Showing user's current location on a map
- Finding nearby restaurants, stores, etc.
- Giving turn-by-turn directions
- Local news or weather based on location

What Data Can You Get?

When the location is found, you get an object with properties like:

Property	What It Tells You
<code>coords.latitude</code>	User's latitude
<code>coords.longitude</code>	User's longitude
<code>coords.accuracy</code>	Accuracy of the result
<code>coords.altitude</code>	Height above sea level (if available)
<code>coords.speed</code>	Speed of movement (if available)
<code>timestamp</code>	Time when location was recorded

Track the User's Movement

You can keep tracking the location as the user moves using `watchPosition()` instead of `getCurrentPosition()`.

Example:

Javascript:

```
navigator.geolocation.watchPosition(showPosition, showError);
```

It is useful for apps like navigation or delivery tracking.

You can stop tracking using:

Javascript:



```
navigator.geolocation.clearWatch(watchID);
```

HTML Drag and Drop

HTML Drag and Drop API

What is Drag and Drop?

Drag and drop is a feature that lets you **move things around on a web page** using your mouse. For example, you can **grab an image** and **drop it into a box**.

```
<!DOCTYPE HTML>
<html>
<head>
<style>
#div1 {
width: 350px;
height: 70px;
padding: 10px;
border: 1px solid #aaaaaa;
}
</style>
<script>
function dragstartHandler(ev) {
ev.dataTransfer.setData("text", ev.target.id);
}

function dragoverHandler(ev) {
ev.preventDefault();
}

function dropHandler(ev) {
ev.preventDefault();
const data = ev.dataTransfer.getData("text");
ev.target.appendChild(document.getElementById(data));
}
</script>
</head>
<body>
```

Drag and Drop API

Drag the W3Schools image into the rectangle:



Browser Support

Most modern browsers like Chrome, Firefox, Edge, and Safari support drag and drop.

Basic Example

Here's a simple example where you can drag an image and drop it into a box:

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function dragstartHandler(ev) {
      ev.dataTransfer.setData("text", ev.target.id);
    }

    function dragoverHandler(ev) {
      ev.preventDefault(); // Ts allows the drop
    }

    function dropHandler(ev) {
      ev.preventDefault();
      const data = ev.dataTransfer.getData("text");
      ev.target.appendChild(document.getElementById(data));
    }
  </script>
  <style>
    #div1 {
      width: 400px;
      height: 150px;
      border: 2px dashed gray;
      margin-bottom: 10px;
    }
  </style>
</head>
<body>

<div id="div1" ondrop="dropHandler(event)"
ondragover="dragoverHandler(event)">
```

```
<!-- Drop items here -->
</div>



</body>
</html>
```

How It Works (Step by Step)

1. Make an Element Draggable

Add `draggable="true"` to make an element draggable.

```
<img draggable="true">
<p draggable="true">Drag me</p>
```

2. Start Dragging - `ondragstart`

When you begin dragging, the `ondragstart` event happens. You use `setData()` to store the data (usually the element's ID):

Javascript:

```
function dragstartHandler(ev) {
  ev.dataTransfer.setData("text", ev.target.id);
}
```

3. Allow Dropping - `ondragover`

By default, you **can't drop items** in most elements. To allow it, you must use `preventDefault()` in the `ondragover` event:

Javascript:

```
function dragoverHandler(ev) {  
  ev.preventDefault();  
}
```

4. Drop the Element - `ondrop`

When the item is dropped, the `ondrop` event runs. You:

- Get the dragged element's ID
- Add (append) it to the drop target

Javascript:

```
function dropHandler(ev) {  
  ev.preventDefault();  
  const data = ev.dataTransfer.getData("text");  
  ev.target.appendChild(document.getElementById(data));  
}
```

More Examples

Drag `<h1>` to a `<div>`


```
<div id="div1" ondrop="dropHandler(event)"  
ondragover="dragoverHandler(event)"></div>
```

```
<h1 id="heading1" draggable="true"  
ondragstart="dragstartHandler(event)">  
  Drag ts Heading  
</h1>
```

Drag a Link to a **<div>**

```
<div id="div1" ondrop="dropHandler(event)"  
ondragover="dragoverHandler(event)"></div>
```

```
<a id="link1" href="https://w3schools.com" draggable="true"  
ondragstart="dragstartHandler(event)">  
  W3Schools.com  
</a>
```

Drag Image Between Two Divs

You can add two drop areas like ts:

```
<div id="div1" ondrop="dropHandler(event)"  
ondragover="dragoverHandler(event)"></div>  
<div id="div2" ondrop="dropHandler(event)"  
ondragover="dragoverHandler(event)"></div>
```

```

```

Now you can move the image back and forth between the two boxes.

Summary

- Use `draggable="true"` to make an element draggable.
- Use `ondragstart` to define what to drag.
- Use `ondragover` with `preventDefault()` to allow dropping.
- Use `ondrop` to place the dragged item where it is dropped.

HTML Web Storage

HTML Web Storage API (Easy Explanation)

What is Web Storage?

Web Storage lets websites **save data in your browser** — without sending it to the server every time.

Before HTML5, we had to use **cookies** to store data, but cookies:

- Could only store a small amount of data.
- Sent data to the server every time (slower and less secure).

With **Web Storage**:

```
<!DOCTYPE html>
<html>
<body>

<h1>Browser test for Web Storage API</h1>

<div id="result"></div>

<script>
const x = document.getElementById("result");
if (typeof(Storage) !== "undefined") {
  x.innerHTML = "Your browser supports Web storage!";
} else {
  x.innerHTML = "Sorry, no Web storage support!";
}
</script>

</body>
</html>
```

Browser test for Web Storage API

Your browser supports Web storage!

- You can store **more data** (at least 5MB).
 - Data is **saved only in the browser** (not sent to server).
 - Data stays **private** to the website.
-

Two Types of Web Storage

1. `localStorage`

- Saves data **without an expiry**.
- Data is saved **even after you close the browser tab**.
- Good for long-term storage.

2. `sessionStorage`

- Saves data **only while the tab is open**.
 - Data is **cleared when you close the tab**.
 - Good for temporary use.
-

Browser Support

Most modern browsers (Chrome, Firefox, Safari, Edge, etc.) support both `localStorage` and `sessionStorage`.

Check if Browser Supports Web Storage

```
<script>
const x = document.getElementById("result");
if (typeof(Storage) !== "undefined") {
  x.innerHTML = "Your browser supports Web Storage!";
} else {
  x.innerHTML = "Sorry, your browser does not support Web Storage.";
}
```

```
}  
</script>
```

How to Use `localStorage`

```
<script>  
const x = document.getElementById("result");  
  
if (typeof(Storage) !== "undefined") {  
  // Save data  
  localStorage.setItem("lastname", "Smith");  
  localStorage.setItem("bgcolor", "yellow");  
  
  // Get data  
  x.innerHTML = localStorage.getItem("lastname");  
  x.style.backgroundColor = localStorage.getItem("bgcolor");  
} else {  
  x.innerHTML = "Web Storage is not supported.";  
}  
</script>
```

What's Happening:

- `setItem()` saves data.
- `getItem()` gets data.
- Data is saved even after you close the browser.
- Data is stored as **text**, so convert if needed.

To remove a saved item:

Javascript:

```
localStorage.removeItem("lastname");
```

Count Button Clicks with **localStorage**

It counts how many times a button has been clicked (and remembers it next time too):

```
<script>
function clickCounter() {
  const x = document.getElementById("result");
  if (typeof(Storage) !== "undefined") {
    if (localStorage.clickcount) {
      localStorage.clickcount = Number(localStorage.clickcount) + 1;
    } else {
      localStorage.clickcount = 1;
    }
    x.innerHTML = "You clicked the button " + localStorage.clickcount
+ " time(s).";
  } else {
    x.innerHTML = "Web Storage is not supported.";
  }
}
</script>
```

Count Button Clicks with **sessionStorage**

It works like the example above, **but only for the current session** (while the tab is open):

```
<script>
function clickCounter() {
```

```
const x = document.getElementById("result");
if (typeof(Storage) !== "undefined") {
  if (sessionStorage.clickcount) {
    sessionStorage.clickcount = Number(sessionStorage.clickcount) +
1;
  } else {
    sessionStorage.clickcount = 1;
  }
  x.innerHTML = "You clicked the button " +
sessionStorage.clickcount + " time(s) in ts session.";
} else {
  x.innerHTML = "Web Storage is not supported.";
}
}</script>
```

Summary

Feature	localStorage	sessionStorage
Saves permanently	Yes	No (only until tab is closed)
Size limit	Around 5MB	Around 5MB
Shared between tabs	Yes	No

HTML Web Workers

Web Workers in HTML (Simple Explanation)

What is a Web Worker?

When you run JavaScript code on a web page, sometimes the page becomes **slow or stuck** if the script takes too long.

A **Web Worker** is a **separate JavaScript file** that runs **in the background** — it doesn't block the page. That means:

- You can still click, scroll, and use the page.
- The heavy or long-running code runs quietly in the background.

Why use Web Workers?

They are helpful when:

- You have a big task that takes time (like calculations, loops).
- You don't want the page to freeze.

Browser Support

Most modern browsers (Chrome, Firefox, Edge, Safari, Opera) support Web Workers.

How to Check Web Worker Support

Before using a Web Worker, you can check if the browser supports it:

```
<script>
const x = document.getElementById("result");
if (typeof(Worker) !== "undefined") {
  x.innerHTML = "Your browser supports Web Workers!";
} else {
  x.innerHTML = "Sorry, your browser does not support Web Workers.";
}
</script>
```

Step-by-Step: Using a Web Worker

1. Create a Worker File

Create a new JavaScript file (example: `worker.js`) with ts code:

```
let i = 0;

function timedCount() {
  i++;
  postMessage(i); // send message to HTML page
  setTimeout(timedCount, 500); // call itself every 0.5 sec
}

timedCount();
```

Here, `postMessage()` sends the number back to the HTML page.

2. Use the Worker in Your HTML

```
<!DOCTYPE html>
<html>
<body>

<p>Counting: <output id="result"></output></p>
<button onclick="startWorker()">Start</button>
<button onclick="stopWorker()">Stop</button>

<script>
let w;

function startWorker() {
  const x = document.getElementById("result");

  if (typeof(Worker) !== "undefined") {
    if (!w) {
      w = new Worker("worker.js"); // link to the worker file
    }

    w.onmessage = function(event) {
      x.innerHTML = event.data; // receive and display data
    };

  } else {
    x.innerHTML = "Web Workers are not supported in your browser.";
  }
}

function stopWorker() {
  w.terminate(); // stop the worker
  w = undefined; // reset the variable
}
</script>
```

```
</body>  
</html>
```

Summary of What's Happening

Step	What it Does
<code>new Worker()</code>	Starts the worker from an external JS file
<code>postMessage()</code>	Sends message from the worker to the main page
<code>onmessage</code>	Receives message from the worker
<code>terminate()</code>	Stops the worker

Limitations of Web Workers

Inside the worker file:

- You **cannot use** things like `document`, `window`, or `alert`.
 - It runs separately from the main page — just for background tasks.
-

When Should You Use Web Workers?

Use them when:

- You are doing something that takes a lot of time (like calculations, loops, or data processing).
- You don't want the web page to freeze while the work is done.

HTML SSE

Server-Sent Events (SSE) in HTML

What are Server-Sent Events?

Server-Sent Events (SSE) allow a server to **automatically send updates** to a web page over the internet.

Usually, web pages need to ask the server for new data. But with SSE, the server **pushes updates** to the browser without being asked.

Examples:

- Facebook or Twitter updates
 - News headlines
 - Live sports scores
 - Stock market prices
-

Browser Support

Most modern browsers support SSE:

- Chrome
- Firefox
- Edge
- Safari

- Opera

How It Works

To use Server-Sent Events, you use the **EventSource** object in JavaScript. It connects to the server and waits for messages.

Example:

```
<script>
const x = document.getElementById("result");

// Check if the browser supports SSE
if (typeof(EventSource) !== "undefined") {
  const source = new EventSource("demo_sse.php");

  source.onmessage = function(event) {
    x.innerHTML += event.data + "<br>";
  };
} else {
  x.innerHTML = "Sorry, your browser does not support Server-Sent
Events.";
}
</script>
```

What Ts Code Does:

- Checks if SSE is supported in the browser.
- Connects to `demo_sse.php`, a server file that sends updates.

- Shows new messages inside the element with `id="result"` whenever the server sends them.

Server Code Example (PHP)

The server must send data in a special format. Here's a basic example using **PHP**:

demo_sse.php:

```
<?php
header('Content-Type: text/event-stream');
header('Cache-Control: no-cache');

$time = date('r');
echo "data: The server time is: {$time}\n\n";
flush();
?>
```

Explanation:

- `Content-Type` is set to `text/event-stream` to tell the browser it's SSE data.
 - `Cache-Control: no-cache` tells the browser not to cache the response.
 - Each message starts with `data:`, then the message.
 - `flush()` sends the data to the browser immediately.
-

Server Code Example (ASP Classic)

If you're using **ASP**:

```
<%  
Response.ContentType = "text/event-stream"  
Response.Expires = -1  
Response.Write("data: The server time is: " & Now())  
Response.Flush()  
%>
```

Events Available in EventSource

Event	What It Does
<code>onopen</code>	Runs when the connection opens
<code>onmessage</code>	Runs when a message is received from server
<code>onerror</code>	Runs when there is a connection error

Summary

- **SSE** is used to **automatically send updates** from the server to the browser.
- You use **EventSource** on the client (browser) side.



- The server sends **simple text data** using the **text/event-stream** format.
- Useful for live updates like news, scores, and social feeds.